

INFORME TECNICO DE LOS BOLETINES DE APRENDIZAJE FEDERADO



Realizado por
Ana Gil Molina
Levi Malest Villarreal
José María García Ortiz

Abstract

The goal of this work is to carry out several experiments of different nature within the *Federated Learning* branch. They will be distributed along five tasks, each of them associated to a single chapter of this document.

Índice general

1	Boletín I	1
1.1.	Introducción a la tarea y ejercicios a resolver	1
1.2.	Preguntas a responder	1
1.3.	Desarrollo de la tarea	2
1.3.1.	Comparaciones del impacto de la distribución de los datos (IID vs non-IID)	3
1.3.2.	Comparaciones del impacto de la arquitectura del modelo (CNN vs simplemobilenet)	14
1.3.3.	Comparaciones del uso de recursos (CPU y red)	17
2	Boletín II	29
2.1.	Introducción a la tarea y ejercicios a resolver	29
2.2.	Preguntas a responder	29
2.3.	Desarrollo de la tarea	30
2.3.1.	Comparaciones del impacto de la distribución de los datos (IID vs non-IID) bajo ataques	31
2.3.2.	Comparaciones del efecto acumulativo de los ataques	37
2.3.3.	Contramedidas para detectar y mitigar los ataques	42
3	Boletín III	46
3.1.	Introducción a la tarea y ejercicios a resolver	46
3.2.	Preguntas a responder	46
3.3.	Desarrollo de la tarea	47
3.3.1.	Análisis de los escenarios	48
3.3.2.	Respuestas a las preguntas	58
4	Boletín IV	67
4.1.	Introducción a la tarea y ejercicios a resolver	67
4.2.	Preguntas a responder	67
4.3.	Desarrollo de la tarea	68
4.3.1.	Comparaciones entre configuración de reputación estática y dinámica	68
4.3.2.	Respuestas a las preguntas	78
5	Boletín V	81
5.1.	Introducción a la tarea y ejercicios a resolver	81
5.2.	Preguntas a responder	81
5.3.	Desarrollo de la tarea	82

1. Boletín I

1.1. Introducción a la tarea y ejercicios a resolver

Tanto para este boletín como para todos los restantes haremos uso de la plataforma **NEBULA** (<https://nebula.enriquetomasmb.com/>) para desarrollar los experimentos de *Aprendizaje Federado* (FL) que requiera cada boletín.

Para esta primera tarea, debemos realizar los siguientes **ejercicios**:

1. Seleccionar un dataset diferente (como CIFAR-10) y otra arquitectura de modelo (como CNN) para entrenar un modelo de FL empleando CFL y DFL.
 - Rondas de entrenamiento en la federación: 10
 - Épocas de entrenamiento por ronda: 1
 - Método de agregación: FedAvg
 - Nodos en la federación: 4, con parámetros por defecto

Y entrenar los modelos usando datos **IID** y **non-IID** (Dirichlet 0.1).

Con lo que se pretende conseguir:

- Realizar la instalación de NEBULA y comprender su funcionamiento.
- Aprender a entrenar modelos de FL con arquitecturas **CFL** (*Centralized Federated Learning*) y **DFL** (*Decentralized Federated Learning*).
- Explorar el impacto que tienen las distribuciones IID VS non-IID de los datos en el entrenamiento de los modelos de FL.

1.2. Preguntas a responder

Además, en cada tarea encontramos algunas **preguntas** que se sugieren para ser respondidas tras llevar a cabo los ejercicios propuestos. En este caso, son:

1. ¿Qué observaciones consideras importantes sobre el impacto de usar una distribución IID o non-IID en los datos, respecto del rendimiento del modelo de FL entrenado? ¿Cuáles son sus matrices de confusión en las rondas 5 y 10?
2. ¿Cómo afecta la arquitectura del modelo a la efectividad del CFL y del DFL?
3. ¿Hay alguna diferencia en el uso de CPU o de la red?

1.3. Desarrollo de la tarea

EJERCICIO 1 - Selección de dataset, arquitectura, y características de la federación.

Para este ejercicio podemos escoger diferentes combinaciones de parámetros en el entrenamiento federado, variando el tipo de federación (centralizada o descentralizada), la distribución de los datos (IID o non-IID) y la arquitectura del modelo (CNN o SimpleMobileNet). Para facilitar la comparación entre estos factores sin incurrir en repeticiones innecesarias, organizaremos el análisis por dimensiones clave: impacto de la distribución de datos, efecto de la arquitectura del modelo y diferencias entre enfoques de federación.

Conviene destacar aquí que para documentar la resolución de ambos ejercicios, nos serviremos principalmente de pantallazos de la GUI de NEBULA y de las gráficas que proporciona la propia aplicación. También, que algunas características de la federación como las rondas de entrenamiento (10), las épocas de entrenamiento (1), el método de agregación (FedAvg) y el número de nodos en la federación (4 con **topología de estrella en CFL** y **topología fully connected en DFL** por defecto); estarán fijas por defecto para este ejercicio, aunque algunas de ellas pueden ser modificadas para realizar pruebas extra.

Los **escenarios de federación** que entrenaremos en NEBULA para esta tarea son los siguientes:

Escenario B1.1:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Centralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg

Escenario B1.2:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Centralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg

Escenario B1.3:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg

Escenario B1.4:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg

Escenario B1.5:

- Dataset: CIFAR-10
- Arquitectura de red: simplemobilenet
- Tipo de federación: Centralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg

Escenario B1.6:

- Dataset: CIFAR-10
- Arquitectura de red: simplemobilenet
- Tipo de federación: Centralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg

Escenario B1.7:

- Dataset: CIFAR-10
- Arquitectura de red: simplemobilenet
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg

Escenario B1.8:

- Dataset: CIFAR-10
- Arquitectura de red: simplemobilenet
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg

1.3.1. Comparaciones del impacto de la distribución de los datos (IID vs non-IID)

En este apartado se analiza cómo influye la distribución de los datos (IID vs. non-IID) en el rendimiento del aprendizaje federado. Se comparan pares de escenarios con los mismos parámetros de arquitectura, tipo de federación y algoritmo de agregación, variando únicamente la distribución de los datos. Para ello, se presentan métricas clave como el accuracy y el F1 score global en el conjunto de test, el accuracy en el train, así como matrices de confusión en rondas específicas para ilustrar los efectos en la clasificación por clase. De esta forma, este apartado nos permitirá responder a la pregunta:

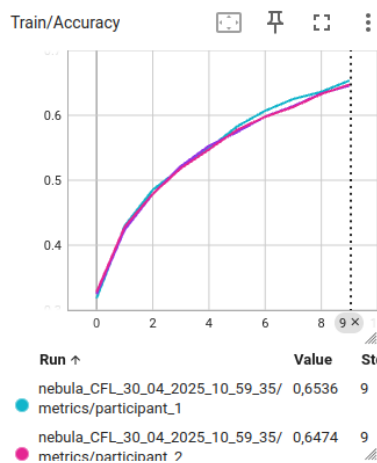
PREGUNTA 1

¿Qué observaciones consideras importantes sobre el impacto de usar una distribución IID o non-IID en los datos, respecto del rendimiento del modelo de FL entrenado?

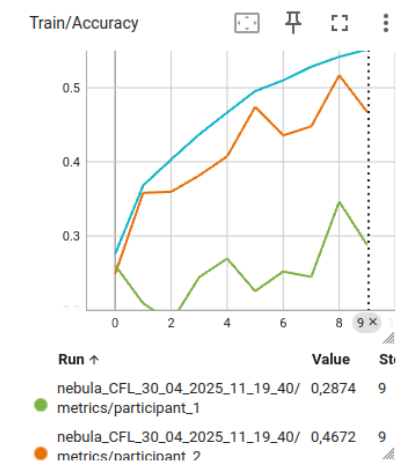
¿Cuáles son sus matrices de confusión en las rondas 5 y 10?

RESPUESTA:

COMPARACIÓN CON CNN Y CFL (B1.1 vs B1.2)



Escenario B1.1 (IID)



Escenario B1.2 (non-IID)

Figura 1.1: Comparación de Train Accuracy entre B1.1 y B1.2

En la **Figura 1.1** se compara el accuracy medido en el conjunto de train durante los entrenamientos locales en los escenarios B1.1 (IID) y B1.2 (non-IID). Se observa que en el escenario B1.1, todos los nodos muestran un crecimiento uniforme y progresivo del accuracy, con valores cercanos a 0,65 en la última ronda. Por otro lado, en el escenario B1.2, solo uno de los nodos alcanza un rendimiento aceptable, aunque inferior al de los nodos de B1.1, mientras que los otros dos nodos muestran grandes fluctuaciones a lo largo de las rondas. Especialmente se destaca por su mal rendimiento el nodo verde, que finaliza con un accuracy de 0,2874 en la última ronda. Esta diferencia se debe a la distribución heterogénea de los datos en B1.2, que causa que algunos nodos no tengan una representación adecuada de las clases, dificultando el aprendizaje.

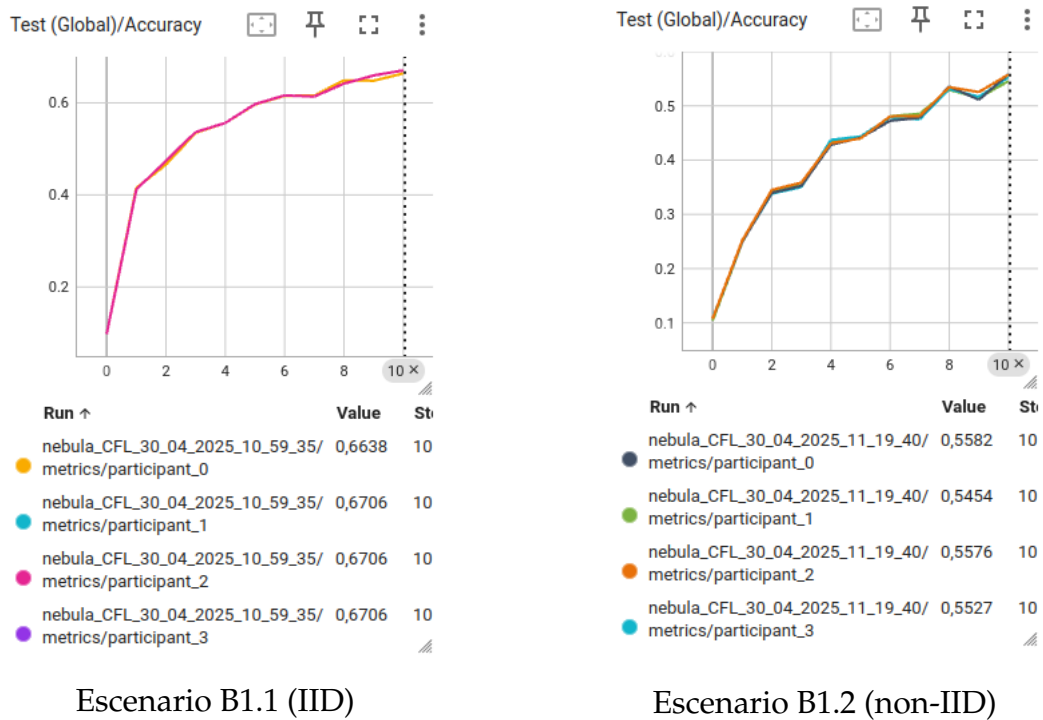
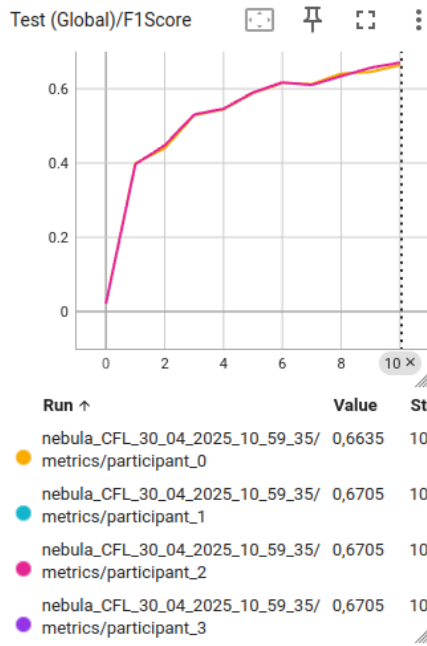
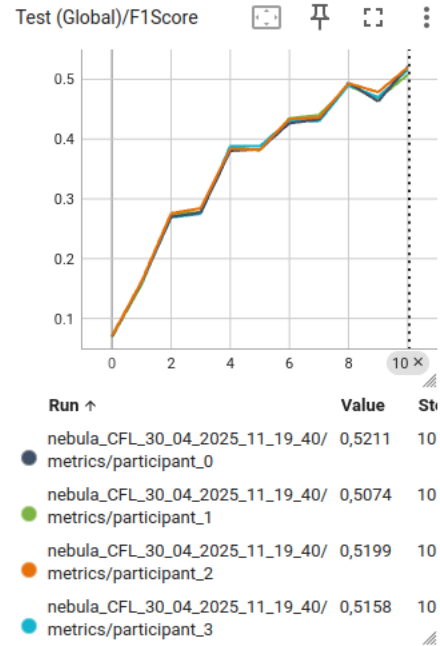


Figura 1.2: Comparación de Test Accuracy Global entre B1.1 y B1.2

En la **Figura 1.2** se muestra la evolución del accuracy global en el conjunto de test para los escenarios B1.1 (IID) y B1.2 (non-IID). De nuevo, se observa un rendimiento inferior en B1.2, donde la convergencia es más lenta y presenta ligeras oscilaciones a lo largo de las rondas. Al finalizar, los nodos en B1.2 alcanzan un accuracy cercano a 0,55, mientras que en B1.1 se observa una convergencia más rápida y estable, y un valor final superior, en torno a 0,67.



Escenario B1.1 (IID)



Escenario B1.2 (non-IID)

Figura 1.3: Comparación de Test F1 Score Global entre B1.1 y B1.2

De manera similar a lo observado con el accuracy, en la **Figura 1.3** se observa que en el escenario B1.2, el F1 Score en el conjunto de test tiene una convergencia lenta y con oscilaciones, alcanzando valores alrededor de 0,51 en la última ronda. En contraste, en B1.1, el F1 Score converge de forma más rápida y estable, alcanzando valores significativamente superiores, en torno a 0,67.

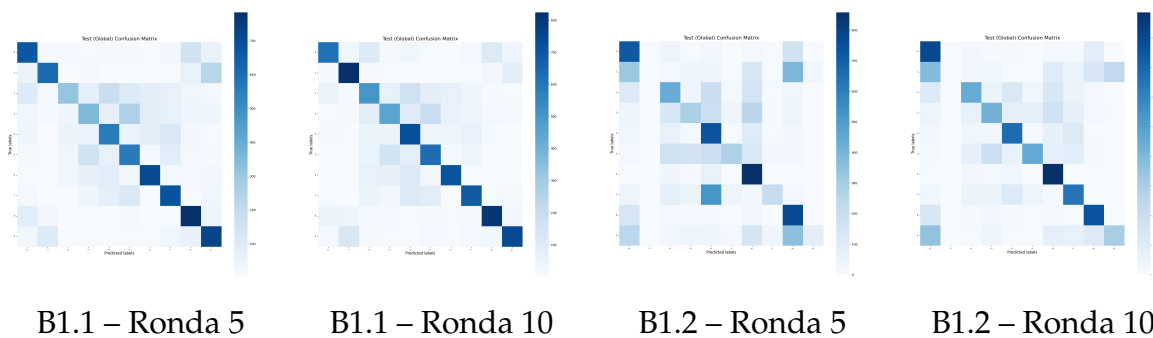


Figura 1.4: Comparación de matrices de confusión entre B1.1 (IID) y B1.2 (non-IID) en rondas 5 y 10.

Finalmente, en la **Figura 1.4** se presentan las matrices de confusión correspondientes a las rondas 5 y 10 de los escenarios B1.1 (IID) y B1.2 (non-IID). En la ronda 5 del escenario B1.1, se observa en general un buen rendimiento del modelo, con ligeras dificultades en las clases 2 y 3, que muestran un mayor número de clasificaciones incorrectas. No obstante, en la ronda 10, el desempeño mejora significativamente, y el modelo predice correctamente la mayoría de las clases. Sin

embargo, la matriz de confusión en la ronda 5 de B1.2 muestra un desempeño deficiente, con muchos falsos positivos y falsos negativos. Aunque en la ronda 10 se observa una ligera mejora, sigue habiendo errores de clasificación, especialmente en la clase 1 y, en menor medida, la clase 9.

En resumen, para los escenarios con arquitectura de red CNN y federación centralizada, se observa un rendimiento inferior en todas las métricas analizadas cuando se emplea una distribución de datos non-IID. Esto reafirma que una distribución no homogénea dificulta el proceso de aprendizaje, especialmente en aquellos nodos que no tienen una representación equilibrada de las clases.

COMPARACIÓN CON CNN Y DFL (B1.3 vs B1.4)

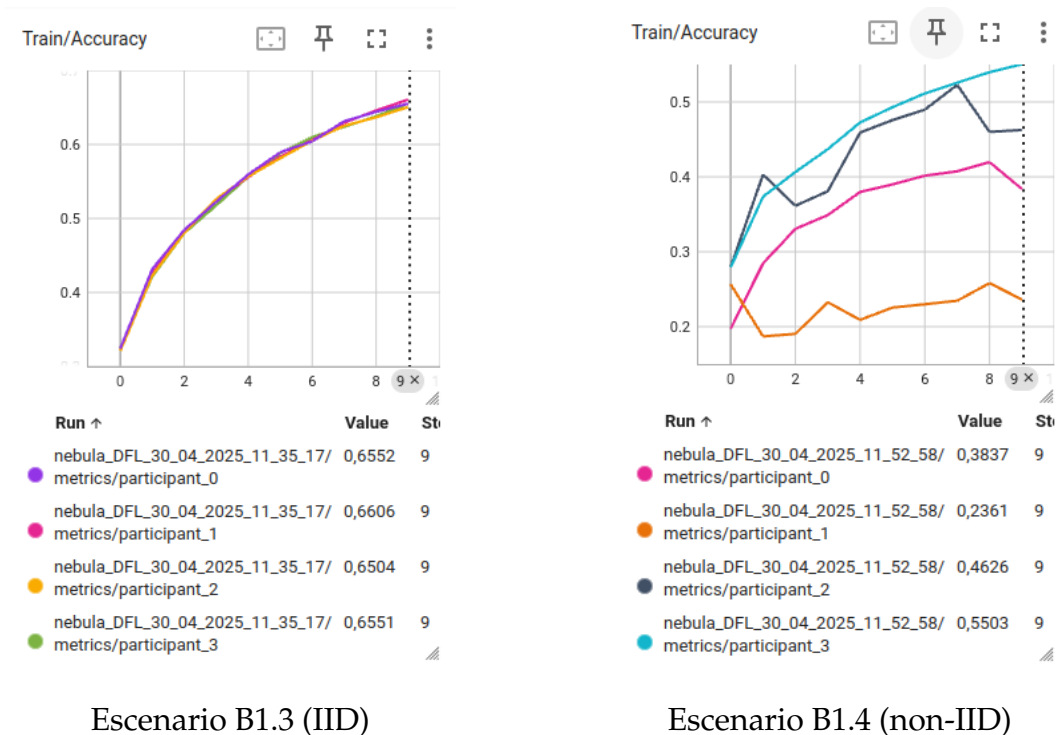
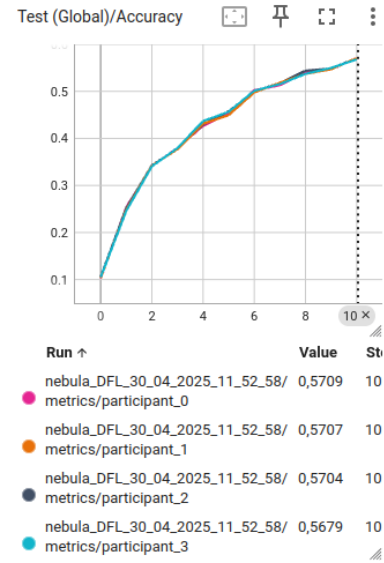


Figura 1.5: Comparación de Train Accuracy entre B1.3 y B1.4

En la **Figura 1.5** se compara el accuracy en el conjunto de train entre los escenarios B1.3 (IID) y B1.4 (non-IID). En el escenario B1.3 todos los nodos muestran un crecimiento uniforme y progresivo del accuracy, con valores cercanos a 0,65 en la última ronda. Por el contrario, en el escenario B1.4, solo un nodo muestra un rendimiento aceptable, aunque inferior que en B1.3, mientras que los otros tres nodos muestran grandes fluctuaciones a lo largo de las rondas. En especial, el nodo naranja a penas mejora su rendimiento a lo largo de las rondas, y finaliza con un accuracy de 0,2361 en la última ronda. De nuevo, la distribución heterogénea de los datos en B1.4 causa que algunos nodos tengan dificultades con el aprendizaje debido a la falta de una adecuada representación de las clases.



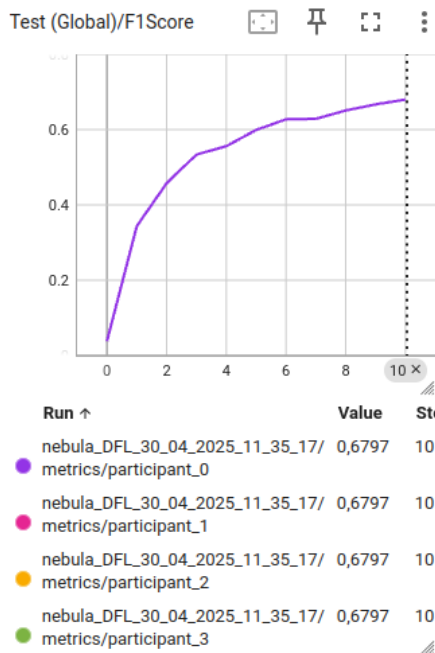
Escenario B1.3 (IID)



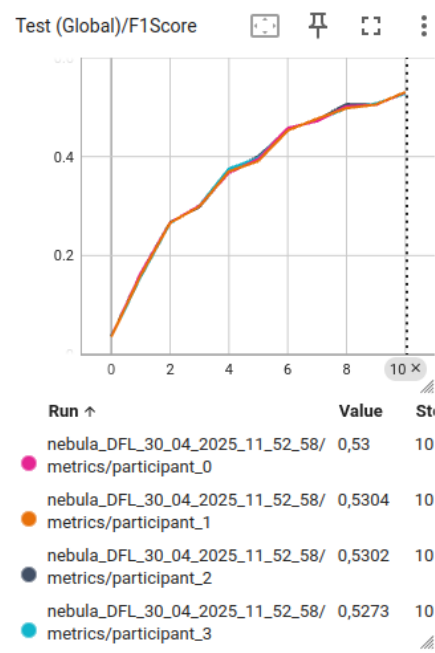
Escenario B1.4 (non-IID)

Figura 1.6: Comparación de Test Accuracy Global entre B1.3 y B1.4

En la **Figura 1.6** se muestra la evolución del accuracy global en el conjunto de test para los escenarios B1.3 (IID) y B1.4 (non-IID). El rendimiento es inferior en B1.4, pues la convergencia es más lenta y alcanza valores inferiores en la última ronda, en torno a 0,57, mientras que en B1.3 se observa una convergencia más rápida y estable, y un valor final superior, en torno a 0,68.



Escenario B1.3 (IID)



Escenario B1.4 (non-IID)

Figura 1.7: Comparación de Test F1 Score Global entre B1.3 y B1.4

En la **Figura 1.7** se muestra la evolución del F1 Score global en el conjunto de test para los escenarios B1.3 y B1.4. Se observa que en el escenario B1.4, el F1 Score tiene una convergencia algo lenta, alcanzando valores alrededor de 0,53 en la última ronda, mientras que en B1.3, el F1 Score converge de forma más rápida y estable, alcanzándose en todos los nodos un valor de 0,6797 en la última ronda.

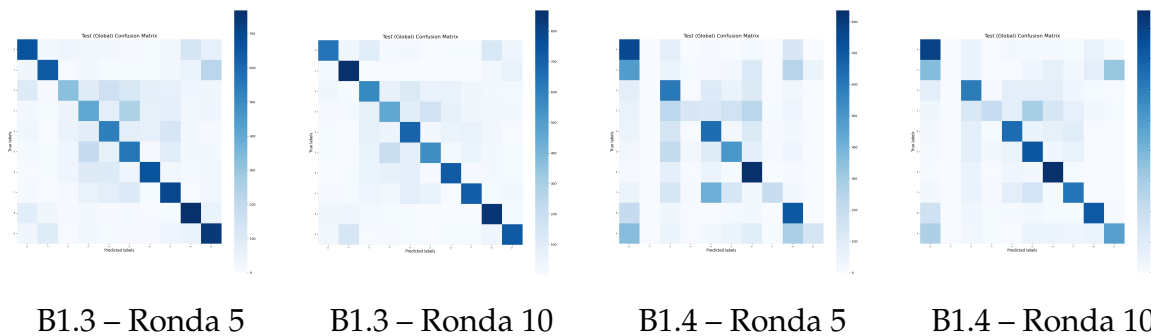


Figura 1.8: Comparación de matrices de confusión entre B1.3 (IID) y B1.4 (non-IID) en rondas 5 y 10.

Por último, en la **Figura 1.8** se presentan las matrices de confusión correspondientes a las rondas 5 y 10 de los escenarios B1.3 (IID) y B1.4 (non-IID). Para el escenario B1.3, se observa en general un buen rendimiento del modelo tanto en la ronda 5 como en la 10. No obstante, en el escenario B1.4, la matriz de confusión en la ronda 5 muestra un desempeño deficiente, pues muchas de las clases se predicen incorrectamente. Aunque en la ronda 10 se observa una ligera mejora, sigue habiendo errores de clasificación, especialmente en la clase 1 y, en menor medida, la clase 3.

En conclusión, para los escenarios con arquitectura de red CNN y federación descentralizada, de nuevo se observa un rendimiento inferior en todas las métricas analizadas al usar datos con distribución non-IID.

COMPARACIÓN CON SIMPLETEMOBILENET Y CFL (B1.5 VS B1.6)

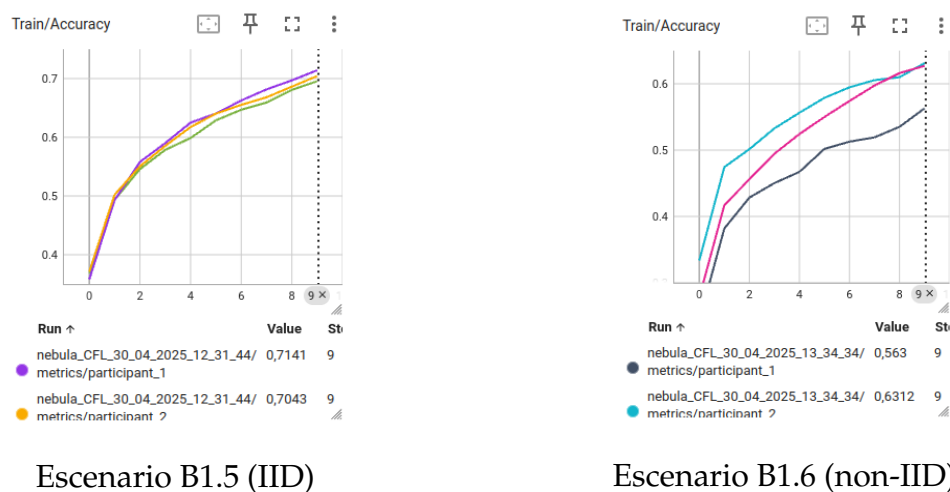


Figura 1.9: Comparación de Train Accuracy entre B1.5 y B1.6

En la **Figura 1.9** se compara el accuracy local en el conjunto de train entre los escenarios B1.5 (IID) y B1.6 (non-IID). En el escenario B1.5 todos los nodos muestran un crecimiento uniforme y progresivo del accuracy, con valores alrededor de 0,7 en la última ronda, aunque con ligeras variaciones entre los tres nodos. Por otro lado, en el escenario B1.6, el accuracy muestra un crecimiento más lento, y con mayores diferencias entre nodos. En particular, el nodo gris es el que peor rendimiento muestra, con el crecimiento más lento de los tres, y alcanzando un accuracy tan solo de 0,563 en la última ronda, mientras que los otros dos nodos alcanzan valores alrededor de 0,63. La distribución heterogénea de los datos en B1.6 causa que algunos nodos tengan dificultades con el aprendizaje por no tener una adecuada representación de las clases.

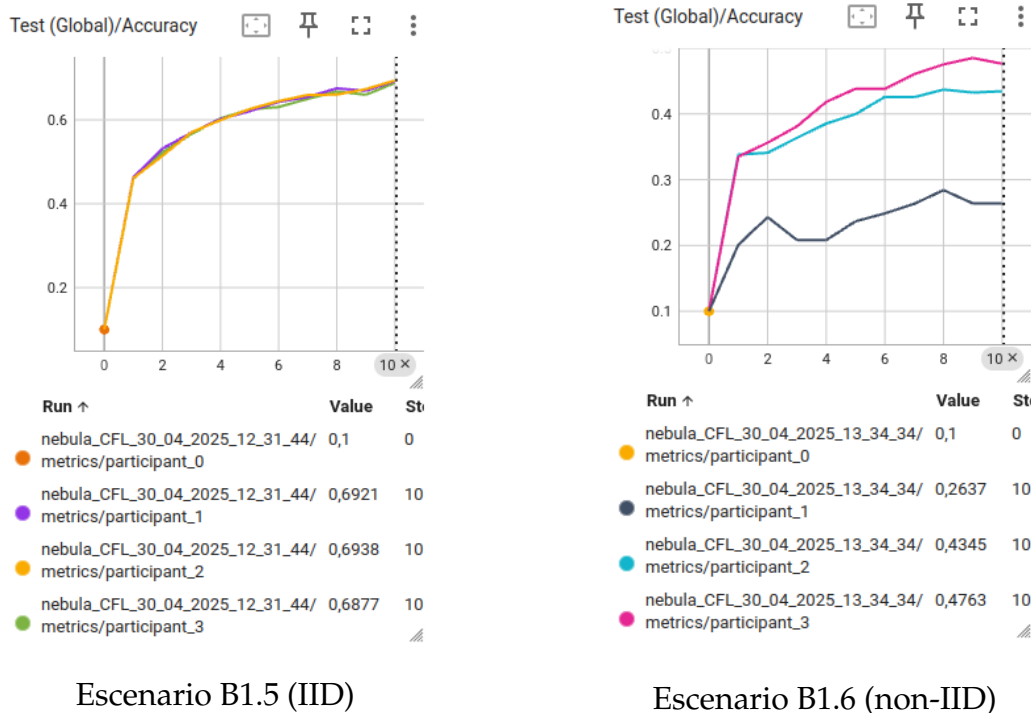
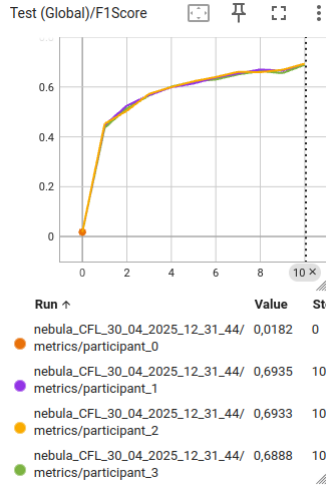
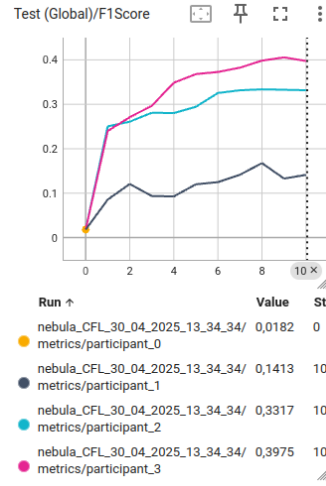


Figura 1.10: Comparación de Test Accuracy Global entre B1.5 y B1.6

En la **Figura 1.10** se muestra la evolución del accuracy global en el conjunto de test para los escenarios B1.5 (IID) y B1.6 (non-IID). El rendimiento en B1.6 es notablemente inferior, con una convergencia más lenta e irregular entre los distintos nodos. En particular, el nodo gris presenta los peores resultados, alcanzando un accuracy de 0,2637 en la última ronda. Aunque los otros nodos muestran un rendimiento algo mejor, sus valores siguen estando por debajo de los obtenidos en B1.5, en torno a 0,68.



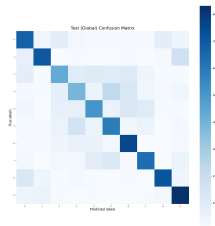
Escenario B1.5 (IID)



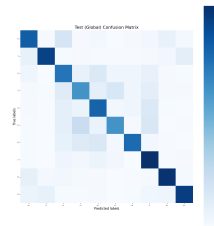
Escenario B1.6 (non-IID)

Figura 1.11: Comparación de Test F1 Score Global entre B1.5 y B1.6

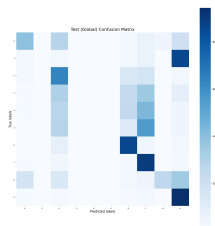
En la **Figura 1.11** se muestra la evolución del F1 Score global en test para los escenarios B1.5 y B1.6. Los resultados son muy similares a los obtenidos para el accuracy, siendo de nuevo B1.6 el escenario con peor rendimiento.



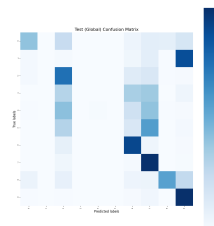
B1.5 – Ronda 5



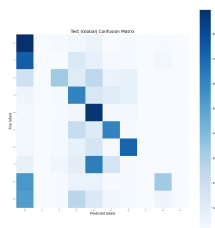
B1.5 – Ronda 10



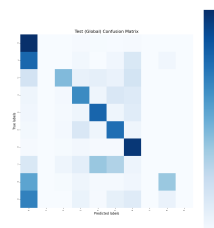
B1.6 Nodo A – Ronda 5



B1.6 Nodo A – Ronda 10



B1.6 Nodo B – Ronda 5



B1.6 Nodo B – Ronda 10

Figura 1.12: Comparación de matrices de confusión entre B1.5 (IID) y B1.6 (non-IID, nodos A y B) en rondas 5 y 10.

En la **Figura 1.12** se presentan las matrices de confusión correspondientes a las rondas 5 y 10 de los escenarios B1.5 (IID) y B1.6 (non-IID). En B1.6 se obtuvieron diferentes matrices dependiendo del nodo, por lo que se han incluido dos por cada ronda para representar esa variabilidad. En el escenario B1.5 se observa, en general, un buen desempeño del modelo, tanto en la ronda 5 como en la 10. No obstante, en el escenario B1.6, se observan diferencias notables entre nodo. En algunos, el desempeño es deficiente en ambas rondas, mientras que en otros se obtienen resultados ligeramente mejores, aunque con dificultades para predecir correctamente algunas de las clases, como la 1, la 7 y la 9, y con poca diferencia entre la ronda 5 y la 10.

En resumen, en escenarios con arquitectura *simplemobilenet* y federación centralizada, se siguen observando diferencias notables dependiendo del tipo de distribución de los datos. En particular, aquellos escenarios con datos non-IID muestran el peor desempeño, además, con diferencias significativas entre los nodos, incluso dentro del mismo escenario.

COMPARACIÓN CON SIMPLEMOBILENET Y DFL (B1.7 vs B1.8)

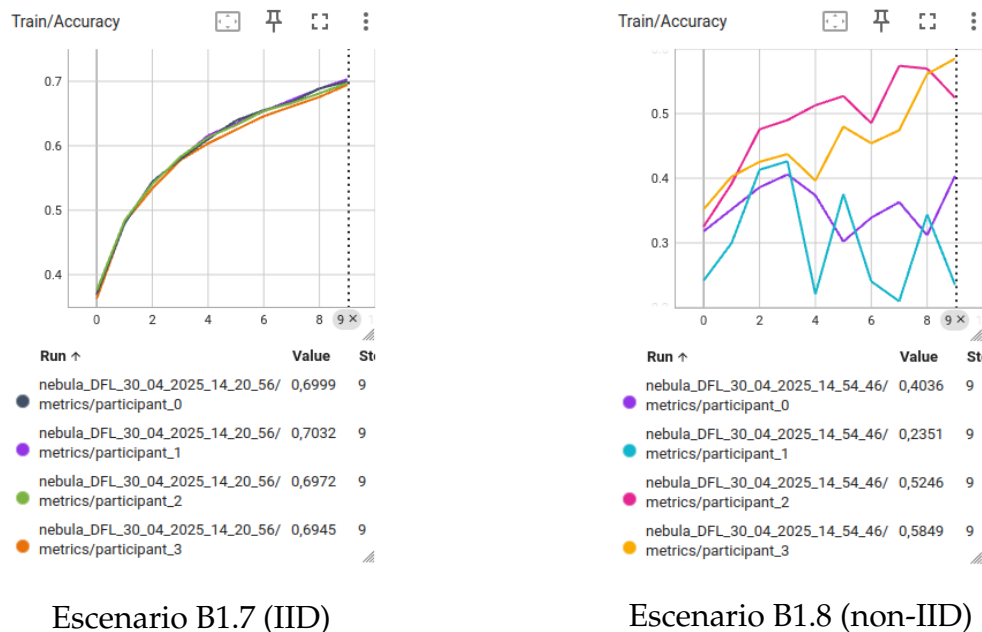
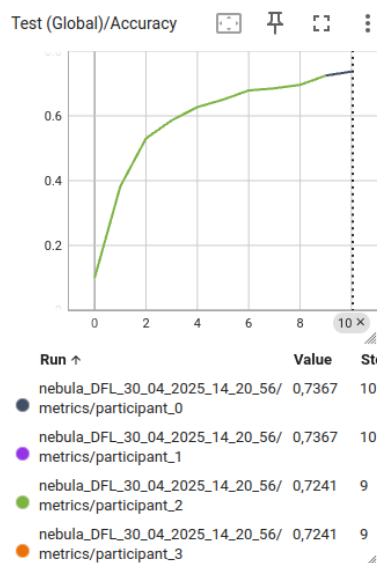
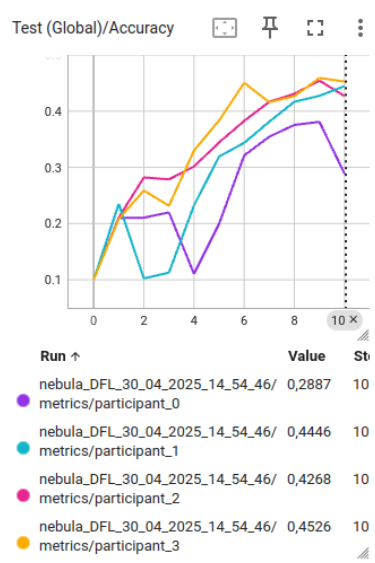


Figura 1.13: Comparación de Train Accuracy entre B1.7 y B1.8

En la **Figura 1.13** se compara el accuracy local en el conjunto de train entre los escenarios B1.7 (IID) y B1.8 (non-IID). En el escenario B1.7 todos los nodos muestran un crecimiento uniforme y progresivo del accuracy, con valores alrededor de 0,7 en la última ronda. No obstante, en el escenario B1.6, el accuracy muestra un crecimiento mucho más irregular, con grandes oscilaciones y diferencias notables entre los nodos. En particular, el nodo morado, que representa al server, muestra un rendimiento con muchas fluctuaciones y valores del accuracy bajos, alcanzando un valor de 0,4036 en la última ronda.



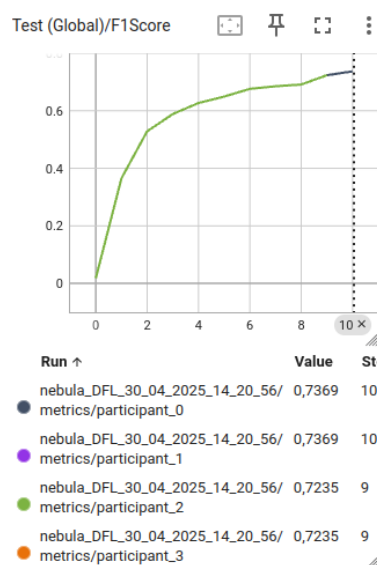
Escenario B1.7 (IID)



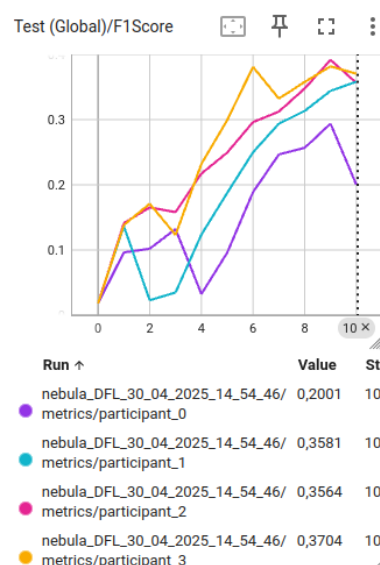
Escenario B1.8 (non-IID)

Figura 1.14: Comparación de Test Accuracy Global entre B1.7 y B1.8

En la **Figura 1.14** se muestra la evolución del accuracy global en el conjunto de test para los escenarios B1.7 (IID) y B1.8 (non-IID). El rendimiento en B1.8 es notablemente inferior, con una convergencia más lenta y con grandes oscilaciones. Tres de los nodos alcanzan valores del accuracy en torno a 0,44 en la última ronda, aunque el nodo morado sufre una bajada del accuracy en las últimas rondas, acabando con 0,2887. No obstante, en B1.7 el accuracy crece uniformemente y alcanza valores más elevados que en B1.8, alrededor de 0,73 en la última ronda para todos los nodos.



Escenario B1.7 (IID)



Escenario B1.8 (non-IID)

Figura 1.15: Comparación de Test F1 Score Global entre B1.7 y B1.8

En la **Figura 1.15** se muestra la evolución del F1 Score global en el conjunto de test para los escenarios B1.7 (IID) y B1.8 (non-IID). Los resultados son similares a los obtenidos para el accuracy global en test, con un rendimiento en B1.8 notablemente inferior debido a una convergencia más lenta y a las grandes oscilaciones a lo largo de las rondas. En B1.7 el F1 score crece uniformemente y alcanza valores más elevados que en B1.8, alrededor de 0,73 en la última ronda para todos los nodos.

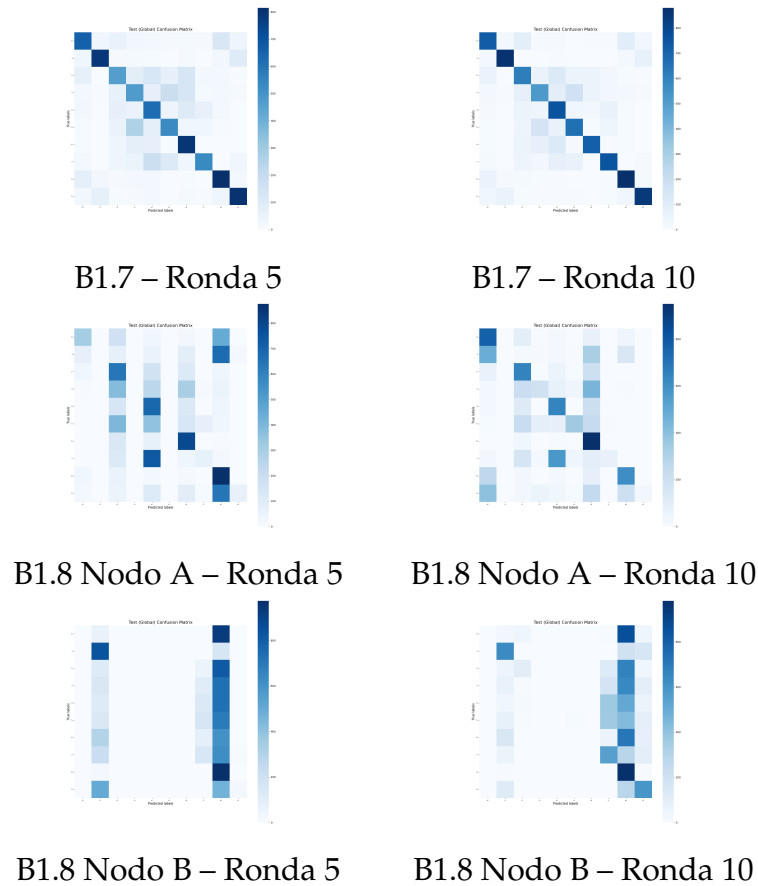


Figura 1.16: Comparación de matrices de confusión entre B1.7 (IID) y B1.8 (non-IID, nodos A y B) en rondas 5 y 10.

En la **Figura 1.16** se presentan las matrices de confusión correspondientes a las rondas 5 y 10 de los escenarios B1.7 (IID) y B1.8 (non-IID). Dado que en B1.8 se obtuvieron resultados distintos según el nodo, se han incluido dos matrices por cada ronda para reflejar esa variabilidad. En el escenario B1.7 se observa, en general, un buen rendimiento del modelo, tanto en la ronda 5 como en la 10. En cambio, en B1.8, se observan diferencias notables entre nodo. Algunos muestran un desempeño bajo en la ronda 5, con ligeras mejoras en la ronda 10, aunque persisten errores frecuentes en ciertas clases, especialmente en la 1, la 3, la 7 y la 9. Otros nodos presentan resultados muy pobres, pues en la ronda 5 solo se predicen correctamente las clases 1 y 8, y aunque hay avances en la ronda 10, el rendimiento general sigue siendo deficiente.

Finalmente, en escenarios con arquitectura *simplemobilenet* y federación

descentralizada, también se evidencian diferencias marcadas según el tipo de distribución de los datos. Aquellos con datos non-IID muestran un rendimiento bastante inferior, además de una fuerte disparidad entre nodos dentro del mismo escenario. En resumen, una distribución de datos heterogénea impacta negativamente en el rendimiento del entrenamiento federado, ya que ciertos nodos carecen de una representación adecuada de las clases, lo que limita su capacidad de aprendizaje y, por extensión, perjudica al desempeño global de la federación.

1.3.2. Comparaciones del impacto de la arquitectura del modelo (CNN vs simplemobilenet)

En este apartado se analiza cómo influye la arquitectura del modelo en el rendimiento del aprendizaje federado, comparando dos arquitecturas distintas: una red convolucional simple (CNN) y una versión simplificada de MobileNet (*simplemobilenet*). Se estudian tanto entornos con federación centralizada (CFL) como descentralizada (DFL), considerando el F1 score bajo distribuciones de datos IID y non-IID. De esta forma, se repondera a la pregunta:

PREGUNTA 2

¿Cómo afecta la arquitectura del modelo a la efectividad del CFL y del DFL?

RESPUESTA:

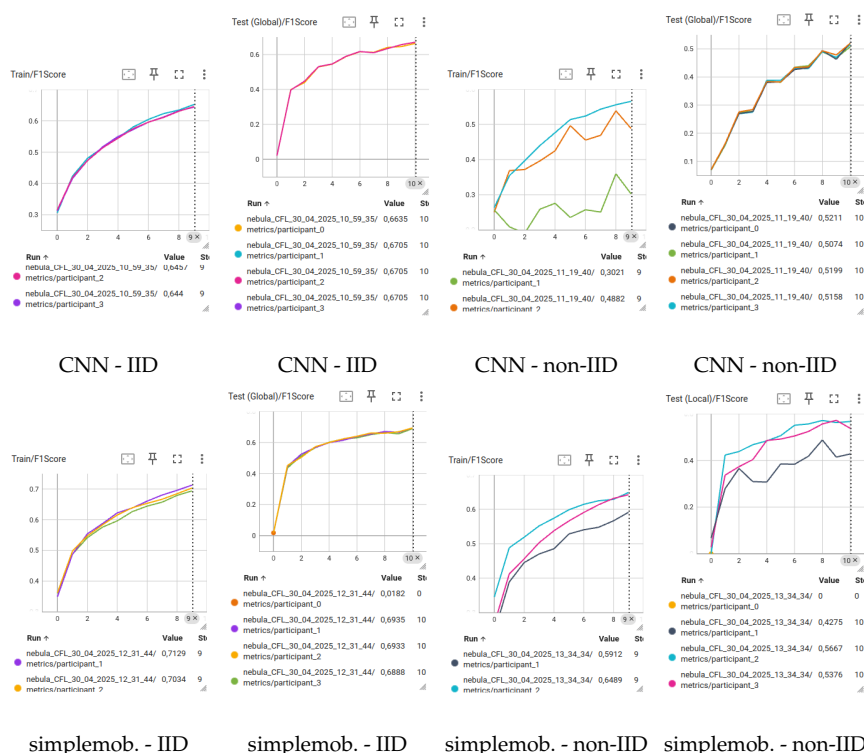


Figura 1.17: Comparación de F1 score en CFL: CNN (↑) vs simplemobilenet (↓), con datos IID (←) y non-IID (→).

En la **Figura 1.17** se muestran los gráficos del F1 score en los conjunto de train y test global para los escenarios CFL analizados, específicamente B1.1 (CNN, IID), B1.2 (CNN, non-IID), B1.5 (*simplemobilenet*, IID) y B1.6 (*simplemobilenet*, non-IID). Para evaluar el impacto de usar una arquitectura de red CNN o una *simplemobilenet*, compararemos B1.1 con B1.5, y después B1.2 con B1.6.

En primer lugar, en el conjunto de train, el F1 score en B1.1 (CNN, IID) muestra un crecimiento consistente, rápido, y uniforme entre los 3 nodos, alcanzando valores cercanos a 0,65 en la última ronda. En cambio, en B1.5 (*simplemobilenet*, IID), aunque el F1 score también crece de manera consistente, se observan diferencias ligeramente más notables entre los distintos nodos, aunque siguen obteniendo valores muy similares entre sí, en torno a 0,7 en la última ronda. Este comportamiento sugiere que, a pesar de la ligera variabilidad entre los nodos, el modelo *simplemobilenet* presenta un desempeño ligeramente superior en términos de F1 score en el conjunto de train.

Por otro lado, el F1 score en el conjunto de test muestra un crecimiento bastante similar en ambos escenarios, con valores al final del entrenamiento que son bastante cercanos entre sí, aunque con una ligera diferencia, pues en B1.1 los nodos toman valores en torno a 0,67, mientras que en B1.5 están en torno a 0,69. De todas formas, aunque el rendimiento es ligeramente mejor en B1.5, este escenario también podría estar sufriendo de algo de overfitting, pues el rendimiento en test es algo peor que en train. Este overfitting podría deberse a que el modelo *simplemobilenet* sea más complejo que CNN.

En segundo lugar, en cuanto al F1 score en el conjunto de train con datos non-IID, se observa que en B1.2 (CNN, non-IID) se muestra un crecimiento consistente para el nodo azul, que finaliza con un F1 score en torno a 0,55, aunque los otros dos nodos presentan fluctuaciones, acabando con valores de 0,3021 y 0,4882 en la última ronda. En cambio, en B1.6 (*simplemobilenet*, non-IID), el F1 score crece de manera algo más consistente en los tres nodos, sin tantas fluctuaciones, aunque con diferencias entre ellos. Dos de ellos se acercan a un F1 score de 0,65 en la última ronda, mientras que el tercero se queda con un valor de 0,5912. Con todo esto, se concluye que el modelo *simplemobilenet* presenta un desempeño ligeramente superior en términos de F1 score en el conjunto de train.

Por otra parte, el F1 score en el conjunto de test muestra algunas diferencias entre ambos escenarios, En B1.2 todos los nodos se comportan de forma muy similar entre sí, alcanzando valores en torno a 0,51 en la última ronda. Sin embargo, en B1.6, hay diferencias más marcadas entre los distintos nodos, siendo el nodo azul el que muestra un mejor rendimiento, con un valor de 0,5667, seguido por el rosa, con un valor de 0,5376, y finalmente el gris, que presenta más fluctuaciones, y finaliza con un F1 score de 0,4275. Esto sugiere que, aunque el modelo *simplemobilenet* parece mostrar resultados mejores en varios nodos, su rendimiento general sigue siendo más variable en comparación con el modelo CNN. Las diferencias entre nodos en B1.6 reflejan cómo las arquitecturas más complejas pueden ser más sensibles a la heterogeneidad de los datos, lo que resulta en un desempeño menos uniforme en una configuración non-IID.

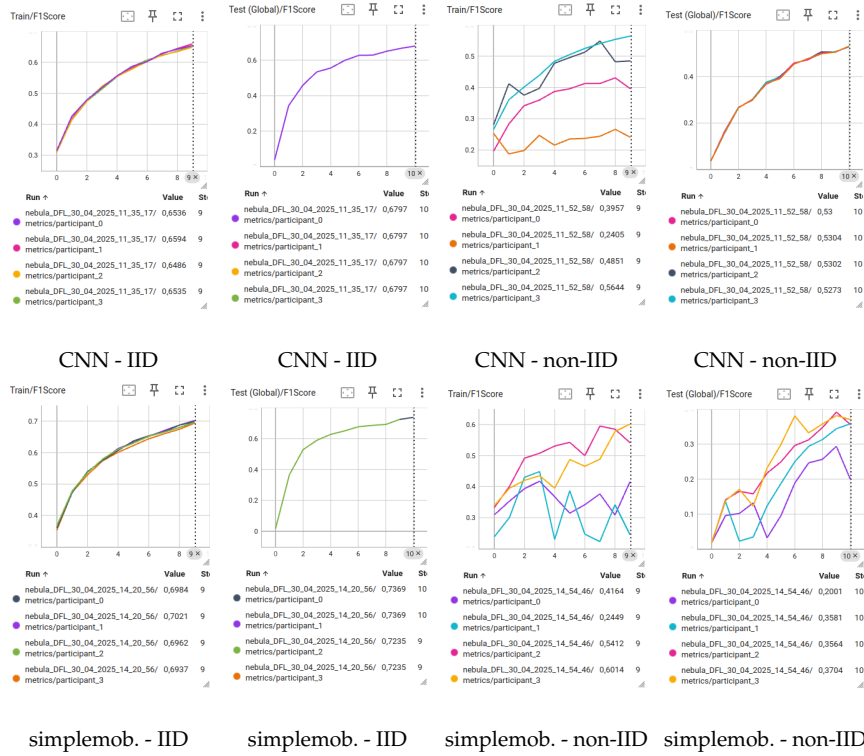


Figura 1.18: Comparación de F1 score en DFL: CNN (↑) vs simplemobilenet (↓), con datos IID (←) y non-IID (→).

En la **Figura 1.18** se muestran los gráficos del F1 score en los conjunto de train y test global para los escenarios DFL analizados, específicamente B1.3 (CNN, IID), B1.4 (CNN, non-IID), B1.7 (*simplemobilenet*, IID) y B1.8 (*simplemobilenet*, non-IID). Para evaluar el impacto de usar una arquitectura de red CNN o una *simplemobilenet*, compararemos B1.3 con B1.7, y después B1.4 con B1.8.

Para empezar, en el conjunto de train, el F1 score en B1.3 (CNN, IID) muestra un crecimiento consistente, rápido, y uniforme entre los 3 nodos, alcanzando valores cercanos a 0,65 en la última ronda. En comparación, en B1.7 (*simplemobilenet*, IID), aunque el F1 score también crece de manera consistente, se observan diferencias ligeramente más notables entre los distintos nodos, aunque siguen obteniendo valores muy similares entre sí, en torno a 0,7 en la última ronda. Este comportamiento sugiere que, a pesar de la ligera variabilidad entre los nodos, el modelo *simplemobilenet* vuelve a presentar un desempeño ligeramente superior en términos de F1 score en el conjunto de train.

Por otro lado, el F1 score en el conjunto de test muestra un crecimiento bastante similar en ambos escenarios, con valores al final del entrenamiento que son bastante cercanos entre sí, aunque con una ligera diferencia, pues en B1.3 todos los nodos toman el valor 0,6797, mientras que en B1.7 están en torno a 0,69. De nuevo, aunque el rendimiento es ligeramente mejor en B1.7, en este escenario podría haber un ligero overfitting, pues el rendimiento en test es algo peor que en train.

En cuanto al F1 score en el conjunto de train con datos non-IID, se observa que en B1.4 (CNN, non-IID) se muestra un crecimiento consistente para el nodo azul,

que finaliza con un F1 score de 0,5644, aunque los otros tres nodos presentan fluctuaciones, acabando con valores de 0,4851, 0,3957 y 0,2405 en la última ronda. En cambio, en B1.8 (*simplemobilenet*, non-IID), el F1 score muestra fructuaciones en todos los nodos, y logran valores muy diferentes en la última ronda: 0,6014, 0,5412, 0,4164 y 0,2449. A pesar de las oscilaciones en ambos modelos, se puede observar que los valores finales obtenidos por el modelo *simplemobilenet* son en general ligeramente superiores a los obtenidos con CNN.

Por último, el F1 score en el conjunto de test muestra algunas diferencias entre ambos escenarios. En B1.4 todos los nodos se comportan de forma muy similar entre sí, alcanzando valores en torno a 0,53 en la última ronda. Sin embargo, en B1.8, se observan diferencias más marcadas entre los distintos nodos, con fluctuaciones marcadas en muchos de ellos. A pesar de esto, tres nodos alcanzan valores cercanos a 0,36, pero el nodo morado muestra un desempeño mucho peor, con un F1 score de solo 0,2001 en la última ronda. En este caso, con datos non-IID, el modelo *simplemobilenet* obtiene resultados considerablemente peores que el modelo CNN. De hecho, *simplemobilenet* parece estar sufriendo de overfitting, lo cual podría estar perjudicando su rendimiento en el conjunto de test. Además, esta tendencia sugiere que modelos más complejos como *simplemobilenet* podrían tener mayores dificultades para manejar datos con distribución no homogénea, ya que su flexibilidad podría hacer que se sobreajusten a las características locales de los nodos sin generalizar adecuadamente.

En conclusión, al comparar el rendimiento de los modelos CNN y *simplemobilenet* en los escenarios CFL y DFL, se observa que, en general, *simplemobilenet* presenta un desempeño ligeramente superior en el conjunto de train, pero con una mayor variabilidad entre los nodos, especialmente en escenarios con datos non-IID. Aunque en algunos casos el rendimiento de *simplemobilenet* en el conjunto de test es superior que el de CNN, también se observa un mayor riesgo de overfitting al ser un modelo más complejo, lo que puede afectar su capacidad de generalización, especialmente con datos con distribución no homogénea. Por ello, los modelos más simples como CNN parecen más robustos frente a datos non-IID.

1.3.3. Comparaciones del uso de recursos (CPU y red)

En este apartado, se analizará el impacto de las configuraciones de los diferentes escenarios en términos de CPU y red. Así, observaremos como las diferentes arquitecturas de modelo (CNN vs *simplemobilenet*), la distribución de los datos (IID vs non-IID), o el tipo de federación (centralizada vs descentralizada) influyen en el consumo de estos recursos a lo largo de las rondas de entrenamiento. Esta evaluación ayudará a entender si la complejidad del modelo o la distribución de los datos afectan al rendimiento en términos de eficiencia computacional y de red. De esta forma, se responderá a la pregunta:

PREGUNTA 3

¿Hay alguna diferencia en el uso de CPU o de la red?

RESPUESTA:

USO DE CPU Y RED EN B1.1



Figura 1.19: Uso de CPU en B1.1

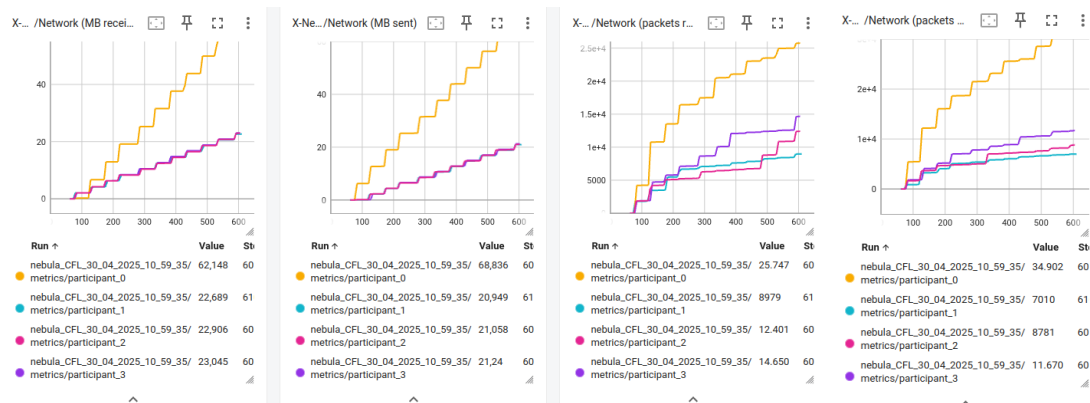


Figura 1.20: Uso de red en B1.1

USO DE CPU Y RED EN B1.2

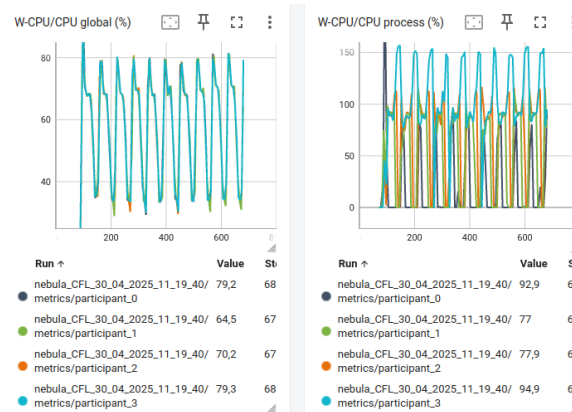


Figura 1.21: Uso de CPU en B1.2

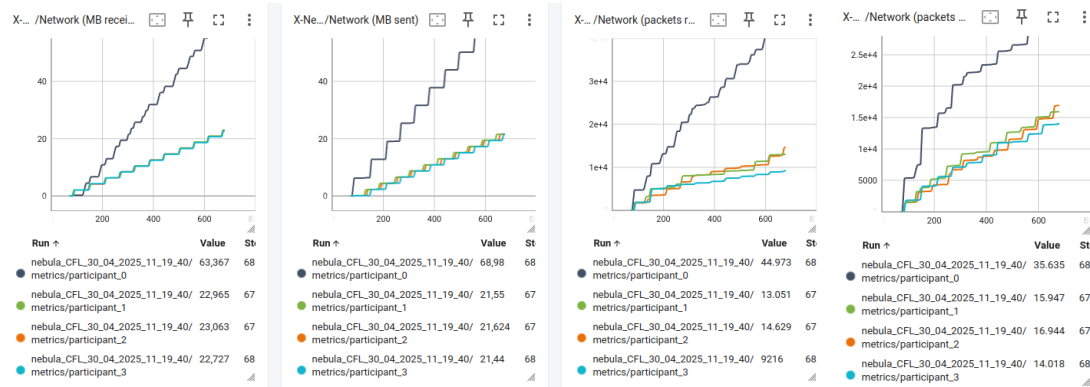


Figura 1.22: Uso de red en B1.2

USO DE CPU Y RED EN B1.3

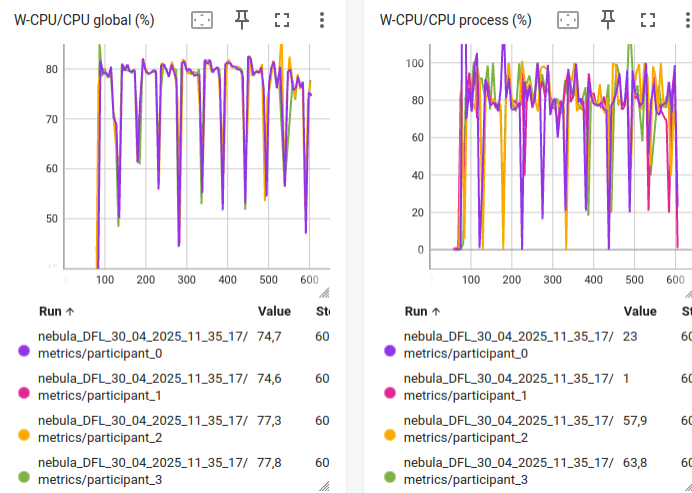


Figura 1.23: Uso de CPU en B1.3

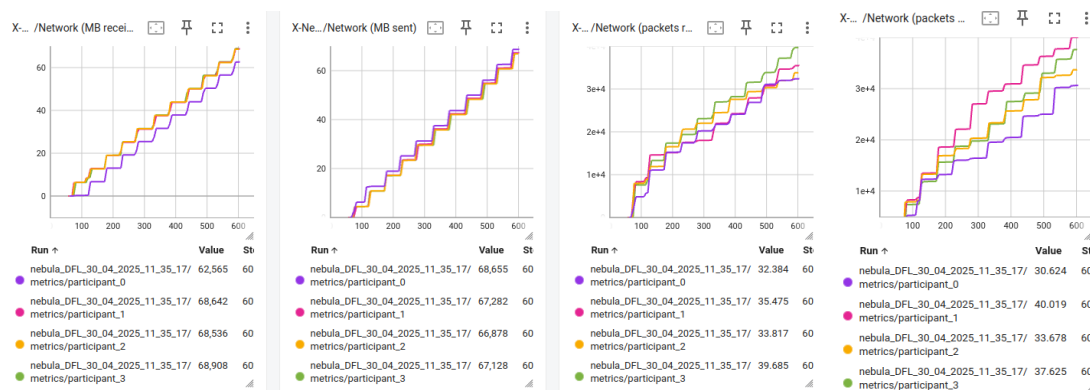


Figura 1.24: Uso de red en B1.3

USO DE CPU Y RED EN B1.4

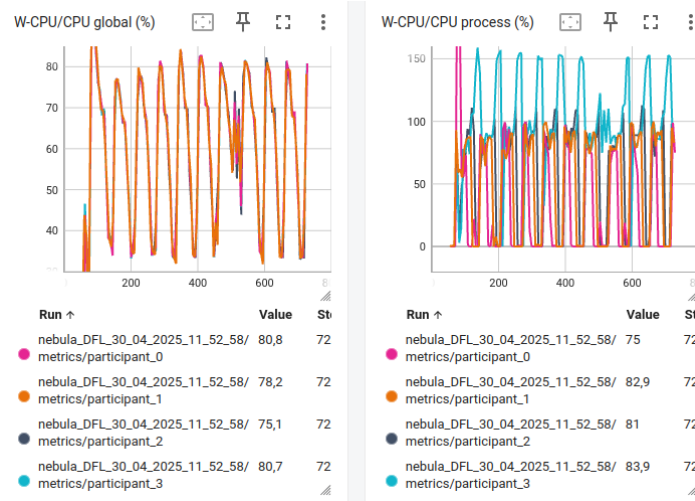


Figura 1.25: Uso de CPU en B1.4

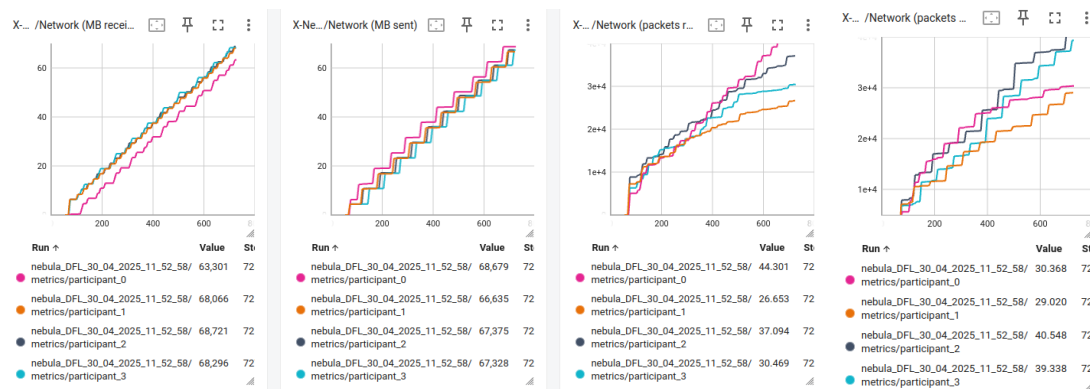


Figura 1.26: Uso de red en B1.4

USO DE CPU Y RED EN B1.5

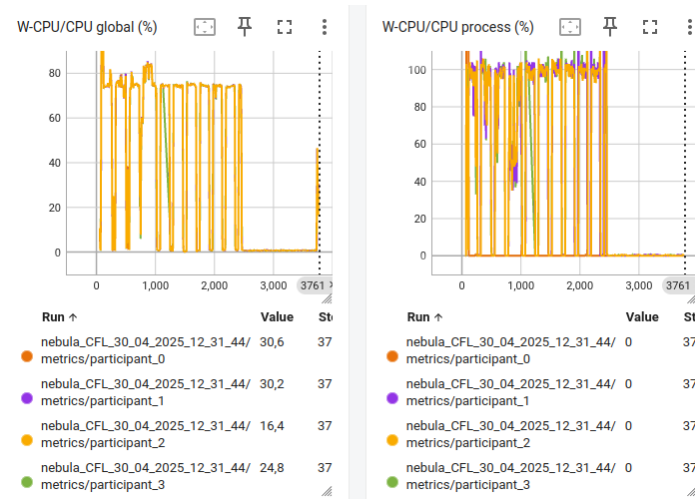


Figura 1.27: Uso de CPU en B1.5

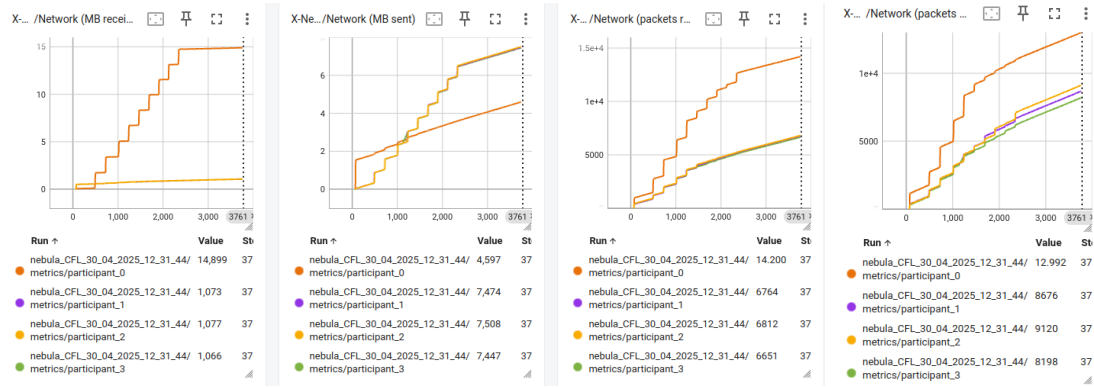


Figura 1.28: Uso de red en B1.5

USO DE CPU Y RED EN B1.6



Figura 1.29: Uso de CPU en B1.6

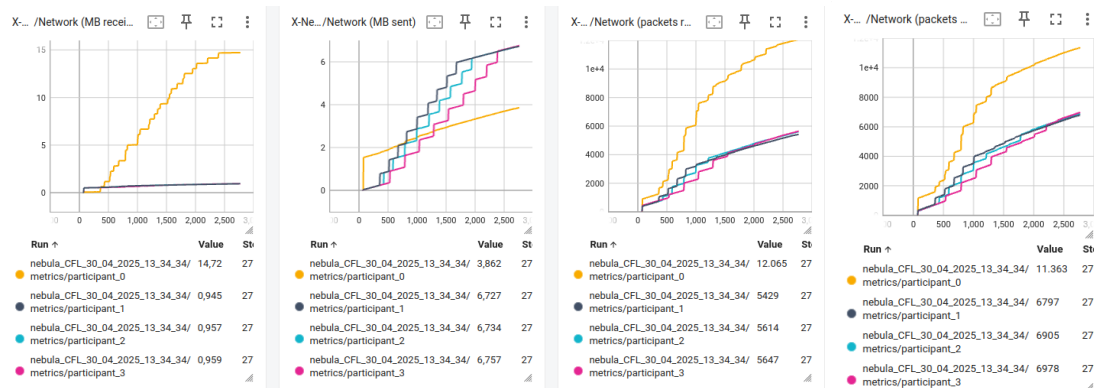


Figura 1.30: Uso de red en B1.6

USO DE CPU Y RED EN B1.7



Figura 1.31: Uso de CPU en B1.7

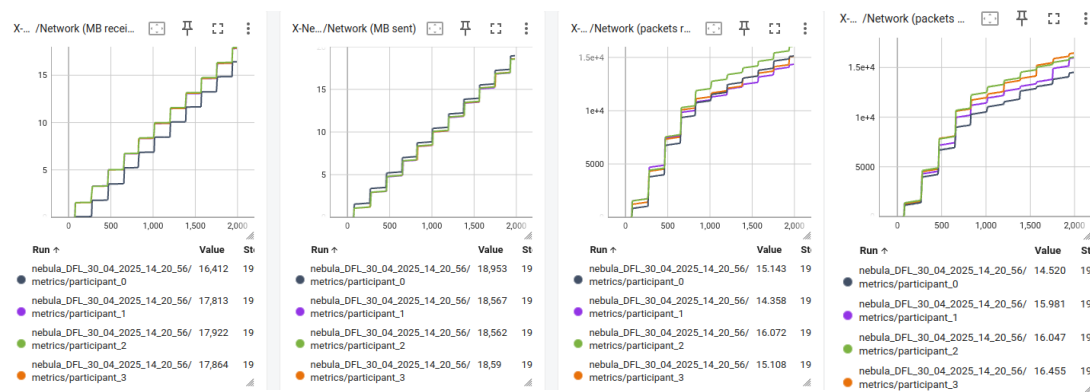


Figura 1.32: Uso de red en B1.7

USO DE CPU Y RED EN B1.8

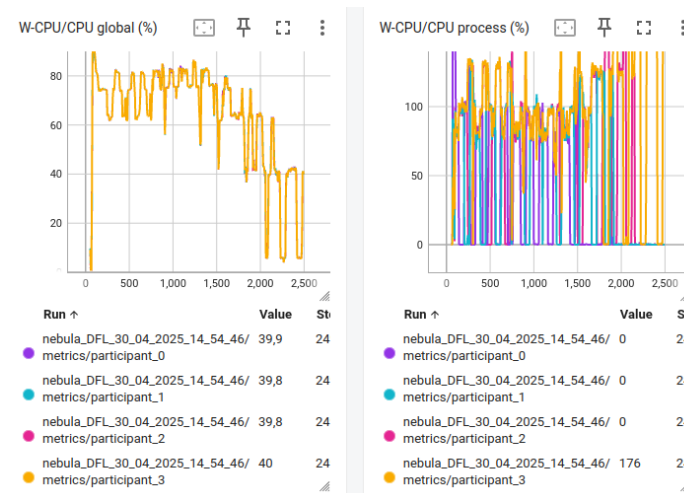


Figura 1.33: Uso de CPU en B1.8

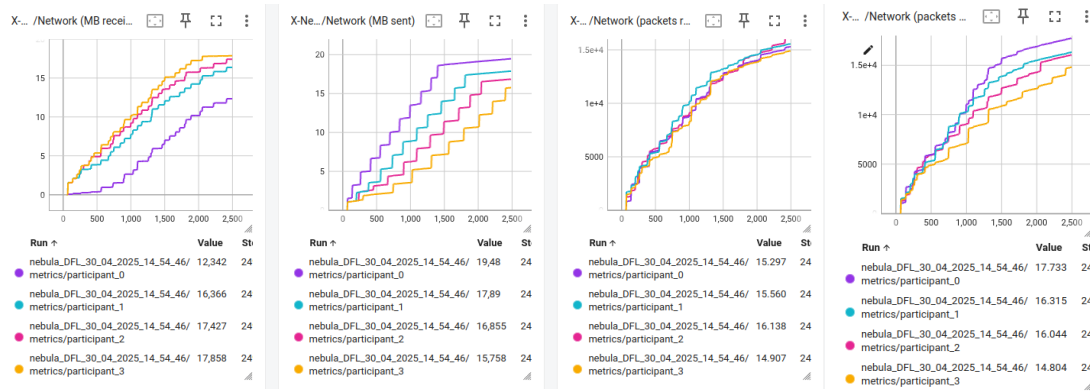


Figura 1.34: Uso de red en B1.8

ANÁLISIS COMPARATIVO DEL USO DE RECURSOS

1. Comparación por arquitectura: CNN vs *simplemobilenet*

■ B1.1 (CNN, IID, CFL) vs B1.5 (SimpleMobileNet, IID, CFL):

Comparando la Figura 1.19 para B1.1 con la Figura 1.27 para B1.5, se puede observar que para la CPU global, ambos escenarios presentan grandes oscilaciones a lo largo del entrenamiento, alcanzando valores de alrededor de 80 % en ambos casos, aunque en el caso de B1.1 ligeramente superiores. Sin embargo, las oscilaciones en B1.5 son más grandes, llegando a bajar hasta cerca del 0 % para luego aumentar hasta cerca del 80 %, mientras que en B1.1 las bajadas son menos pronunciadas, no bajando por debajo del 40 %. En cuanto a la CPU process, esta es mayor en general para B1.5, con grandes oscilaciones, pero alcanzando valores en torno al 100 % en múltiples pasos.

Ahora, comparando la Figura 1.20 para B1.1 con la Figura 1.28 para B1.5, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es mayor para el escenario B1.1 que para el B1.5.

■ B1.2 (CNN, non-IID, CFL) vs B1.6 (SimpleMobileNet, non-IID, CFL):

Comparando la Figura 1.21 para B1.2 con la Figura 1.29 para B1.6, se observa que la CPU global alcanza porcentajes de uso mayores para B1.2 (ligeramente superiores a 80 %), pero en el caso de B1.6 las oscilaciones son mayores, llegando en algunos casos a valores cercanos al 0 % de uso. En cuanto a la CPU process, los porcentajes son mayores para B1.6 (llegando a valores alrededor de 200 %).

Ahora, comparando la Figura 1.22 para B1.2 con la Figura 1.30 para B1.6, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es mayor para el escenario B1.2 que para el B1.6.

■ B1.3 (CNN, IID, DFL) vs B1.7 (SimpleMobileNet, IID, DFL):

Comparando la **Figura 1.23** para B1.3 con la **Figura 1.31** para B1.7, se observa que la CPU global alcanza porcentajes de uso muy similares entre ambos escenarios, aunque ligeramente mayores en algunos pasos para B1.7. En cuanto a la CPU process, los porcentajes también son similares, aunque en el caso de B1.3, presentan oscilaciones más marcadas.

Ahora, comparando la **Figura 1.24** para B1.3 con la **Figura 1.32** para B1.7, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es considerablemente mayor para el escenario B1.3 que para el B1.7.

- **B1.4 (CNN, non-IID, DFL) vs B1.8 (SimpleMobileNet, non-IID, DFL):**

Comparando la **Figura 1.25** para B1.4 con la **Figura 1.33** para B1.8, se observa que la CPU global alcanza porcentajes de uso similares en ambos escenarios, aunque en B1.8 se produce una disminución del uso de la CPU global en los últimos pasos. En cuanto a la CPU process, los porcentajes son también similares en ambos escenarios.

Ahora, comparando la **Figura 1.26** para B1.4 con la **Figura 1.34** para B1.8, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es considerablemente mayor para el escenario B1.4 que para el B1.8.

En general, la comparación entre los escenarios con arquitectura CNN y *simplemobilenet* revela algunas tendencias interesantes en cuanto al uso de recursos. En los escenarios con arquitectura CNN, se observa un uso más constante y ligeramente más alto de la CPU global en varias de las comparaciones. Sin embargo, en los escenarios con *simplemobilenet*, el uso de la CPU process tiende a ser algo más elevada en general. En cuanto al uso de la red, en todos los escenarios comparados, los valores de tráfico de red (MB recibidos, MB enviados, paquetes) son consistentemente más bajos en los escenarios con *simplemobilenet*, lo que sugiere que, a pesar de ser una arquitectura más compleja, el modelo *simplemobilenet* no requiere tanto intercambio de datos entre nodos como el modelo CNN. En resumen, en términos de red, *simplemobilenet* parece ser menos demandante.

2. Comparación por tipo de federación: Centralizada vs Descentralizada

- **B1.1 (CNN, IID, CFL) vs B1.3 (CNN, IID, DFL):**

Comparando la **Figura 1.19** para B1.1 con la **Figura 1.23** para B1.3, se observa que la CPU global alcanza porcentajes de uso muy similares entre ambos escenarios, aunque en el caso de B1.1 presenta un pico inicial que alcanza valores más elevados, aunque luego se estabiliza y sigue un patrón similar a B1.3. En cuanto a la CPU process, los porcentajes también son similares, con valores que oscilan entre el 0 % y alrededor del 100 %.

Ahora, comparando la **Figura 1.20** para B1.1 con la **Figura 1.24** para B1.3, se observa que en todos los casos (MB received, MB sent, packets

received y packets sent), el uso de red es mayor para el escenario B1.3 que para el B1.1, y además, mientras que en B1.3 todos los nodos presentan crecimientos similares, en el caso de B1.1, el nodo central tiene un uso de red que crece más rápidamente que el resto.

■ **B1.2 (CNN, non-IID, CFL) vs B1.4 (CNN, non-IID, DFL):**

Comparando la **Figura 1.21** para B1.2 con la **Figura 1.25** para B1.4, se observa que la CPU global alcanza porcentajes de uso muy similares entre ambos escenarios. En cuanto a la CPU process, los porcentajes también son similares, con valores que oscilan entre el 0 % y alrededor del 150 %.

Ahora, comparando la **Figura 1.22** para B1.2 con la **Figura 1.26** para B1.4, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es mayor para el escenario B1.4 que para el B1.2 y, de nuevo, mientras que en B1.4 todos los nodos presentan crecimientos similares, en el caso de B1.2, el nodo central tiene un uso de red que crece más rápidamente que el resto.

■ **B1.5 (SimpleMobileNet, IID, CFL) vs B1.7 (SimpleMobileNet, IID, DFL):**

Comparando la **Figura 1.27** para B1.5 con la **Figura 1.31** para B1.7, se observa que la CPU global alcanza porcentajes de uso ligeramente superiores en B1.7, donde se alcanzan valores superiores a 80 %, que en B1.5, aunque las oscilaciones en B1.5 son más marcadas, fluctuando entre valores cercanos al 0 % y el 80 %. En cuanto a la CPU process, los porcentajes en B1.5 muestran oscilaciones entre el 0 % y el 100 % a lo largo del entrenamiento, mientras que en B1.7 rondan más en torno al 80 %, con oscilaciones no tan marcadas.

Ahora, comparando la **Figura 1.28** para B1.5 con la **Figura 1.32** para B1.7, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es mayor para el escenario B1.7 que para el B1.5, y otra vez se observa que, mientras que en B1.7 todos los nodos presentan crecimientos similares, en el caso de B1.5, el nodo central tiene un uso de red que crece más rápidamente que el resto (excepto para MB sent, donde el uso de red del nodo central de hecho crece más lento que el resto).

■ **B1.6 (SimpleMobileNet, non-IID, CFL) vs B1.8 (SimpleMobileNet, non-IID, DFL):**

Comparando la **Figura 1.29** para B1.6 con la **Figura 1.33** para B1.8, se observa que la CPU global alcanza porcentajes de uso ligeramente superiores en B1.8, donde se alcanzan valores en torno al 80 %, que en B1.6. Además, en ambos escenarios se observa un decrecimiento en el uso de CPU global en los últimos pasos del entrenamiento. En cuanto a la CPU process, los porcentajes en B1.6 alcanzan valores algo más elevados que en B1.8, cercanos al 200 %.

Ahora, comparando la **Figura 1.30** para B1.6 con la **Figura 1.34** para B1.8, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), el uso de red es mayor para el escenario B1.8 que para el B1.6, y otra vez se observa que, mientras que en B1.8 todos los nodos presentan crecimientos similares, en el caso de B1.6, el nodo central tiene un uso de red que crece más rápidamente que el resto (excepto para MB sent, donde el uso de red del nodo central de hecho crece más lento que el resto).

La comparación entre los escenarios de federación centralizada y descentralizada muestra que, aunque el uso de CPU (tanto global como de proceso) es en general similar entre ambos enfoques, el patrón de uso puede variar ligeramente dependiendo de la arquitectura utilizada, siendo algo más estable en los escenarios descentralizados. Sin embargo, las diferencias más destacables se observan en el uso de la red: en todos los casos, los escenarios con federación descentralizada presentan un mayor volumen de datos transferidos (MB y paquetes), con un reparto más homogéneo entre nodos. En contraste, en los escenarios centralizados, el nodo central asume una carga de red significativamente mayor. Esto sugiere que, aunque la descentralización implica mayor uso global de la red, puede mejorar el equilibrio de carga entre nodos y evitar sobrecargar un nodo específico.

3. Comparación por distribución de datos: IID vs non-IID

■ B1.1 (CNN, IID, CFL) vs B1.2 (CNN, non-IID, CFL):

Comparando la **Figura 1.19** para B1.1 con la **Figura 1.21** para B1.2, se observa que la CPU global alcanza porcentajes muy similares en ambos escenarios, alcanzando valores en torno al 80 %, aunque en B1.2 las oscilaciones son algo más marcadas que en B1.1. En cuanto a la CPU process, los porcentajes en B1.2 alcanzan valores más elevados, cercanos al 150 % en algunos nodos, mientras que en B1.1 se quedan en torno al 100 %.

Ahora, comparando la **Figura 1.20** para B1.1 con la **Figura 1.22** para B1.2, se observa que para MB received y MB sent, los valores son muy similares en ambos escenarios, aunque para packets received y packets sent, son algo más elevados en B1.2, especialmente para el nodo central.

■ B1.3 (CNN, IID, DFL) vs B1.4 (CNN, non-IID, DFL):

Comparando la **Figura 1.23** para B1.3 con la **Figura 1.25** para B1.4, se observa que la CPU global alcanza porcentajes muy similares en ambos escenarios, alcanzando valores en torno al 80 %, aunque en B1.4 las oscilaciones son algo más marcadas que en B1.3. En cuanto a la CPU process, los porcentajes en B1.4 alcanzan valores más elevados, cercanos al 150 % en algunos nodos, mientras que en B1.3 se quedan en torno al 100 %.

Ahora, comparando la **Figura 1.24** para B1.3 con la **Figura 1.26** para B1.4, se observa que para MB received y MB sent, los valores son muy similares

en ambos escenarios, aunque para packets received y packets sent, en B1.4 se observa una diferencia algo más marcada entre los distintos nodos, con algunos alcanzando valores ligeramente más elevados, especialmente al final del entrenamiento.

- **B1.5 (SimpleMobileNet, IID, CFL) vs B1.6 (SimpleMobileNet, non-IID, CFL):**

Comparando la **Figura 1.27** para B1.5 con la **Figura 1.29** para B1.6, se observa que la CPU global alcanza porcentajes muy similares en ambos escenarios, alcanzando valores ligeramente inferiores a 80 %, aunque en B1.5 hay una caída bastante marcada del uso de la CPU global al final del entrenamiento. En cuanto a la CPU process, los porcentajes en B1.6 alcanzan valores más elevados, cercanos al 200 % en algunos nodos, mientras que en B1.5 se quedan en torno al 100 %.

Ahora, comparando la **Figura 1.28** para B1.5 con la **Figura 1.30** para B1.6, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), los valores son muy similares en ambos escenarios.

- **B1.7 (SimpleMobileNet, IID, DFL) vs B1.8 (SimpleMobileNet, non-IID, DFL):**

Comparando la **Figura 1.31** para B1.7 con la **Figura 1.33** para B1.8, se observa que la CPU global alcanza porcentajes muy similares en ambos escenarios, aunque ligeramente superiores y con oscilaciones más marcadas en B1.7. Además, en B1.8 hay una caída en el porcentaje de uso de la CPU global al final del entrenamiento. En cuanto a la CPU process, los porcentajes en B1.4 alcanzan valores más elevados, algo superiores al 100 % en algunos nodos, mientras que en B1.3 se quedan en torno al 80 %, pero alcanzando porcentajes cercanos al 100 % en algunos pasos.

Ahora, comparando la **Figura 1.32** para B1.7 con la **Figura 1.34** para B1.8, se observa que en todos los casos (MB received, MB sent, packets received y packets sent), los valores son muy similares en ambos escenarios, aunque en B1.8 se observan algunas diferencias más marcadas entre diferentes nodos.

La comparación entre escenarios con datos distribuidos de forma IID y non-IID muestra que el uso de CPU global es muy similar en ambos casos, aunque en los escenarios non-IID tienden a presentarse oscilaciones más marcadas y un uso de CPU process más elevado. En cuanto al uso de red, las diferencias son generalmente pequeñas, aunque en algunos escenarios non-IID se observa una mayor variabilidad entre nodos, especialmente en términos de paquetes transmitidos, lo que podría reflejar una mayor heterogeneidad en la carga de trabajo.

2. Boletín II

2.1. Introducción a la tarea y ejercicios a resolver

Para este boletín, se deben abordar los siguientes **ejercicios**:

1. Utilizar la configuración definida en la Sesión I (CIFAR-10, CNN, FedAvg, 1 época, 10 rondas, cuatro nodos, topología fully-connected, IID/Non-IID).
2. Configurar los siguientes ataques al modelo:
 - Model Poisoning con un 10 % de ruido envenenado, gaussiano. Intervalo de ataque predeterminado.
 - Label Flipping con un 80 % de muestras envenenadas, sin target específico. Intervalo de ataque predeterminado.
3. Experimentar con diferentes porcentajes de nodos maliciosos (20 %, 50 %, es decir, uno/dos nodos maliciosos) para evaluar su impacto.
4. Comparar el F1-score entre los experimentos y proporcionar las matrices de confusión para cada ronda.

Con lo que se pretende conseguir:

- Comprender el papel de los sistemas de reputación en el DFL como un mecanismo para evaluar a los nodos vecinos.
- Explorar cómo se calcula la reputación en función del comportamiento de comunicación y la similitud del modelo, y cómo esto influye en la integración del modelo.
- Evaluar diferentes configuraciones de reputación, incluidas estrategias de peso fijo y peso adaptativo, para detectar y mitigar comportamientos maliciosos.
- Comparar el rendimiento del modelo, la convergencia y la estabilidad con y sin filtrado basado en reputación.

2.2. Preguntas a responder

Según los resultados de los experimentos, responder las siguientes **preguntas**:

1. ¿Cómo cambia el impacto de las distribuciones IID frente a Non-IID bajo diferentes condiciones de ataque?

2. ¿Cómo afecta la exposición prolongada a los ataques en la velocidad de convergencia del modelo a lo largo de múltiples rondas? ¿Hay signos de efectos acumulativos de los ataques con el tiempo?
3. ¿Qué posibles contramedidas se podrían implementar para detectar y mitigar estos ataques? ¿Son ciertas arquitecturas de modelos más resistentes a ataques específicos? En caso afirmativo, ¿por qué?

2.3. Desarrollo de la tarea

Los **escenarios de federación** que entrenaremos en NEBULA para esta tarea son los siguientes:

Escenario B2.1:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg
- Ataque: Model Poisoning con 10 % de ruido gaussiano
- Porcentaje de nodos maliciosos: 20 %

Escenario B2.2:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Ataque: Model Poisoning con 10 % de ruido gaussiano
- Porcentaje de nodos maliciosos: 20 %

Escenario B2.3:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg
- Ataque: Model Poisoning con 10 % de ruido gaussiano
- Porcentaje de nodos maliciosos: 50 %

Escenario B2.4:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Ataque: Model Poisoning con 10 % de ruido gaussiano
- Porcentaje de nodos maliciosos: 50 %

Escenario B2.5:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg
- Ataque: Label Flipping con 80 % de muestras envenenadas
- Porcentaje de nodos maliciosos: 20 %

Escenario B2.6:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Ataque: Label Flipping con 80 % de muestras envenenadas
- Porcentaje de nodos maliciosos: 20 %

Escenario B2.7:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: IID
- Algoritmo de agregación: FedAvg
- Ataque: Label Flipping con 80 % de muestras envenenadas
- Porcentaje de nodos maliciosos: 50 %

Escenario B2.8:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Ataque: Label Flipping con 80 % de muestras envenenadas
- Porcentaje de nodos maliciosos: 50 %

2.3.1. Comparaciones del impacto de la distribución de los datos (IID vs non-IID) bajo ataques

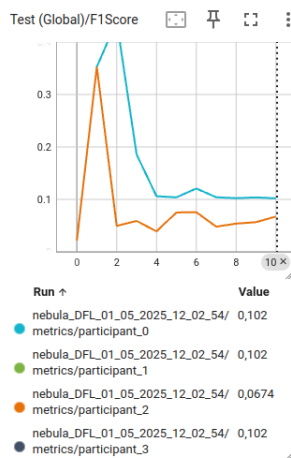
En este apartado se analiza cómo influye la distribución de los datos (IID vs. non-IID) en el rendimiento del aprendizaje federado bajo ataques. Se comparan pares de escenarios con los mismos parámetros de arquitectura, tipo de federación, algoritmo de agregación y tipo de ataque, variando únicamente la distribución de los datos. Para ello, se presentan métricas clave como el F1 score global en el conjunto de test, así como matrices de confusión en rondas específicas para ilustrar los efectos en la clasificación por clase. De esta forma, este apartado nos permitirá responder a la pregunta:

PREGUNTA 1

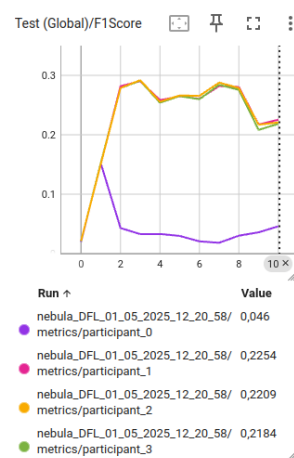
¿Cómo cambia el impacto de las distribuciones IID frente a Non-IID bajo diferentes condiciones de ataque?

RESPUESTA:

COMPARACIÓN CON MODEL POISONING CON 10% DE RUIDO GAUSSIANO Y 20% DE NODOS MALICIOSOS (B2.1 vs B2.2)



Escenario B2.1 (IID)



Escenario B2.2 (non-IID)

Figura 2.1: Comparación de Test F1 Score Global entre B2.1 y B2.2

En la **Figura 2.1** se muestra la evolución del F1 score global en test en los escenarios B2.1 (IID) y B2.2 (non-IID) bajo un ataque de Model Poisoning con un 20 % de nodos maliciosos. En el caso IID (B2.1), se puede intuir que el nodo naranja es el nodo malicioso, pues su F1-score sufre una caída abrupta entre las rondas 1 y 2, lo que indica la ejecución del ataque mediante ruido gaussiano. Posteriormente, se observa una degradación progresiva del rendimiento en los nodos restantes, reflejando cómo la agregación de las actualizaciones envenenadas afecta al modelo global a lo largo del tiempo.

En el escenario non-IID (B2.2), el nodo morado realiza un ataque similar desde las primeras rondas. Aunque su F1 score también se desploma, el efecto sobre los nodos benignos es más moderado. Si bien es cierto que el crecimiento del F1 score se detiene por culpa del ataque, no llega a decaer de una forma tan pronunciada como en B2.1. Esto sugiere que, en configuraciones non-IID, la heterogeneidad de los datos puede amortiguar parcialmente los efectos inmediatos del ataque, posiblemente debido a una menor similitud entre los modelos locales durante la agregación.

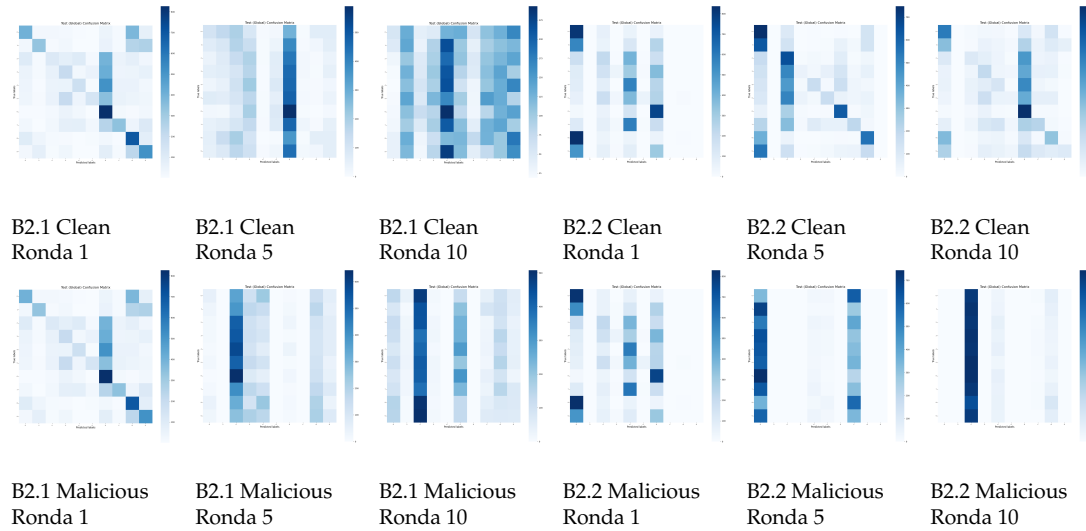


Figura 2.2: Comparación de las matrices de confusión entre B2.1 (IID) y B2.2 (non-IID): en nodos **Clean** (↑) y **Malicious** (↓), en los escenarios **B2.1 (IID)** (←) y **B2.2 (non-IID)** (→).

En la **Figura 2.2** se muestran las matrices de confusión de los escenarios B2.1 y B2.2 en las rondas 1, 5 y 10, tanto para el nodo malicioso como para los nodos benignos. De nuevo, se aprecia que en el escenario IID (B2.1), conforme el nodo malicioso va aplicando el ataque, su rendimiento decae, pero también lo hace el de los nodos restantes, como se puede observar en el aumento de falsos positivos y falsos negativos a lo largo de las rondas.

Sin embargo, en el escenario non-IID (B2.2), a pesar de que el nodo malicioso aplica el ataque y su rendimiento decae rápidamente, el resto de nodos consiguen mantener un rendimiento medianamente aceptable, aunque todavía con bastantes errores. De hecho, la matriz de confusión en estos nodos en la ronda 10 muestra un empeoramiento del rendimiento con respecto a la ronda 5, especialmente para las clases 0, 2 y 8.

COMPARACIÓN CON MODEL POISONING CON 10% DE RUIDO GAUSSIANO Y 50% DE NODOS MALICIOSOS (B2.3 vs B2.4)

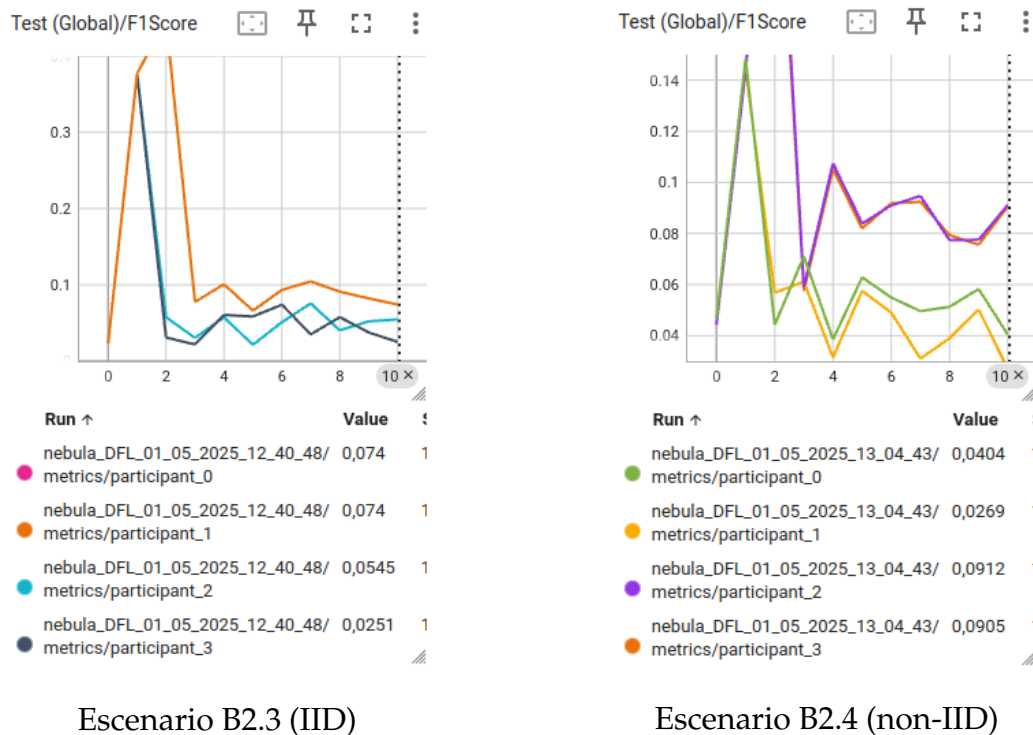


Figura 2.3: Comparación de Test F1 Score Global entre B2.3 y B2.4

En la **Figura 2.3** se muestra la evolución del F1 score global en test en los escenarios B2.3 (IID) y B2.4 (non-IID) bajo un ataque de Model Poisoning con un 50% de nodos maliciosos. En el caso IID (B2.3), se puede intuir que los nodos maliciosos son el azul y el gris, pues su F1-score sufre una caída abrupta entre las rondas 1 y 2, lo que indica la ejecución del ataque mediante ruido gaussiano. Posteriormente, se observa una degradación progresiva del rendimiento en los nodos restantes, reflejando cómo la agregación de las actualizaciones envenenadas afecta al modelo global a lo largo del tiempo.

En el escenario non-IID (B2.4), los nodos amarillo y verde realizan un ataque similar desde las primeras rondas. Su F1 score se desploma en la ronda 2, y a continuación también lo hace el de los nodos benignos, a partir de la ronda 3. Sin embargo, en las siguientes rondas consigue mejorar ligeramente, aunque estabilizándose por debajo de 0,1, por lo que en esta ocasión, con dos nodos malignos, el ataque es más eficiente que cuando solo había uno.

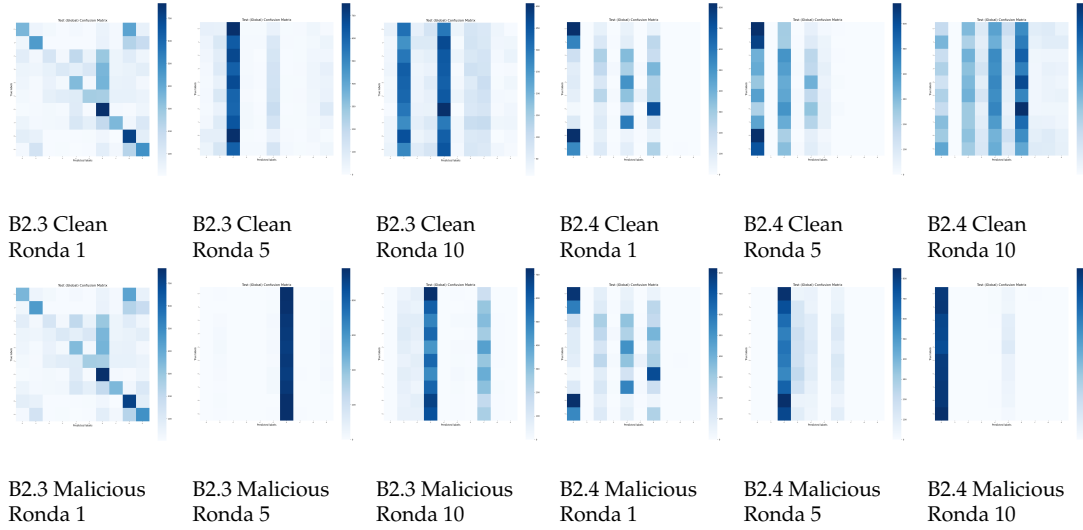


Figura 2.4: Comparación de las matrices de confusión entre B2.3 (IID) y B2.4 (non-IID): en nodos **Clean** (↑) y **Malicious** (↓), en los escenarios **B2.3 (IID)** (←) y **B2.4 (non-IID)** (→).

En la **Figura 2.4** se muestran las matrices de confusión de los escenarios B2.3 y B2.4 en las rondas 1, 5 y 10, tanto para los nodos maliciosos como para los benignos. De nuevo, se aprecia que en el escenario IID (B2.3), conforme el nodo malicioso va aplicando el ataque, su rendimiento decae, pero también lo hace el de los nodos restantes, como se puede observar en el aumento de falsos positivos y falsos negativos a lo largo de las rondas.

De forma similar, en el escenario non-IID (B2.4), los nodos maliciosos aplican el ataque y su rendimiento decae rápidamente, al igual que el de los nodos benignos, resultando en matrices de confusión con muchos falsos positivos y falsos negativos.

COMPARACIÓN CON LABEL FLIPPING CON 80 % DE MUESTRAS ENVENENADAS Y 20 % DE NODOS MALICIOSOS (B2.5 VS B2.6)

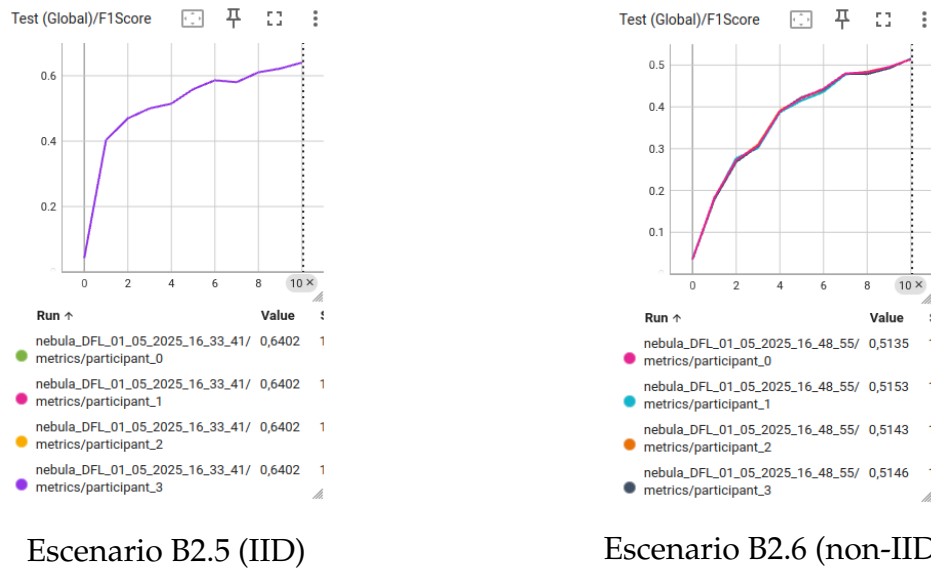


Figura 2.5: Comparación de Test F1 Score Global entre B2.5 y B2.6

En la **Figura 2.5** se muestra la evolución del F1 score global en test en los escenarios B2.5 (IID) y B2.6 (non-IID), ambos bajo un ataque de Label Flipping con un 20% de nodos maliciosos. En el caso IID (B2.5), el ataque tiene un impacto limitado, ya que, si bien es cierto que el crecimiento del F1 score se ralentiza a partir de la ronda 1, el rendimiento del modelo continúa aumentando de forma sostenida, sin evidencias de degradación significativa.

En el escenario non-IID (B2.6) el comportamiento es similar, pero con un crecimiento del F1 score global más lento desde las primeras rondas. Sin embargo, es probable que esto se deba a la propia naturaleza de la distribución non-IID, y no necesariamente al ataque, pues esta distribución introduce heterogeneidad entre nodos y puede dificultar la convergencia. En conjunto, estos resultados sugieren que, en estos escenarios y bajo las condiciones del experimento, el ataque de Label Flipping con un 20% de nodos maliciosos no logra comprometer significativamente el rendimiento del modelo global.

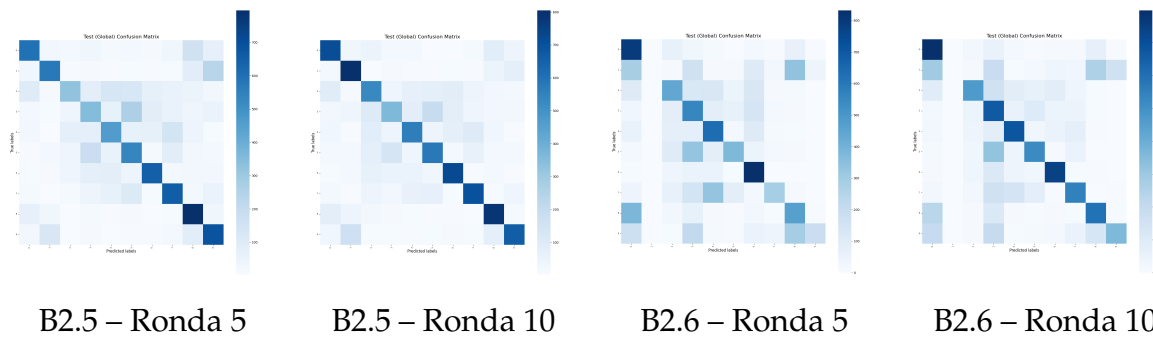
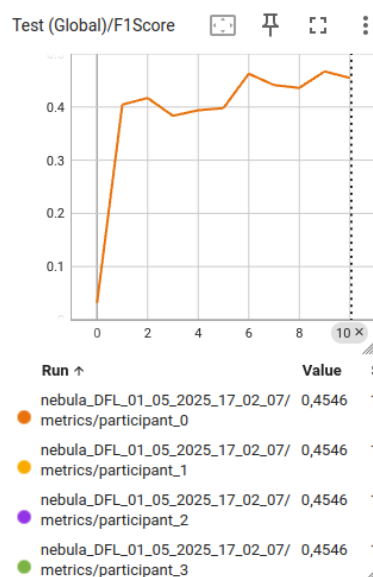


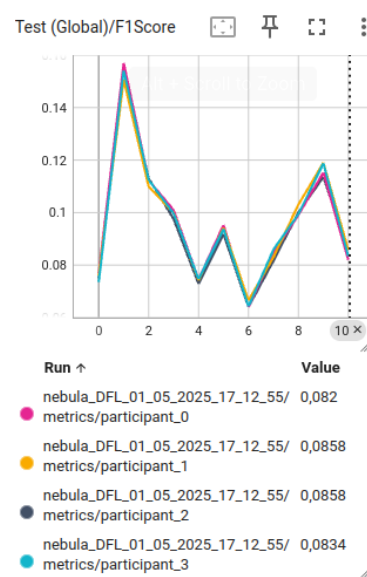
Figura 2.6: Comparación de matrices de confusión entre B2.5 (IID) y B2.6 (non-IID) en rondas 5 y 10.

En la **Figura 2.6** se presentan las matrices de confusión correspondientes a los escenarios B2.5 (IID) y B2.6 (non-IID) en las rondas 5 y 10. En ambos escenarios, se aprecia que el ataque de Label Flipping no logra deteriorar significativamente el rendimiento del modelo global, porque las matrices muestran un rendimiento aceptable que, de hecho, mejora de la ronda 5 a la 10, pues aumenta la cantidad de verdaderos positivos de una ronda a la otra. Esta evolución sugiere que el modelo sigue aprendiendo a pesar de la presencia de nodos maliciosos, lo cual refuerza la conclusión de que el ataque, bajo estas condiciones, no es lo suficientemente eficaz.

COMPARACIÓN CON LABEL FLIPPING CON 80 % DE MUESTRAS ENVENENADAS Y 50 % DE NODOS MALICIOSOS (B2.7 vs B2.8)



Escenario B2.7 (IID)



Escenario B2.8 (non-IID)

Figura 2.7: Comparación de Test F1 Score Global entre B2.7 y B2.8

En la **Figura 2.7** se muestra la evolución del F1 score global en test para los escenarios B2.7 (IID) y B2.8 (non-IID), ambos bajo un ataque de Label Flipping con un 50 % de nodos maliciosos. En el caso IID (B2.7), el ataque logra detener el crecimiento del F1 score a partir de la ronda 1, aunque sin provocar una caída pronunciada. El rendimiento se estabiliza alrededor de 0,4, lo que indica una cierta resiliencia del sistema.

Sin embargo, el escenario non-IID (B2.8) muestra una mayor vulnerabilidad al ataque. El F1 score global experimenta una caída brusca entre las rondas 1 y 2 para todos los nodos por igual, y se mantiene estancado por debajo de 0,1 en las rondas posteriores. Esto sugiere que la combinación de una distribución no homogénea de datos y una alta proporción de nodos maliciosos amplifica la efectividad del ataque, comprometiendo gravemente la capacidad de generalización del modelo global.

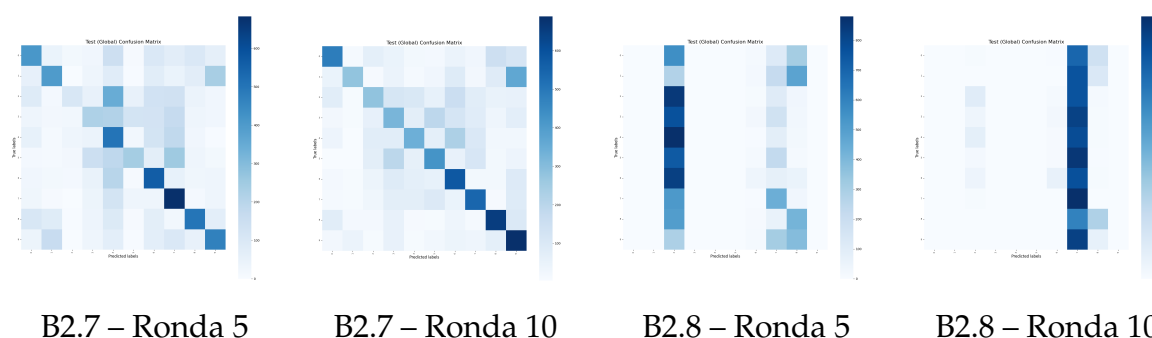


Figura 2.8: Comparación de matrices de confusión entre B2.7 (IID) y B2.8 (non-IID) en rondas 5 y 10.

Finalmente, en la **Figura 2.8** se presentan las matrices de confusión correspondientes a los escenarios B2.7 (IID) y B2.8 (non-IID) en las rondas 5 y 10. En el escenario B2.7, el ataque de Label Flipping no logra degradar gravemente el rendimiento del modelo global. Aunque se observa un deterioro en comparación con un entorno sin ataques, el modelo aún mantiene una capacidad de clasificación razonable. De hecho, entre las rondas 5 y 10 se produce una ligera mejora, pues se observa un aumento de verdaderos positivos y una reducción de errores de clasificación en varias clases.

En contraste, el escenario B2.8 evidencia un impacto mucho más severo del ataque. Las matrices de confusión muestran una alta proporción de falsos positivos y falsos negativos tanto en la ronda 5 como en la 10, lo que indica que el modelo no consigue recuperarse del efecto del ataque. Esto refuerza la conclusión de que el entorno non-IID, combinado con un alto porcentaje de nodos maliciosos, representa un escenario altamente vulnerable para los ataques de Label Flipping.

2.3.2. Comparaciones del efecto acumulativo de los ataques

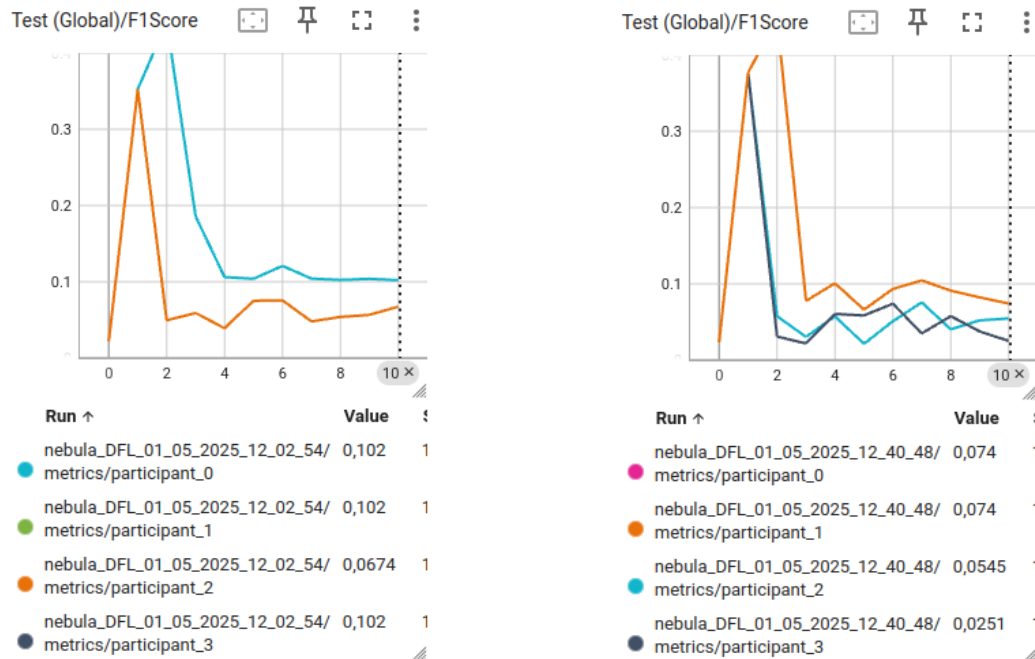
Para evaluar el impacto de la exposición prolongada a ataques, se analiza la evolución del F1-score global en función del número de rondas de entrenamiento en diferentes escenarios, con variaciones tanto en el tipo de ataque como en el porcentaje de nodos maliciosos (20 % y 50 %). El objetivo es observar si el modelo logra estabilizar su rendimiento, es decir, su convergencia, y si se aprecian efectos acumulativos del ataque a lo largo del tiempo. De esta forma, este apartado nos permitirá responder a la pregunta:

PREGUNTA 2

¿Cómo afecta la exposición prolongada a los ataques en la velocidad de convergencia del modelo a lo largo de múltiples rondas? ¿Hay signos de efectos acumulativos de los ataques con el tiempo?

RESPUESTA:

COMPARACIÓN CON MODEL POISONING CON 10% DE RUIDO GAUSSIANO Y DATOS IID (B2.1 vs B2.3)



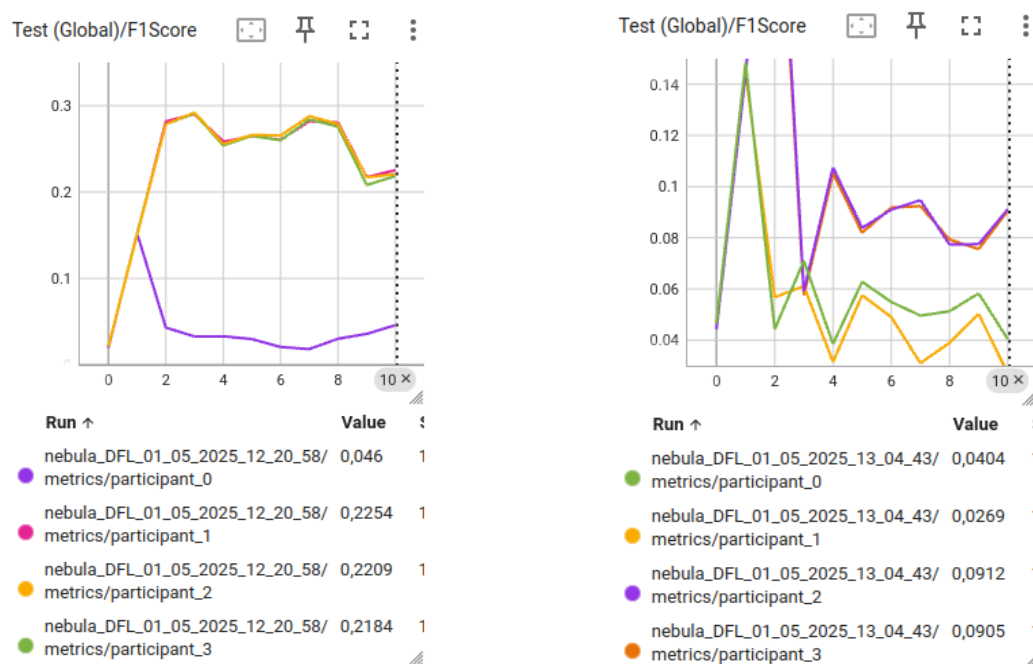
Escenario B2.1 (20 % nodos maliciosos) Escenario B2.3 (50 % nodos maliciosos)

Figura 2.9: Comparación de Test F1 Score Global entre B2.1 y B2.3

En la **Figura 2.9** se muestra la evolución del F1 score global en test en los escenarios B2.1 (20 % de nodos maliciosos) y B2.3 (50 % de nodos maliciosos) bajo un ataque de Model Poisoning. En B2.1, donde el nodo malicioso es el naranja, se observa que el F1-score de este nodo sufre una caída abrupta entre las rondas 1 y 2, lo que indica la ejecución del ataque mediante ruido gaussiano. Posteriormente, se observa una degradación progresiva del rendimiento en los nodos restantes, que finalmente se estabilizan ligeramente por encima de 0,1, reflejando cómo la agregación de las actualizaciones envenenadas afecta al modelo global a lo largo del tiempo.

Por otro lado, en B2.3, donde los nodos maliciosos son el azul y el gris, se aprecia que el F1 score de estos dos nodos sufre una caída abrupta entre las rondas 1 y 2, al realizarse el ataque mediante ruido gaussiano. Posteriormente, se observa una degradación del rendimiento en los nodos restantes, que finalmente se estabilizan ligeramente por debajo de 0,1. Así, el ataque con 2 nodos maliciosos afecta de forma más pronunciada el desempeño de los otros nodos, aunque la diferencia con respecto a usar un único nodo malicioso no es demasiado grande.

COMPARACIÓN CON MODEL POISONING CON 10% DE RUIDO GAUSSIANO Y DATOS NON-IID (B2.2 vs B2.4)



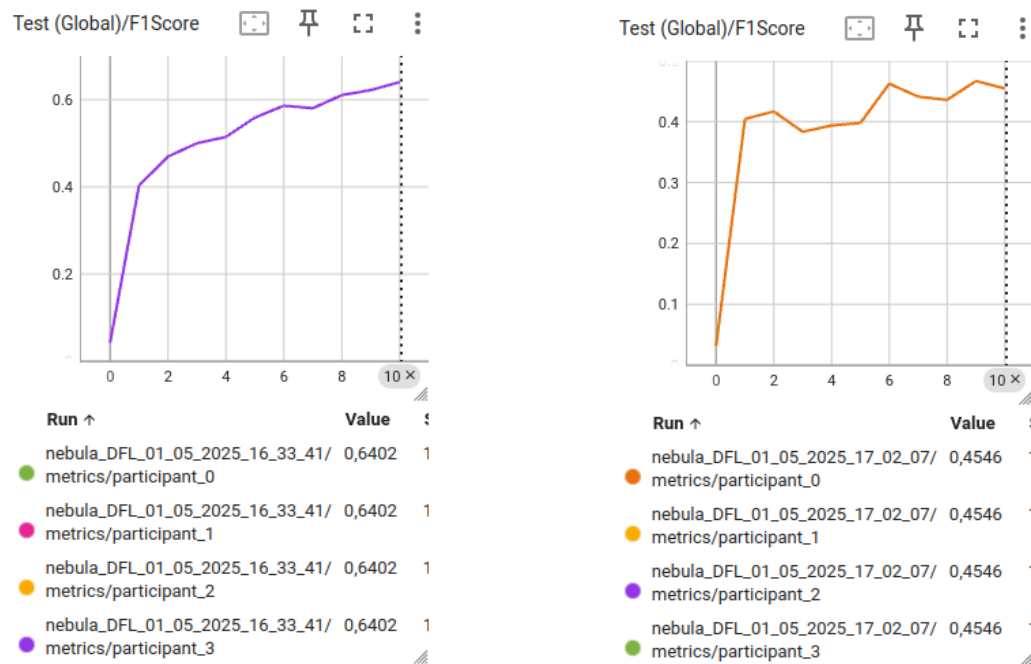
Escenario B2.2 (20 % nodos maliciosos) Escenario B2.4 (50 % nodos maliciosos)

Figura 2.10: Comparación de Test F1 Score Global entre B2.2 y B2.4

En la **Figura 2.10** se muestra la evolución del F1 score global en test en los escenarios B2.2 (20% de nodos maliciosos) y B2.4 (50% de nodos maliciosos), ambos bajo un ataque de Model Poisoning. En el escenario B2.2, el nodo morado ejecuta el ataque desde las primeras rondas. Aunque su F1 score cae abruptamente, el impacto sobre los nodos benignos es más moderado. En estos, el crecimiento del F1 score se detiene por culpa del ataque, pero no experimenta una caída significativa. Esto sugiere que con solo un nodo malicioso, el ataque tiene una efectividad limitada sobre el modelo global.

En cambio, en el escenario B2.4, dos nodos (amarillo y verde) participan en el ataque desde el inicio. Sus F1 scores se desploman en la ronda 2, y este efecto se transmite rápidamente a los nodos benignos, cuyo rendimiento también cae a partir de la ronda 3. Aunque en las siguientes rondas se observa una leve recuperación, el F1 global se estabiliza finalmente en valores inferiores a 0,1, lo que evidencia un deterioro severo del modelo. Este escenario demuestra que aumentar la proporción de nodos maliciosos intensifica significativamente el impacto del ataque y reduce la capacidad del modelo para recuperarse.

COMPARACIÓN CON LABEL FLIPPING CON 80 % DE MUESTRAS ENVENENADAS Y DATOS IID (B2.5 vs B2.7)



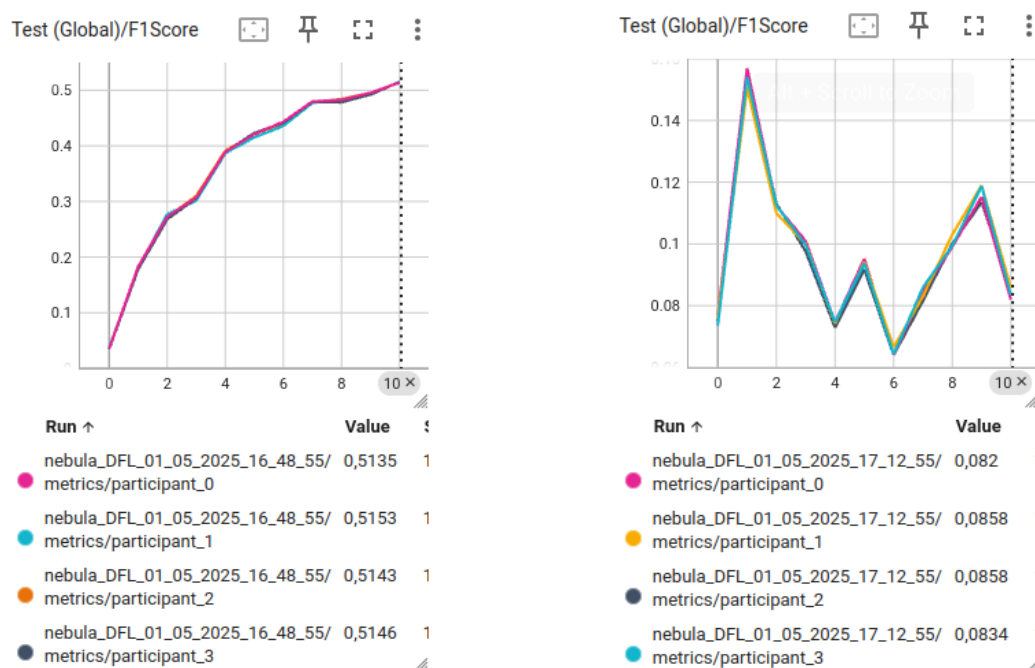
Escenario B2.5 (20 % nodos maliciosos) Escenario B2.7 (50 % nodos maliciosos)

Figura 2.11: Comparación de Test F1 Score Global entre B2.5 y B2.7

En la **Figura 2.11** se muestra la evolución del F1 score global en test en los escenarios B2.5 (20 % de nodos maliciosos) y B2.6 (50 % de nodos maliciosos), ambos bajo un ataque de Label Flipping. En B2.5, el ataque tiene un impacto limitado pues, aunque el crecimiento del F1 score se ralentiza a partir de la ronda 1, el rendimiento del modelo continúa mejorando de forma sostenida a lo largo de las rondas, alcanzando un valor de 0,6402 en la última ronda. No se aprecian signos claros de degradación, lo que sugiere una cierta tolerancia del sistema ante una proporción reducida de nodos maliciosos.

Por otro lado, en el escenario B2.7, con un 50 % de nodos maliciosos, el ataque logra detener el crecimiento del F1 score a partir de la ronda 1. Aunque no se produce una caída pronunciada, el rendimiento global se estabiliza en torno a 0,4, lo que evidencia un impacto más significativo en la capacidad de generalización del modelo, aunque todavía con cierta resistencia frente al ataque.

COMPARACIÓN CON LABEL FLIPPING CON 80 % DE MUESTRAS ENVENENADAS Y DATOS NON-IID (B2.6 vs B2.8)



Escenario B2.6 (20 % nodos maliciosos)

Escenario B2.8 (50 % nodos maliciosos)

Figura 2.12: Comparación de Test F1 Score Global entre B2.6 y B2.8

En la **Figura 2.12** se muestra la evolución del F1 score global en test en los escenarios B2.7 (20 % de nodos maliciosos) y B2.8 (50 % de nodos maliciosos), ambos bajo un ataque de Label Flipping. En el escenario B2.6 se observa un crecimiento del F1 score global algo lento desde las primeras rondas. No obstante, esta ralentización parece deberse más a la distribución non-IID de los datos, que introduce heterogeneidad entre nodos y puede dificultar la convergencia, que al propio ataque, ya que no se observa una desaceleración repentina ni una caída significativa del rendimiento.

Por el contrario, el escenario B2.8 presenta una mayor vulnerabilidad al ataque. El F1 score global experimenta una caída brusca entre las rondas 1 y 2 para todos los nodos por igual, y permanece estancado por debajo de 0,1 en las rondas siguientes. Esto indica que la combinación de una distribución no homogénea de datos y una alta proporción de nodos maliciosos potencia la efectividad del ataque, comprometiendo gravemente la capacidad de generalización del modelo global.

2.3.3. Contramedidas para detectar y mitigar los ataques

En este apartado se responde a la pregunta:

PREGUNTA 2

¿Qué posibles contramedidas se podrían implementar para detectar y mitigar estos ataques? ¿Son ciertas arquitecturas de modelos más resistentes a ataques específicos? En caso afirmativo, ¿por qué?

RESPUESTA:

Para mitigar y detectar ataques en un entorno de Federated Learning (FL), como los ataques de Model Poisoning y Label Flipping, existen diversas contramedidas que se pueden implementar con este fin. Algunas técnicas relevantes en este sentido son:

- **Técnicas de Agregación Robusta:** Se utilizan para filtrar o limitar la influencia de actualizaciones atípicas, con el fin de mitigar el efecto de las contribuciones envenenadas. Algunas de las más comunes son:
 - **Trimmed Mean:** Elimina los valores extremos y agrega el resto.
 - **Median Aggregation:** Toma la mediana en lugar de promediar todas las actualizaciones.
 - **Krum:** Selecciona la actualización más cercana a un consenso mayoritario.
 - **Gradient Clipping:** Limita la magnitud de las actualizaciones para evitar modificaciones extremas.

Aunque estas técnicas son efectivas para mitigar ataques, también pueden afectar las actualizaciones legítimas si se implementan de manera demasiado agresiva. El desafío radica en encontrar un equilibrio adecuado.

- **Defensas a Nivel de Cliente:** Este enfoque se basa en asignar un puntaje de confianza a cada cliente y detectar anomalías en sus actualizaciones. Las técnicas que se pueden utilizar incluyen:
 - **Sistemas de Reputación:** Se confía más en los clientes con comportamientos consistentes.
 - **Detección de Anomalías:** Utiliza técnicas estadísticas para señalar actualizaciones sospechosas.
 - **Rechazo de Outliers:** Identifica y elimina actualizaciones que se desvían de la norma.

Estas técnicas pueden ser propensas a cometer errores. Por ejemplo, los clientes nuevos o subrepresentados pueden ser penalizados injustamente por su comportamiento atípico, aunque no sean maliciosos.

- **Privacidad Diferencial:** Preserva la privacidad agregando ruido a los gradientes, dificultando que los atacantes influyan en el modelo. Algunas

características de esta técnica son:

- Los clientes agregan ruido aleatorio a sus gradientes antes de enviarlos. Esto asegura que ninguna actualización individual tenga un impacto desproporcionado en el modelo global.
- El servidor aplica técnicas de agregación robusta para suavizar el impacto de las actualizaciones envenenadas y mejorar la resistencia del modelo frente a los ataques.

Sin embargo, si se agrega demasiado ruido, la precisión del modelo podría verse afectada negativamente, ya que las actualizaciones legítimas también pueden perder precisión.

Ahora, con base en los análisis previos de los diferentes escenarios y ataques, podemos identificar qué arquitecturas de modelos parecen ser más resistentes a ataques específicos. Para determinar cuáles son las más resistentes, observaremos cada uno de los escenarios, centrándonos en aquellos que mostraron un comportamiento más robusto frente a la degradación del rendimiento debido a los ataques, y consideraremos tanto la naturaleza de los ataques como el porcentaje de nodos maliciosos presentes en cada escenario. Recordamos que los escenarios que mostraron una mayor robustez frente a los ataques fueron:

■ B2.2:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** FedAvg
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 20 %

Las razones por las que este escenario podría estar mostrando una mayor resistencia incluyen:

- **Distribución non-IID:** Aunque la distribución non-IID podría dificultar la convergencia en términos generales, también puede ayudar a que el modelo sea más robusto frente a ataques, ya que los datos heterogéneos pueden hacer que las actualizaciones de los nodos sean más variadas y menos predecibles para un atacante.
- **Ataque de Model Poisoning con ruido gaussiano (10 %) y 20 % de nodos maliciosos:** El ruido gaussiano puede tener un impacto limitado si se utiliza con un porcentaje relativamente bajo de nodos maliciosos, ya que la mayoría de las actualizaciones provienen de nodos benignos. Esto permite que el modelo global aún se beneficie de la mayoría de las actualizaciones legítimas, minimizando el impacto del ataque. Además, el algoritmo de agregación FedAvg puede ayudar a mitigar los efectos

del ataque, ya que al promediar las actualizaciones, reduce el impacto del ruido introducido por los nodos maliciosos.

■ **B2.5:**

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** IID
- **Algoritmo de agregación:** FedAvg
- **Ataque:** Label Flipping con 80 % de muestras envenenadas
- **Porcentaje de nodos maliciosos:** 20 %

La principal razón por la que este escenario podría estar mostrando una mayor resistencia es:

- **Ataque de Label Flipping con 80 % de muestras envenenadas y 20 % de nodos maliciosos:** Un porcentaje bajo de nodos maliciosos permite que el sistema mantenga un cierto control sobre las actualizaciones del modelo, pues el ataque puede ser parcialmente mitigado por el algoritmo de agregación FedAvg, lo que genera un modelo con una alta tolerancia frente al ataque de Label Flipping.

■ **B2.6:**

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** FedAvg
- **Ataque:** Label Flipping con 80 % de muestras envenenadas
- **Porcentaje de nodos maliciosos:** 20 %

Las razones por las que este escenario podría estar mostrando una mayor resistencia incluyen:

- **Distribución non-IID:** Al igual que en B2.2, la distribución non-IID introduce heterogeneidad en los datos, lo que puede dificultar la convergencia, pero también hace que las actualizaciones de los nodos sean más diversas, ayudando a que el sistema sea más resistente a ataques.
- **Ataque de Label Flipping con 80 % de muestras envenenadas y 20 % de nodos maliciosos:** Al igual que en B2.5, debido al bajo porcentaje de nodos maliciosos, el impacto del ataque se limita considerablemente. Las actualizaciones benignas consiguen mantener el sistema funcionando con un rendimiento relativamente alto.

3. Boletín III

3.1. Introducción a la tarea y ejercicios a resolver

Para este boletín, se deben abordar los siguientes **ejercicios**:

1. Utilizar la configuración de model poisoning definida en la Sesión II (usando FedAvg, IID/Non-IID).
2. Configurar la misma con los siguientes algoritmos de agregación:
 - Trimmed Mean
 - Krum
3. Comparar el F1-score entre los experimentos y proporciona las matrices de confusión para cada ronda.

Con lo que se pretende conseguir:

- Comprender el papel de los algoritmos de agregación en el mantenimiento de la robustez e integridad en el Aprendizaje Federado.
- Explorar la implementación de FedAvg, Median, Trimmed Mean y Krum dentro del entorno NEBULA.
- Analizar cómo cada algoritmo maneja condiciones adversas, como el envenenamiento del modelo y la corrupción de datos.
- Evaluar el rendimiento y la resiliencia de los sistemas de Aprendizaje Federado bajo diversas configuraciones de agregación y modelos de amenaza.

3.2. Preguntas a responder

Según los resultados de los experimentos, responder las siguientes **preguntas**:

1. ¿Qué algoritmo demostró mayor resiliencia frente a actualizaciones adversarias, y por qué?
2. ¿Algún método de agregación sobreajustó o tuvo bajo rendimiento en clases específicas? ¿Puede esto correlacionarse con la presencia de clientes maliciosos?
3. ¿Cómo cambiaría el rendimiento si los ataques fueran más sigilosos (por ejemplo, manipulación de gradientes)?

3.3. Desarrollo de la tarea

Los **escenarios de federación** que entrenaremos en NEBULA para esta tarea son los siguientes:

Escenario B3.1:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** IID
- **Algoritmo de agregación:** Trimmed Mean
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 20 %

Escenario B3.2:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** Trimmed Mean
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 20 %

Escenario B3.3:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** IID
- **Algoritmo de agregación:** Trimmed Mean
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 50 %

Escenario B3.4:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** Trimmed Mean
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 50 %

Escenario B3.5:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** IID
- **Algoritmo de agregación:** Krum
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 20 %

Escenario B3.6:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** Krum
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 20 %

Escenario B3.7:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** IID
- **Algoritmo de agregación:** Krum
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 50 %

Escenario B3.8:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Algoritmo de agregación:** Krum
- **Ataque:** Model Poisoning con 10 % de ruido gaussiano
- **Porcentaje de nodos maliciosos:** 50 %

3.3.1. Análisis de los escenarios

ANÁLISIS DE B3.1 (IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 20% DE NODOS MALICIOSOS, TRIMMED MEAN)

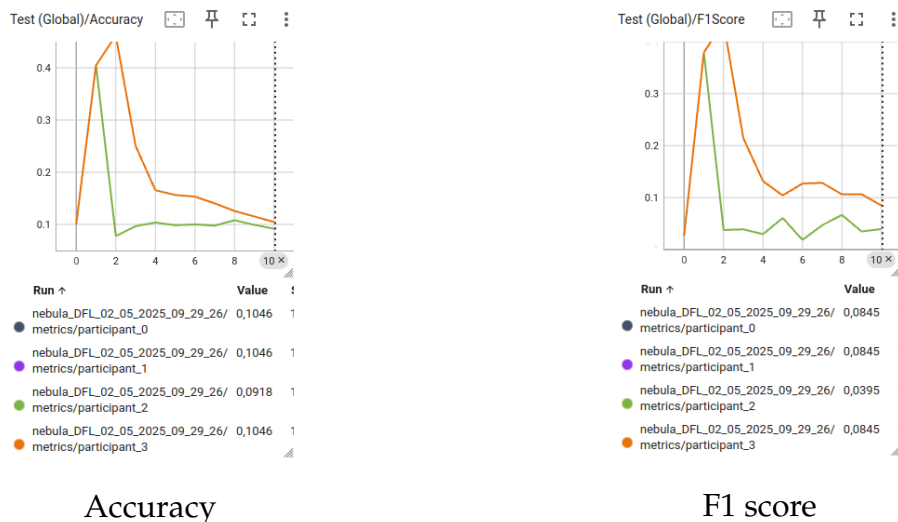


Figura 3.1: Métricas en test en el escenario B3.1

En la **Figura 3.1** se muestra el accuracy y el F1 score en el conjunto de test para el escenario B3.1. En ambas métricas, se puede observar una caída abrupta en el rendimiento del nodo verde, que parece ser el nodo malicioso, entre la ronda 1 y la ronda 2. Se observa que tras este ataque, el resto de nodos también experimentan una disminución de su desempeño para ambas métricas, por lo que parece que el algoritmo de agregación Trimmed Mean no está logrando mitigar eficazmente los efectos del ataque. Esta caída generalizada podría indicar que el algoritmo de agregación no filtra adecuadamente las actualizaciones envenenadas, afectando tanto al nodo malicioso como a los nodos benignos.

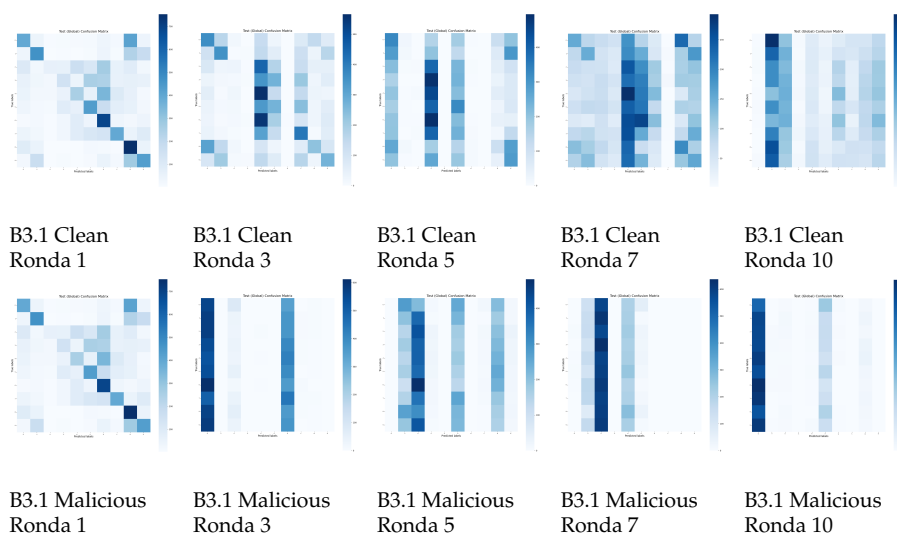


Figura 3.2: Matrices de confusión de B3.1: en nodos **Clean** (↑) y **Malicious** (↓).

Algo similar se observa en la **Figura 3.2**, donde se presentan las matrices de confusión para el escenario B3.1 en las rondas 1, 3, 5, 7 y 10, tanto para los nodos benignos como para el malicioso. En la ronda 1, los resultados son aceptables en todos los nodos. Sin embargo, a partir de la ronda 3, el nodo malicioso ya ha ejecutado el ataque, lo que provoca una caída significativa en su rendimiento. Posteriormente, el desempeño de los nodos benignos también comienza a decaer. Aunque en la ronda 3 la matriz de confusión todavía muestra un desempeño hasta cierto punto aceptable, en las siguientes rondas se observa un aumento considerable en los falsos positivos y falsos negativos, lo que indica una pérdida significativa de precisión en las predicciones.

ANÁLISIS DE B3.2 (NON-IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 20% DE NODOS MALICIOSOS, TRIMMED MEAN)

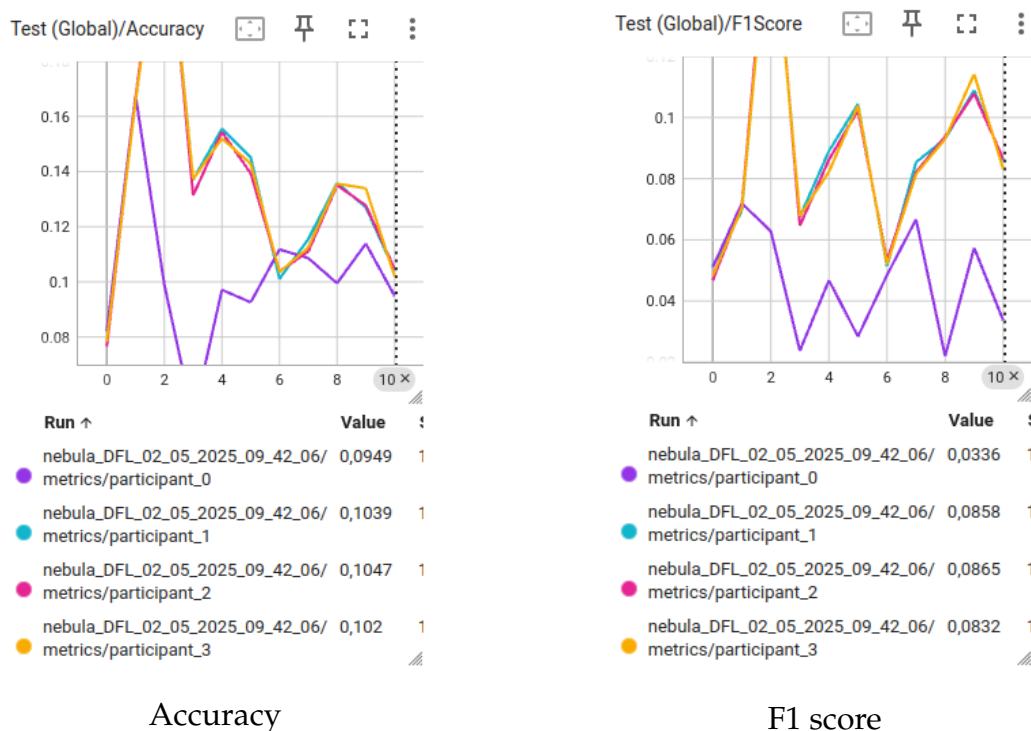


Figura 3.3: Métricas en test en el escenario B3.2

En la **Figura 3.3** se muestra el accuracy y el F1 score en el conjunto de test para el escenario B3.2. Ambas métricas revelan una caída abrupta en el rendimiento del nodo morado, que parece ser el nodo malicioso, entre las rondas 1 y 2. Tras este descenso, los nodos benignos también sufren una degradación en su desempeño, aunque menos pronunciada. En las rondas posteriores, las métricas de los nodos benignos comienzan a oscilar, y en algunas rondas logran recuperar ligeramente el rendimiento, pero luego vuelven a decaer. Esto sugiere que el algoritmo de agregación Trimmed Mean no está mitigando eficazmente el impacto del ataque, probablemente porque no filtra adecuadamente las actualizaciones envenenadas, afectando tanto al nodo malicioso como a los benignos.

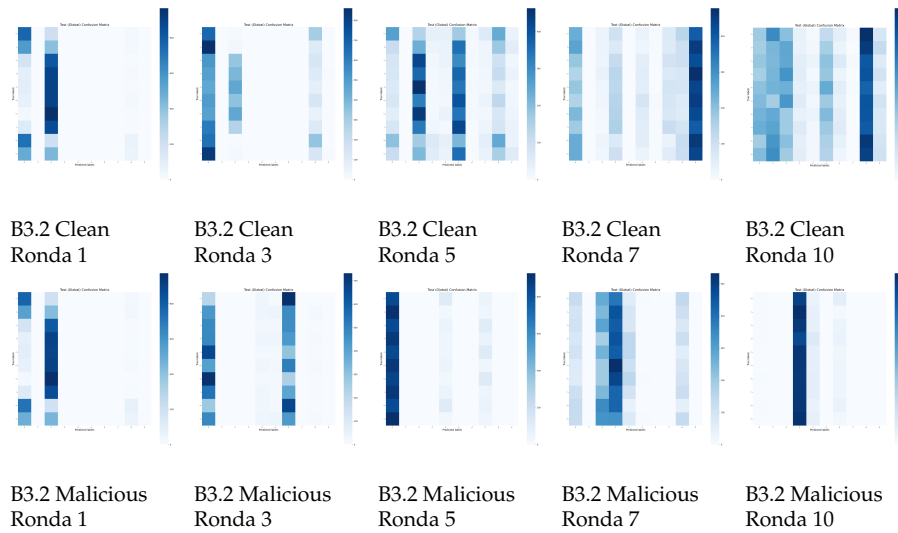


Figura 3.4: Matrices de confusión de B3.2: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.4** se muestran las matrices de confusión para el escenario B3.2 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y el malicioso. Se observa que para todos los nodos, el desempeño es bastante deficiente desde el principio. En la primera ronda solo se logran predecir correctamente las clases 0 y 2, pero a raíz del ataque, el rendimiento en todos los nodos decae todavía más.

ANÁLISIS DE B3.3 (IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 50% DE NODOS MALICIOSOS, TRIMMED MEAN)

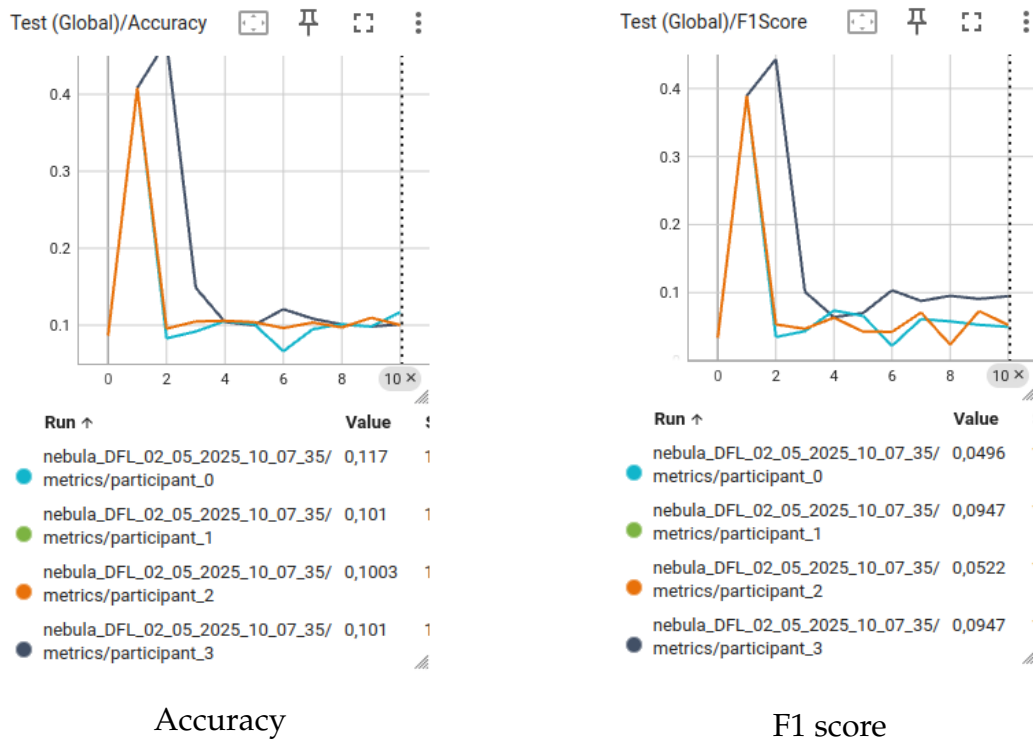


Figura 3.5: Métricas en test en el escenario B3.3

En la **Figura 3.5** se muestra el accuracy y el F1 score en el conjunto de test para el escenario B3.3. En ambas métricas, se puede observar una caída abrupta en el rendimiento de los nodos naranja y azul, que parecen ser los nodos maliciosos, entre la ronda 1 y la ronda 2. Se observa que tras este ataque, el resto de nodos también experimentan una disminución bastante considerable de su desempeño para ambas métricas, por lo que parece que el algoritmo de agregación Trimmed Mean no está logrando mitigar eficazmente los efectos del ataque. Esto tiene sentido, pues ya habíamos visto antes que en B3.1, donde solo había un nodo malicioso, tampoco lo lograba, por lo que en B3.3, con dos nodos maliciosos, era todavía menos probable.

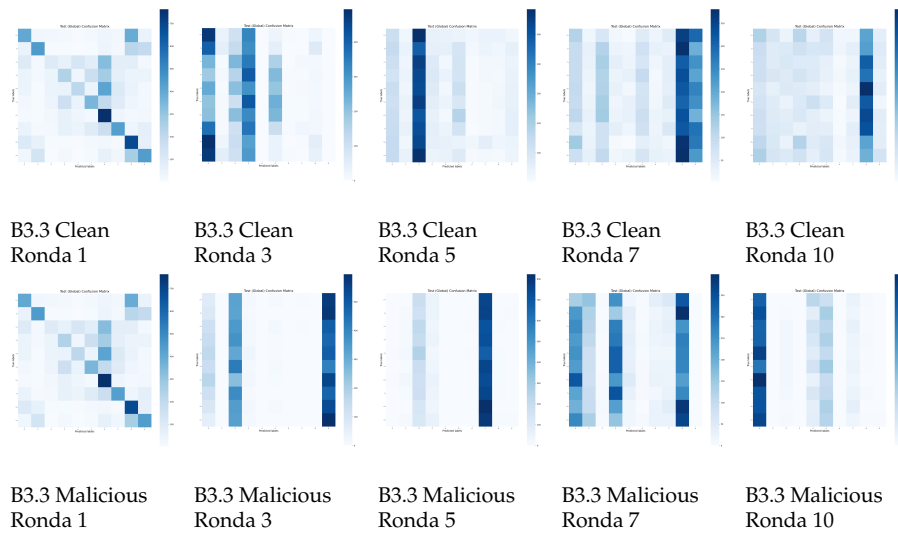


Figura 3.6: Matrices de confusión de B3.3: en nodos **Clean** (↑) y **Malicious** (↓).

Algo similar se observa en la **Figura 3.6**, donde se presentan las matrices de confusión para el escenario B3.3 en las rondas 1, 3, 5, 7 y 10, tanto para los nodos benignos como para el malicioso. En la ronda 1, los resultados son aceptables en todos los nodos. Sin embargo, a partir de la ronda 3, el nodo malicioso ya ha ejecutado el ataque, lo que provoca una caída significativa en su rendimiento. A partir de esta ronda también comienza a decaer el desempeño de los nodos benignos, observándose un aumento considerable en los falsos positivos y falsos negativos. Esta caída generalizada parece indicar que el algoritmo de agregación no consigue filtrar adecuadamente las actualizaciones envenenadas, afectando tanto al nodo malicioso como a los nodos benignos.

ANÁLISIS DE B3.4 (NON-IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 50 % DE NODOS MALICIOSOS, TRIMMED MEAN)

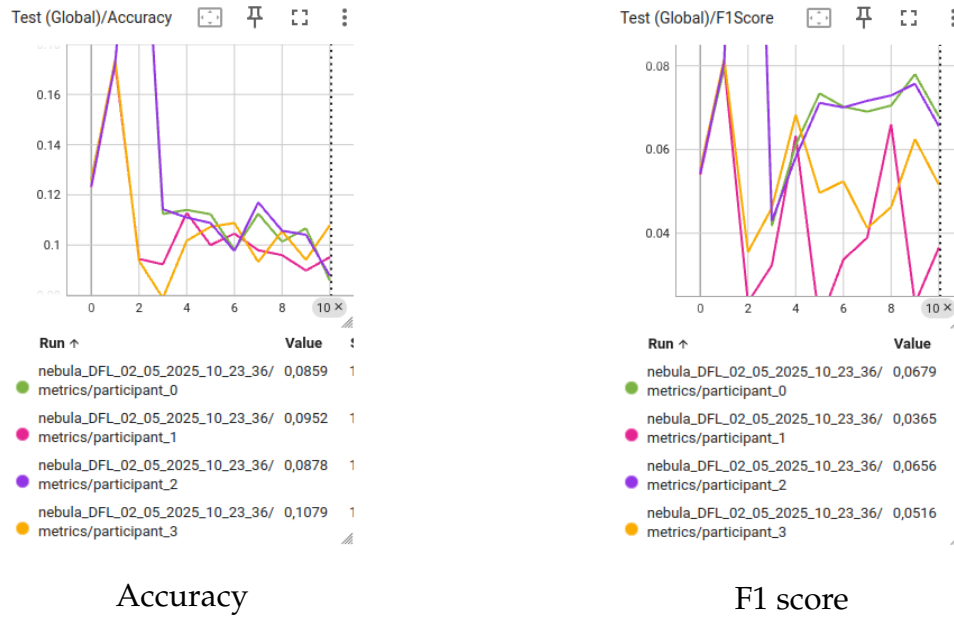


Figura 3.7: Métricas en test en el escenario B3.4

En la **Figura 3.7** se muestra el accuracy y el F1 score en el conjunto de test para el escenario B3.4. Para ambas métricas, se puede observar una caída abrupta en el rendimiento de los nodos amarillo y rosa, que parecen ser los nodos maliciosos, entre la ronda 1 y la ronda 2. Se observa que tras este ataque, el resto de nodos también experimentan una disminución de su desempeño para ambas métricas, por lo que de nuevo, el algoritmo de agregación Trimmed Mean no está logrando mitigar eficazmente los efectos del ataque. Esto es lógico, ya que si en B3.2, con un solo nodo mailicioso, no se lograba mitigar dicho efecto, en B3.4 con dos nodos maliciosos, todavía era menos probable.

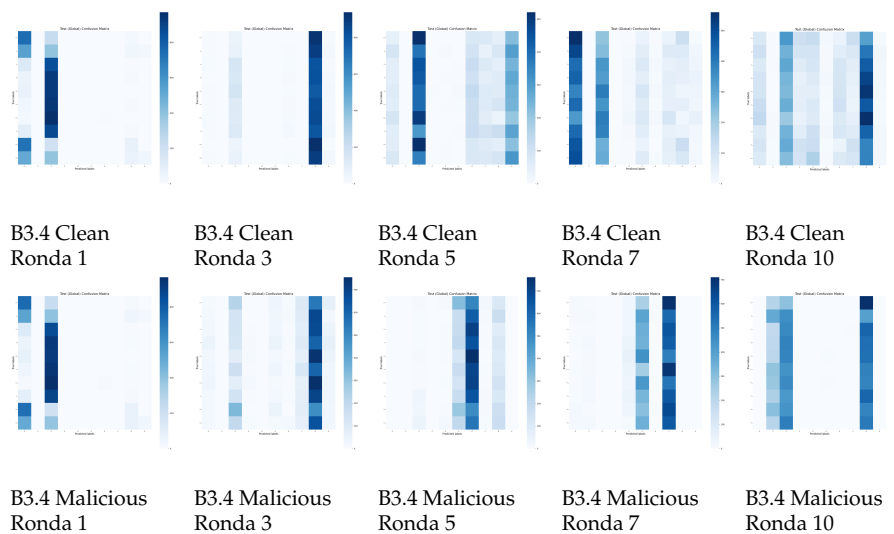


Figura 3.8: Matrices de confusión de B3.4: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.8** se muestran las matrices de confusión para el escenario B3.4 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y los maliciosos. Se observa que para todos los nodos, el desempeño es bastante deficiente desde el principio. En la primera ronda solo se logran predecir correctamente las clases 0 y 2, pero a raíz del ataque, el rendimiento en todos los nodos decae todavía más.

ANÁLISIS DE B3.5 (IID, MODEL POISONING CON 10 % DE RUIDO GAUSSIANO, 20 % DE NODOS MALICIOSOS, KRUM)

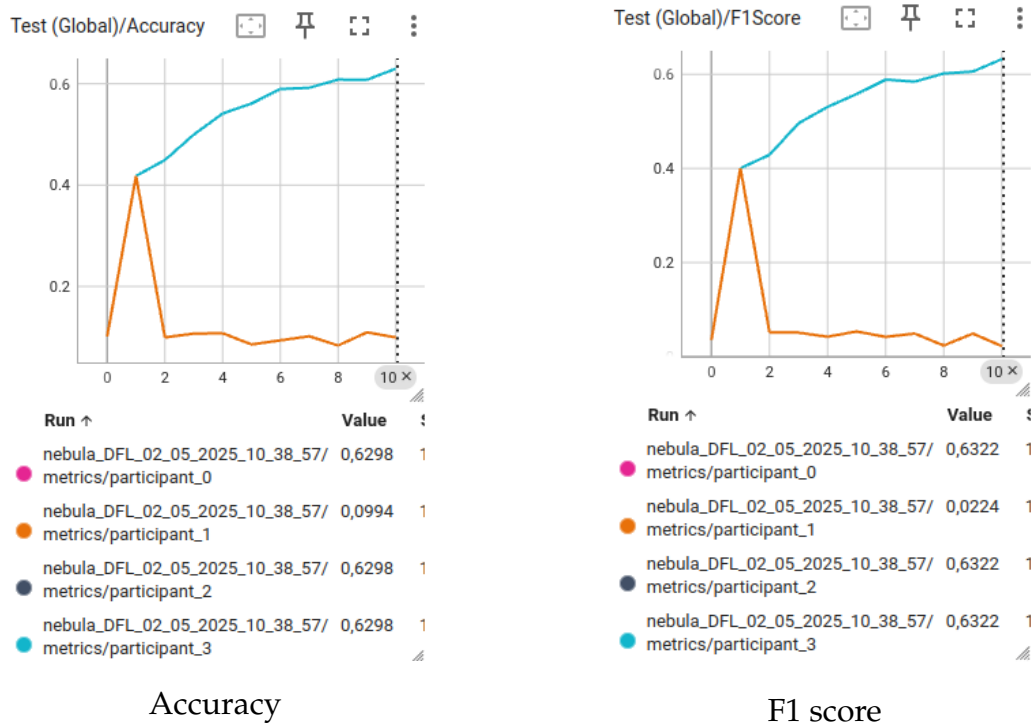


Figura 3.9: Métricas en test en el escenario B3.5

En la **Figura 3.9** se muestran el accuracy y el F1 score en el conjunto de test para el escenario B3.5. Ambas métricas presentan una caída abrupta en el rendimiento del nodo naranja, probablemente malicioso, entre las rondas 1 y 2. Sin embargo, se observa que tras este ataque, los nodos benignos mantienen un desempeño sólido y, aunque el crecimiento de las métricas se ralentiza ligeramente, este no se detiene del todo, sino que continúa aumentando de forma sostenida en las siguientes rondas. Esto indica que el algoritmo de agregación de Krum logra mitigar eficazmente los efectos del ataque en los nodos benignos. En este escenario, la estrategia de seleccionar la actualización más cercana a un consenso mayoritario demuestra ser efectiva frente a ataques de Model Poisoning en este escenario, al menos cuando el porcentaje de nodos maliciosos es reducido.

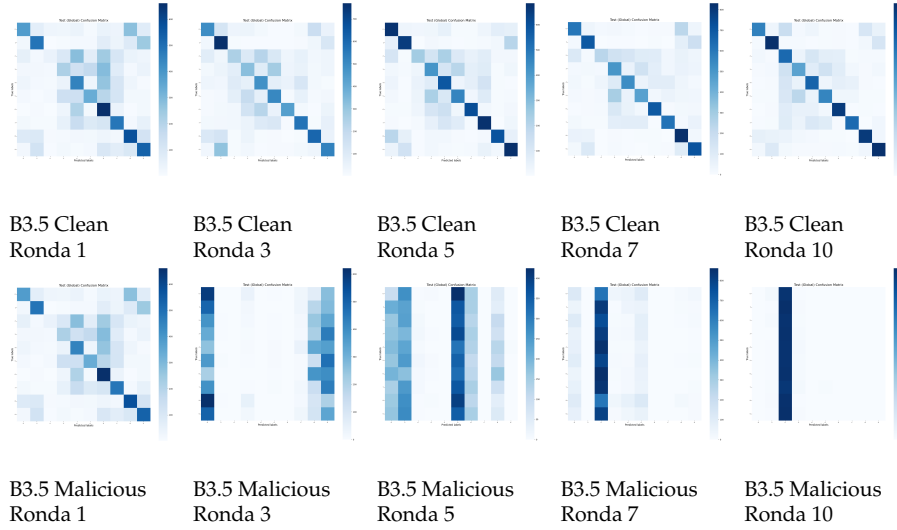
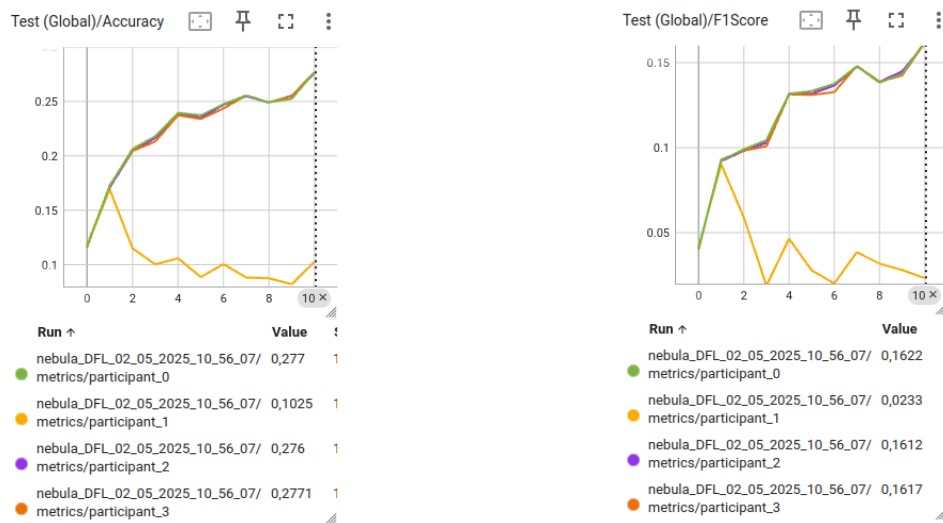


Figura 3.10: Matrices de confusión de B3.5: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.10** se muestran las matrices de confusión para el escenario B3.5 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y el malicioso. Se observa que el nodo malicioso tiene un desempeño aceptable en la primera ronda, pero tras ejecutar el ataque, su rendimiento decae notablemente en las rondas siguientes, con una clara degradación en la capacidad de clasificación. Sin embargo, los nodos benignos logran mantener un buen desempeño a lo largo de todas las rondas, y de hecho, en la ronda 10, se logra una clasificación precisa en prácticamente todas las clases, lo que refuerza la idea de que el algoritmo Krum ha logrado aislar eficazmente las actualizaciones maliciosas y preservar la integridad del modelo global.

ANÁLISIS DE B3.6 (NON-IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 20% DE NODOS MALICIOSOS, KRUM)



Accuracy

F1 score

Figura 3.11: Métricas en test en el escenario B3.6

En la **Figura 3.11** se muestran el accuracy y el F1 score en el conjunto de test para el escenario B3.6. Ambas métricas muestran una caída abrupta en el rendimiento del nodo amarillo, que probablemente sea malicioso, entre las rondas 1 y 2. No obstante, tras este ataque, los nodos benignos logran mantener un desempeño relativamente sólido. Aunque el crecimiento de sus métricas se ralentiza, las métricas no se detienen completamente y continúan mejorando con el tiempo. Es importante señalar que, si bien los nodos benignos no alcanzan valores muy altos en estas métricas, esto podría estar relacionado con la distribución non-IID de los datos. En cualquier caso, la continuación del aumento en ambas métricas sugiere que el algoritmo de agregación de Krum es efectivo para mitigar, al menos parcialmente, los efectos del ataque en los nodos benignos.

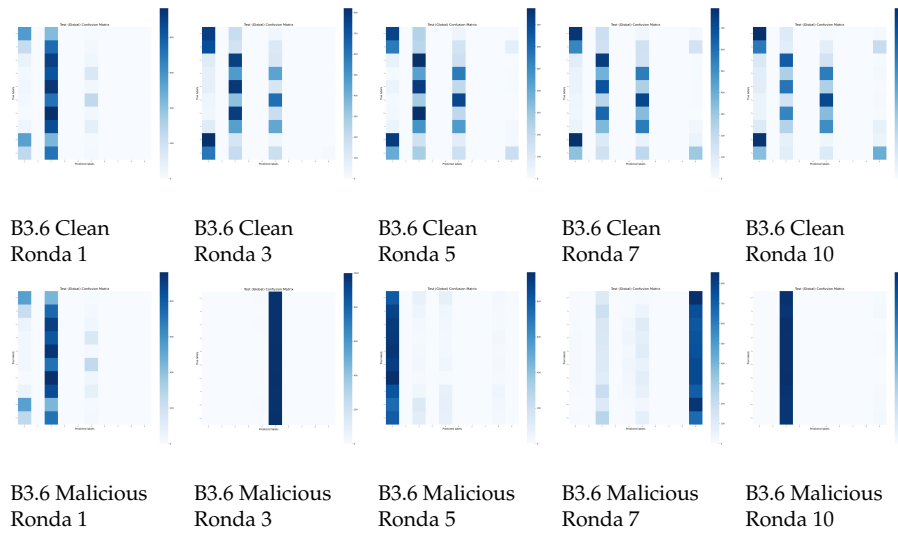


Figura 3.12: Matrices de confusión de B3.6: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.12** se muestran las matrices de confusión para el escenario B3.6 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y el malicioso. Se observa que el nodo malicioso tiene un mal desempeño desde la primera ronda, y que tras ejecutar el ataque, su rendimiento decae todavía más en las rondas siguientes, con una clara degradación en la capacidad de clasificación. Sin embargo, los nodos benignos logran mantener un desempeño algo más aceptable a lo largo de las rondas, aunque con algunas limitaciones, como ya hemos explicado antes. Sin embargo, en comparación con el nodo maligno, los nodos benignos logran clasificar correctamente varias de las clases, por lo que Krum parece estar siendo efectivo.

ANÁLISIS DE B3.7 (IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 50% DE NODOS MALICIOSOS, KRUM)

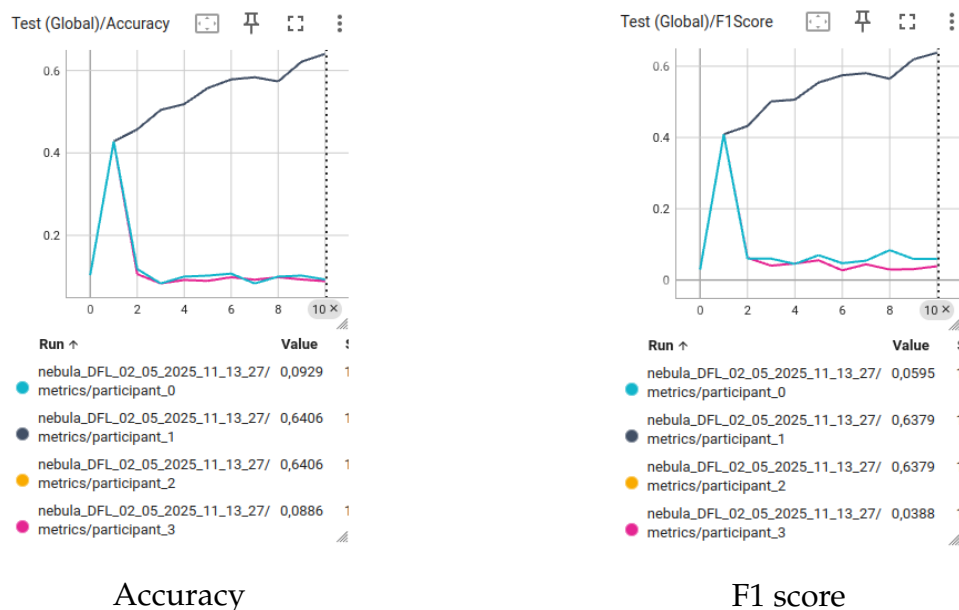


Figura 3.13: Métricas en test en el escenario B3.7

En la **Figura 3.13** se muestran el accuracy y el F1 score en el conjunto de test para el escenario B3.7. Ambas métricas presentan una caída abrupta en el rendimiento de los nodos azul y rosa, probablemente maliciosos, entre las rondas 1 y 2. Sin embargo, se observa que tras este ataque, los nodos benignos mantienen un desempeño sólido y, aunque el crecimiento de las métricas se ralentiza ligeramente, este no se detiene del todo, sino que continúa aumentando de forma sostenida en las siguientes rondas, al igual que ocurría en B3.5. Esto indica que el algoritmo de agregación de Krum logra mitigar eficazmente los efectos del ataque en los nodos benignos, incluso cuando el porcentaje de nodos maliciosos es más elevado.

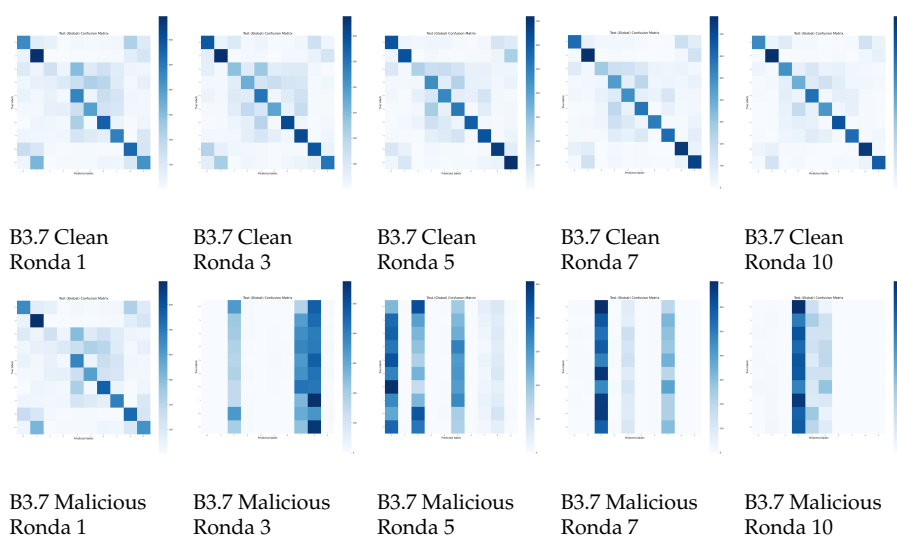


Figura 3.14: Matrices de confusión de B3.7: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.14** se muestran las matrices de confusión para el escenario B3.7 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y los maliciosos. Se observa que los nodos maliciosos tienen un desempeño aceptable en la primera ronda, pero tras ejecutar el ataque, su rendimiento decae notablemente en las rondas siguientes, con una clara degradación en la capacidad de clasificación. Sin embargo, los nodos benignos logran mantener un buen desempeño a lo largo de todas las rondas, consiguiendo en la ronda 10 una clasificación precisa en prácticamente todas las clases. Esto refuerza la idea de que el algoritmo Krum logra aislar eficazmente las actualizaciones maliciosas y preservar la integridad del modelo global, incluso con una proporción más elevada de nodos maliciosos.

ANÁLISIS DE B3.8 (NON-IID, MODEL POISONING CON 10% DE RUIDO GAUSSIANO, 50% DE NODOS MALICIOSOS, KRUM)

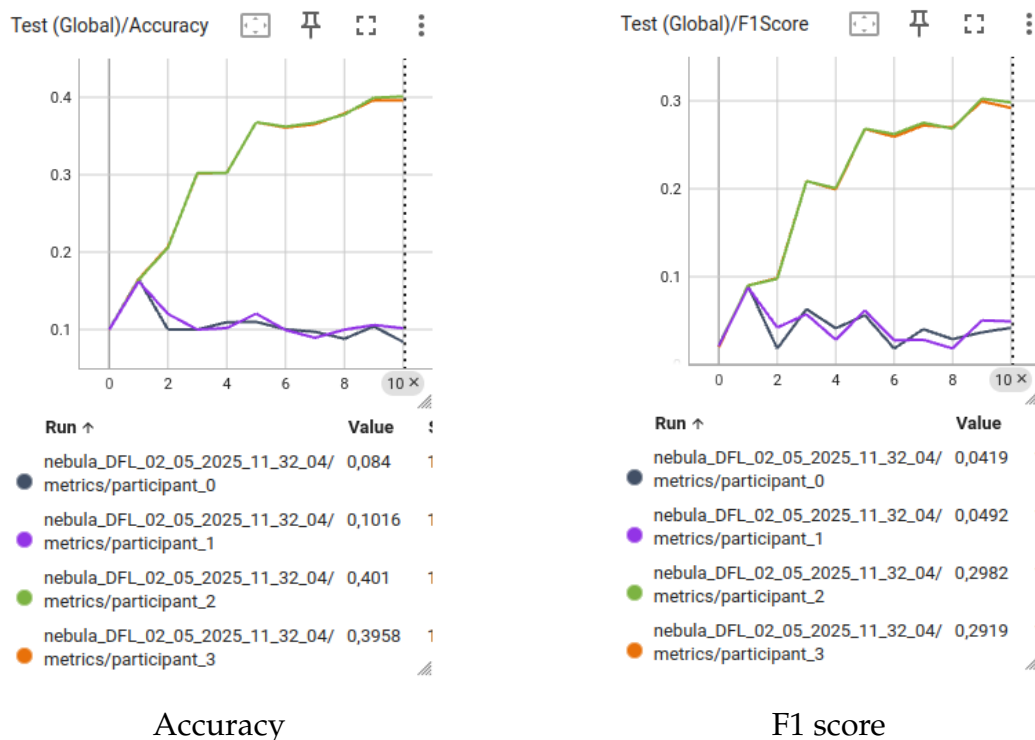


Figura 3.15: Métricas en test en el escenario B3.8

En la **Figura 3.15** se muestran el accuracy y el F1 score en el conjunto de test para el escenario B3.8. Ambas métricas presentan una caída abrupta en el rendimiento de los nodos gris y morado, probablemente maliciosos, entre las rondas 1 y 2. Sin embargo, se observa que tras este ataque, los nodos benignos mantienen un desempeño sólido y sus métricas continúan aumentando de forma sostenida en las siguientes rondas. Al igual que en B3.6, los nodos benignos no alcanzan valores muy altos en el accuracy y en el F1 score, pero esto podría estar principalmente relacionado con la distribución non-IID de los datos. Esto de nuevo demuestra que el algoritmo de agregación de Krum logra mitigar eficazmente los efectos del ataque en los nodos benignos, incluso con un porcentaje de nodos maliciosos más elevado.

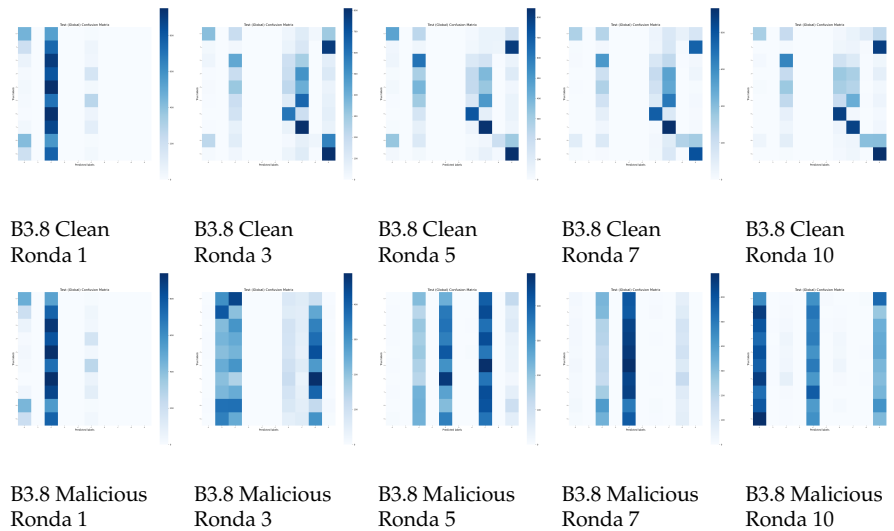


Figura 3.16: Matrices de confusión de B3.8: en nodos **Clean** (↑) y **Malicious** (↓).

En la **Figura 3.16** se muestran las matrices de confusión para el escenario B3.8 en las rondas 1, 3, 5, 7 y 10, para los nodos benignos y los maliciosos. Se observa que los nodos maliciosos tienen un mal desempeño desde la primera ronda, y que tras ejecutar el ataque, su rendimiento decae todavía más en las rondas siguientes, con una clara degradación en la capacidad de clasificación. Sin embargo, los nodos benignos logran mantener un desempeño algo más aceptable a lo largo de las rondas, aunque con algunas limitaciones. Sin embargo, en comparación con el nodo maligno, los nodos benignos logran clasificar correctamente varias de las clases, por lo que Krum parece estar siendo efectivo.

3.3.2. Respuestas a las preguntas

En esta sección, nos basaremos en los análisis realizados sobre cada uno de los escenarios para responder a las preguntas solicitadas.

PREGUNTA 1

¿Qué algoritmo demostró mayor resiliencia frente a actualizaciones adversarias, y por qué?

RESPUESTA:

De los algoritmos de agregación evaluados en los distintos escenarios, Krum demostró una mayor resiliencia frente a actualizaciones adversarias en comparación con Trimmed Mean. De hecho, incluso en los escenarios con un ataque de Model Poisoning con un porcentaje elevado de nodos maliciosos (como en B3.7 y B3.8, con un 50 % de nodos maliciosos), el algoritmo Krum logró mitigar eficazmente los efectos de los ataques en los nodos benignos. A pesar de que los nodos maliciosos presentaron una caída abrupta en el rendimiento desde las primeras rondas, los nodos benignos siguieron mostrando una mejora sostenida en

sus métricas (accuracy y F1 score) a lo largo de las rondas. Esto podría deberse a que Krum puede identificar y excluir las actualizaciones malignas de manera más eficiente, mediante la selección de la actualización más cercana a un consenso mayoritario de las demás.

Por el contrario, Trimmed Mean mostró una caída en el rendimiento de los nodos benignos incluso cuando solo un pequeño porcentaje de nodos era malicioso (como en B3.1 y B3.2, donde solo había un 20% de nodos maliciosos). A pesar de que Trimmed Mean intenta eliminar las actualizaciones extremas y agregar el resto, no parece ser lo suficientemente robusto para contrarrestar los ataques de Model Poisoning, al menos en los escenarios presentados. Esto podría deberse a que los envenenamientos llevados a cabo no dieron lugar a actualizaciones lo suficientemente extremas como para ser filtradas por este algoritmo.

PREGUNTA 2

¿Algún método de agregación sobreajustó o tuvo bajo rendimiento en clases específicas? ¿Puede esto correlacionarse con la presencia de clientes maliciosos?

RESPUESTA:

Para poder analizar si algún método sobreajustó, vamos a comparar el F1 score en el conjunto de train y test para cada uno de los escenarios:

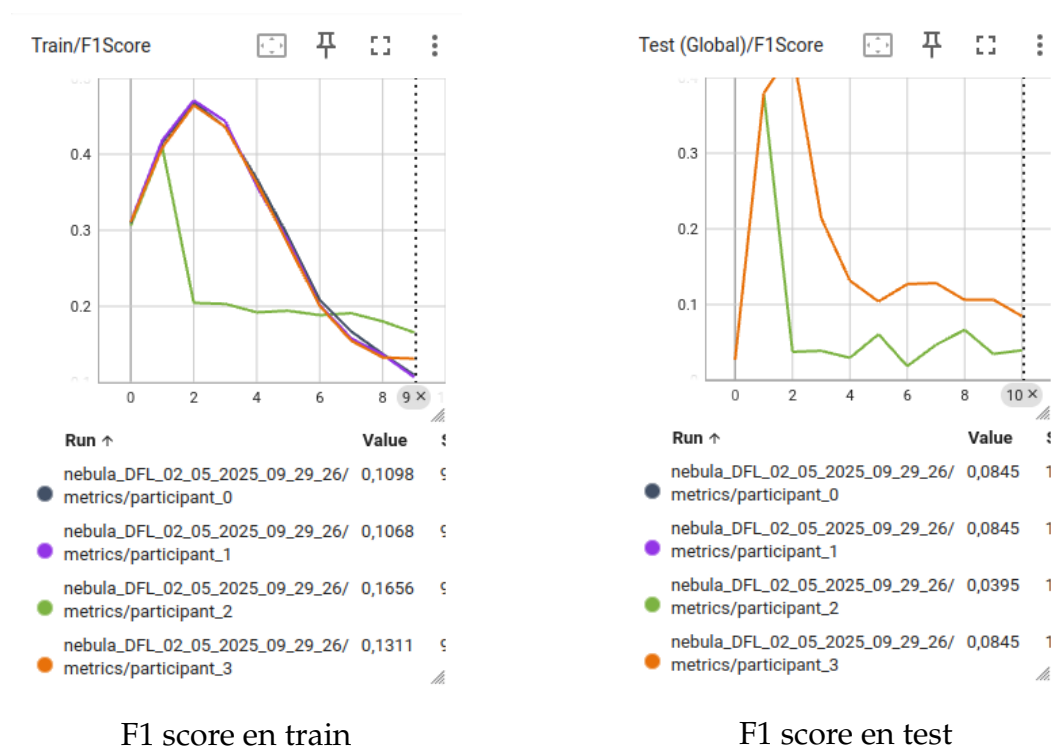


Figura 3.17: F1 score en el escenario B3.1

En la **Figura 3.17** se observa una ligera diferencia en el F1 score de B3.1 entre el conjunto de train y el de test, siendo los resultados algo mejores en train. Por esto,

podría haber un ligero overfitting en B3.1, aunque es bastante mínimo.

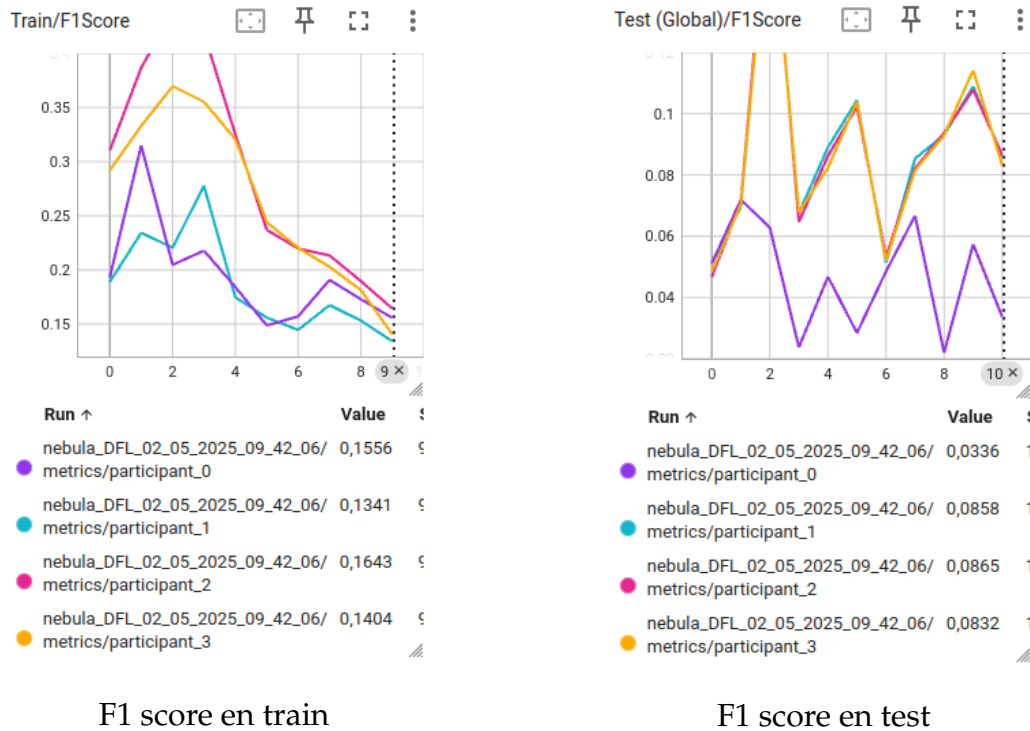


Figura 3.18: F1 score en el escenario B3.2

En la **Figura 3.18** se observa una diferencia en el F1 score de B3.2 entre el conjunto de train y el de test, siendo los resultados mejores en train. En este escenario, parece que el overfitting es algo más marcado que en B3.1.

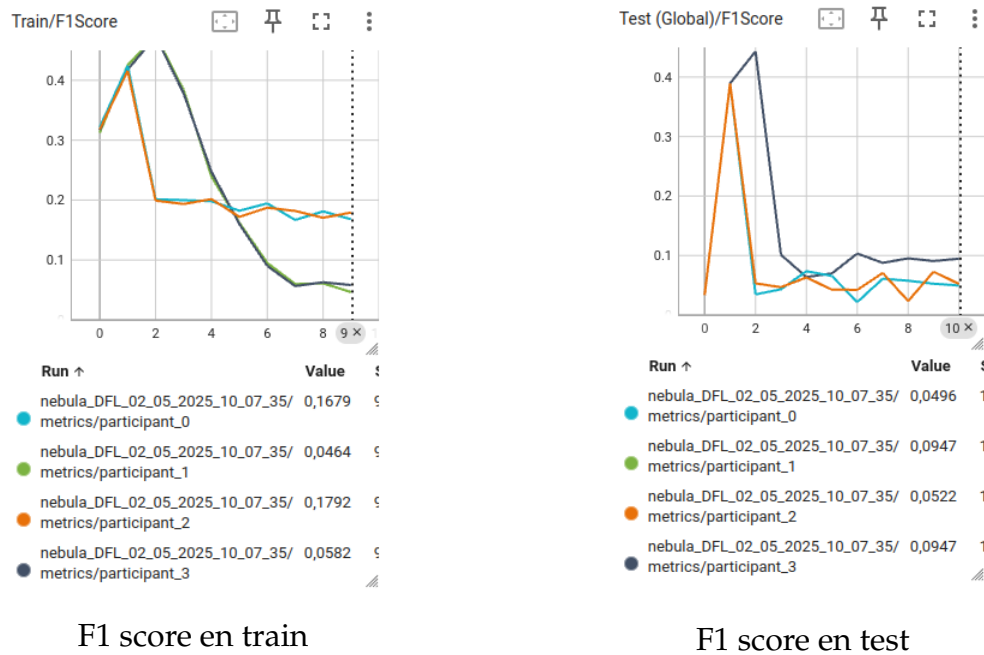


Figura 3.19: F1 score en el escenario B3.3

En la **Figura 3.19** no se observa una gran diferencia en el F1 score de B3.3 entre el conjunto de train y el de test para los nodos benignos. Sin embargo, sí que se observa que en los nodos maliciosos, se obtienen mejores resultados en train que en test. En este escenario, parece que el overfitting es más evidente en los nodos maliciosos que en los benignos.

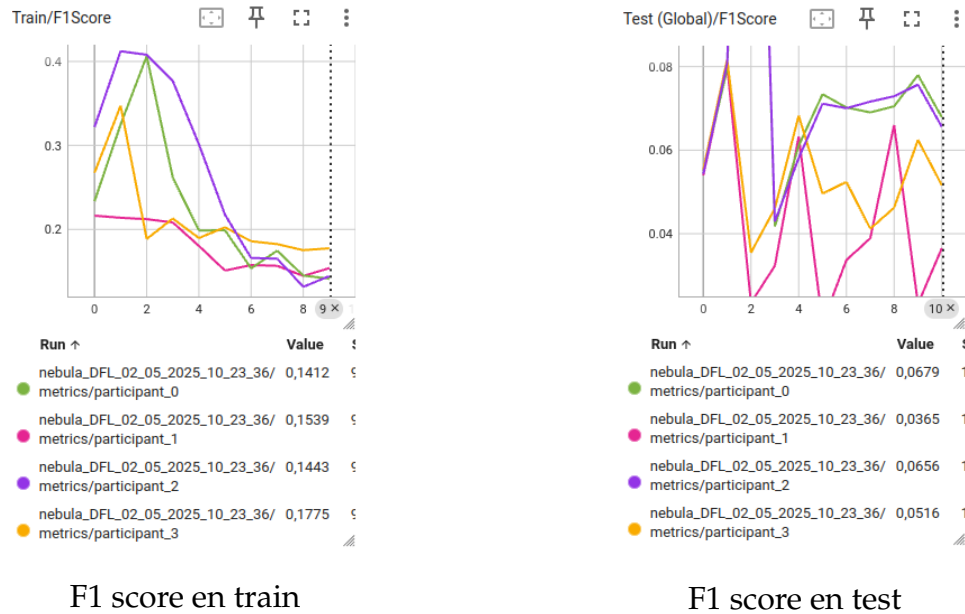


Figura 3.20: F1 score en el escenario B3.4

En la **Figura 3.20** se observa una diferencia importante en el F1 score de B3.4 entre el conjunto de train y el de test. Esta diferencia es especialmente notable en las primeras rondas del entrenamiento, donde podría ser más marcado el overfitting.

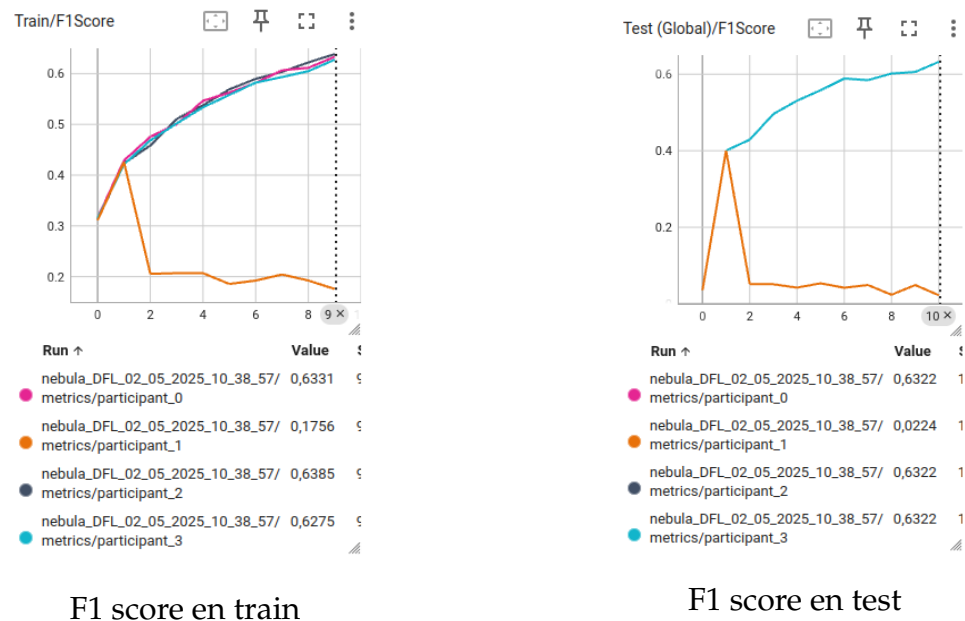


Figura 3.21: F1 score en el escenario B3.5

En la **Figura 3.21** no se observa una gran diferencia en el F1 score de B3.5 entre el conjunto de train y el de test. Por tanto, no parece que el escenario B3.5 esté sobreajustando.

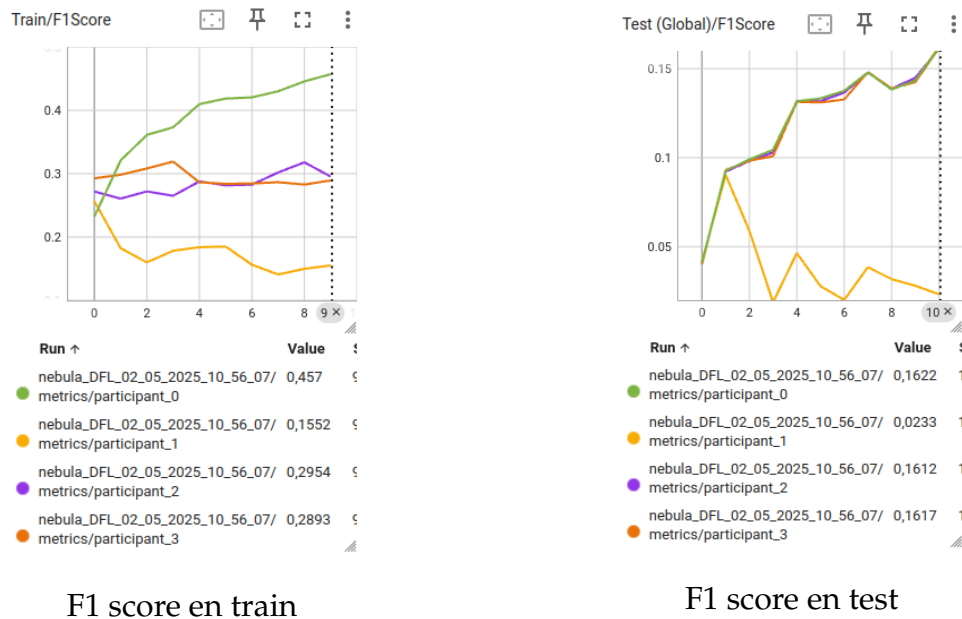


Figura 3.22: F1 score en el escenario B3.6

En la **Figura 3.22** se observa una diferencia marcada en el F1 score de B3.6 entre el conjunto de train y el de test, especialmente en el nodo verde. Este nodo tiene un F1 score de 0,457 en la última ronda con el conjunto de train, mientras que tiene un F1 score de 0,1622 en la última ronda con test, lo cual es una diferencia bastante notable. Además, en el resto de nodos también parece haber overfitting, aunque no tan marcado.

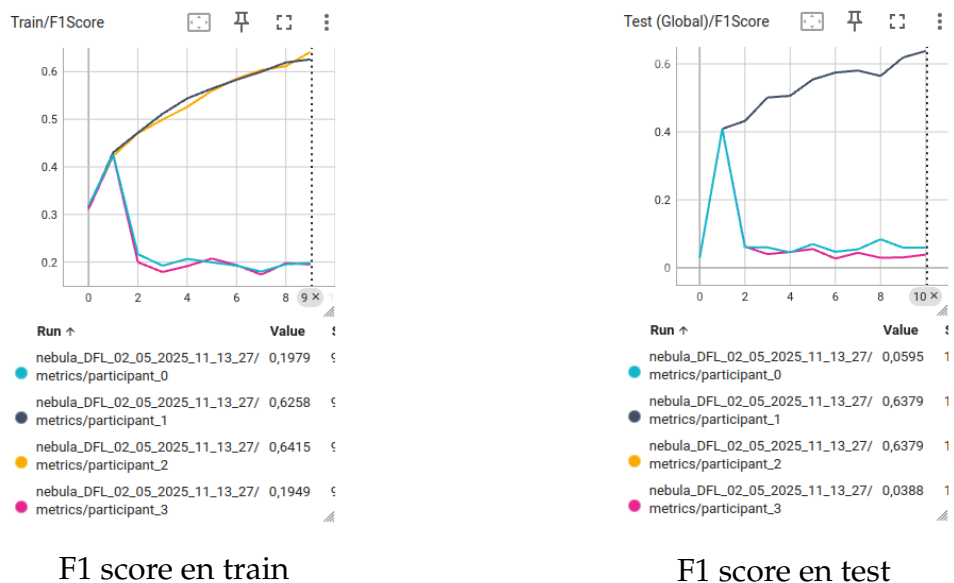


Figura 3.23: F1 score en el escenario B3.7

En la **Figura 3.23** no se observa una gran diferencia en el F1 score de B3.7 entre el conjunto de train y el de test. Por tanto, no parece que el escenario B3.7 esté sobreajustando.

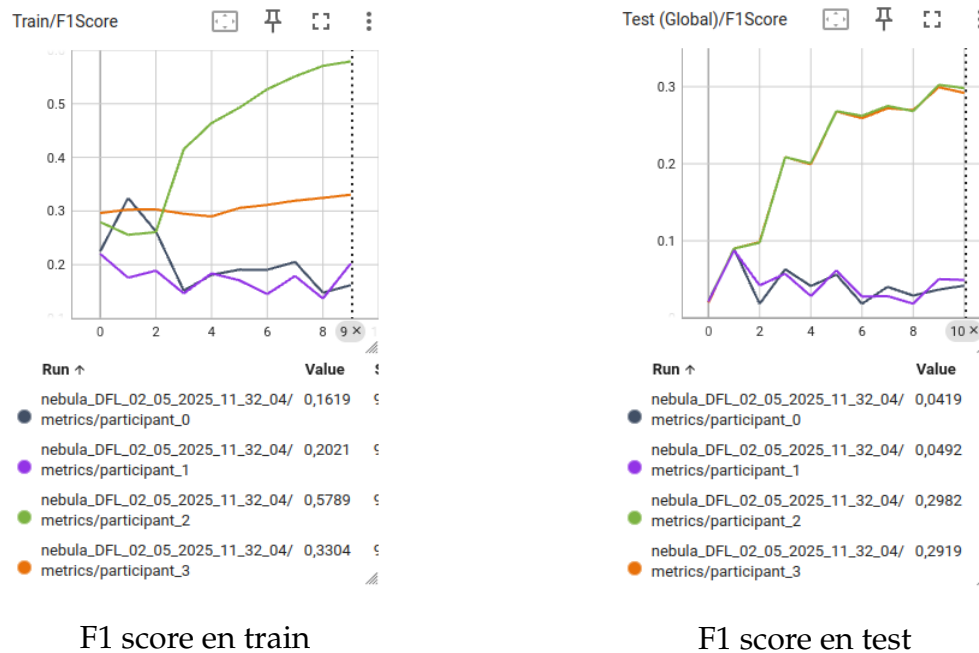


Figura 3.24: F1 score en el escenario B3.8

En la **Figura 3.24** se observa algo de diferencia en el F1 score de B3.8 entre el conjunto de train y el de test, especialmente en el nodo verde. Este nodo tiene un F1 score de 0,5789 en la última ronda con el conjunto de train, mientras que tiene un F1 score de 0,2982 en la última ronda con test, lo cual es una diferencia bastante notable. Además, en el resto de nodos también parece haber overfitting, aunque no tan marcado.

En conclusión, a partir del análisis de las métricas de F1 score en los conjuntos de train y test, se puede afirmar que algunos métodos de agregación sí mostraron signos de sobreajuste. En particular, los escenarios B3.2, B3.4, B3.6 y B3.8 fueron los casos más claros de overfitting, donde se encontraron diferencias notables entre el rendimiento en train y en test, especialmente en algunos nodos concretos.

Este fenómeno de overfitting parece correlacionarse tanto con la presencia de clientes maliciosos como con la distribución non-IID de los datos. Por un lado, los nodos maliciosos pueden hacer que el modelo aprenda patrones erróneos o sesgados, dificultando su generalización. Por otro lado, la naturaleza non-IID de los datos puede agravar este efecto (de hecho, los escenarios con más overfitting coincidieron justamente con aquellos que tenían datos non-IID), ya que cada nodo entrena con una distribución local diferente, lo que dificulta la construcción de un modelo global coherente y favorece el sobreajuste a los datos locales.

Ahora, para determinar los escenarios con bajo rendimiento en clases específicas, podemos observar las matrices de correlación antes mostradas.

Comenzamos por los escenarios que utilizaban Trimmed Mean como método de agregación. Por ejemplo, fijándonos en las matrices de correlación en la ronda 10, observamos que en B3.1, los nodos benignos tuvieron dificultades con prácticamente todas las clases menos la 1, tal como se puede observar en la **Figura 3.2**. En B3.2, mostraron un bajo rendimiento también en todas las clases menos en la 8, como se observa en la **Figura 3.4**. En B3.3, los nodos benignos mostraron un bajo rendimiento prácticamente en todas las clases menos tal vez en la 8, como se observa en la **Figura 3.6**. En B3.4, también tuvieron dificultades con todas las clases menos la 8, como se puede apreciar en la **Figura 3.8**.

Por otro lado, para los métodos que usaban Krum como método de agregación, se tiene que en B3.5 todas las clases fueron predichas con una buena precisión por los nodos benignos en la ronda 10, como se muestra en la **Figura 3.10**. No obstante, en B3.6 tuvieron dificultades con las clases 1, 3, 4, 6, 7 y 8, tal como se puede apreciar en la **Figura 3.12**. En B3.7 todas las clases fueron predichas con una buena precisión por los nodos benignos, como se puede observar en la **Figura 3.14**. Finalmente, en B3.8, los nodos benignos tuvieron dificultades para predecir correctamente las clases 0, 1, 3, 4, 5 y 8 en la última ronda, como se puede apreciar en la **Figura 3.16**.

A partir del análisis de las matrices de confusión, se observa que algunos métodos de agregación mostraron bajo rendimiento en clases específicas. En particular, los escenarios que utilizaron Trimmed Mean (B3.1–B3.4) mostraron un desempeño débil en la mayoría de las clases, con excepciones puntuales como la clase 1 en B3.1 o la clase 8 en B3.2–B3.4. Esto sugiere que el método no logró filtrar eficazmente las actualizaciones envenenadas, lo que afectó negativamente la capacidad de los modelos para generalizar adecuadamente.

Por el contrario, en los escenarios con Krum como algoritmo de agregación, el rendimiento por clase fue en general más robusto. En B3.5 y B3.7, todas las clases fueron predichas con buena precisión por los nodos benignos. Sin embargo, en B3.6 y B3.8, se detectaron algunas deficiencias importantes en varias clases, lo que indica que aunque Krum mejora la resistencia frente a ataques, no siempre logra proteger el rendimiento en todas las clases, especialmente en escenarios con datos non-IID.

Estos resultados permiten concluir que el bajo rendimiento en clases específicas se correlaciona en gran medida con la presencia de clientes maliciosos, especialmente cuando el algoritmo de agregación no logra mitigar eficazmente los efectos de sus ataques.

PREGUNTA 3

¿Cómo cambiaría el rendimiento si los ataques fueran más sigilosos (por ejemplo, manipulación de gradientes)?

RESPUESTA:

Si los ataques fueran más sigilosos, es probable que el rendimiento general de los sistemas se viera más afectado, especialmente en aquellos con métodos de agregación menos robustos. Esto es debido a que, si el ataque no es tan evidente y es capaz de camuflarse dentro del rango de valores típicos de los gradientes,

podrían evadir filtros que sean demasiado simples.

Por ejemplo, en el caso de Trimmed Mean, este algoritmo de agregación elimina los valores extremos y agrega el resto. Si las actualizaciones maliciosas no se desvían mucho numéricamente del rango típico de valores, entonces Trimmed Mean podría no detectarlas, y por tanto, conservarlas en la agregación. Por tanto, este algoritmo sería especialmente vulnerable a ataques sigilosos.

Por otro lado, el algoritmo de agregación Krum selecciona la actualización más cercana a un consenso mayoritario. En principio, este algoritmo podría ser más resistente a ciertos ataques, pero si varios nodos maliciosos colaboraran, podrían engañarlo haciéndole creer que sus actualizaciones son benignas.

Los ataques más sutiles podrían reducir la precisión de forma progresiva y sostenida, pero sin causar grandes caídas repentinas. Esto hace más difícil su detección, especialmente para métodos de agregación simples como Trimmed Mean o Krum.

4. Boletín IV

4.1. Introducción a la tarea y ejercicios a resolver

Para este boletín, se deben abordar los siguientes **ejercicios**:

1. Utilizar la configuración de model poisoning definida en la Sesión II junto con un ataque de flooding (con el mismo porcentaje de nodos maliciosos y parámetros de ataque predeterminados).
2. Configurar la configuración previa con las siguientes dos variantes del sistema de reputación usando todas las métricas:
 - Reputación con Pesos Estáticos: 0,30 / 0,15 / 0,30 / 0,25, respectivamente.
 - Habilitar los pesos dinámicos, lo cual ajusta la importancia de las métricas del modelo y de comunicación según el comportamiento observado.
3. Rastrear las puntuaciones de reputación por nodo en cada ronda, registrar el F1-score y la matriz de confusión, y compararlas con el baseline en términos de robustez y accuracy.

Con lo que se pretende conseguir:

- Comprender el papel de los sistemas de reputación en el DFL como mecanismo para evaluar a los nodos vecinos.
- Explorar cómo se calcula la reputación en función del comportamiento de comunicación y la similitud del modelo, y cómo esto influye en la integración del modelo.
- Evaluar diferentes configuraciones de reputación, incluidas estrategias de peso fijo y peso adaptativo, para detectar y mitigar comportamientos maliciosos.
- Comparar el rendimiento del modelo, su convergencia y estabilidad con y sin el uso de filtrado basado en reputación.

4.2. Preguntas a responder

Según los resultados de los experimentos, responder las siguientes **preguntas**:

1. ¿Qué configuración (estática o dinámica) ofreció el mejor equilibrio entre robustez y accuracy en el aprendizaje?

2. ¿El sistema de reputación logró detectar y aislar los nodos maliciosos desde el inicio del entrenamiento o a partir de una ronda específica?

4.3. Desarrollo de la tarea

Los **escenarios de federación** que entrenaremos en NEBULA para esta tarea son los siguientes:

Escenario B4.1:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Ataque:** Model Poisoning (10 % de ruido gaussiano)
- **Porcentaje de nodos maliciosos:** 20 %
- **Configuración de reputación:**
Pesos fijos (0.3, 0.15, 0.3, 0.25)

Escenario B4.2:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Ataque:** Flooding
- **Porcentaje de nodos maliciosos:** 20 %
- **Configuración de reputación:**
Pesos fijos (0.3, 0.15, 0.3, 0.25)

Escenario B4.3:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Ataque:** Model Poisoning (10 % de ruido gaussiano)
- **Porcentaje de nodos maliciosos:** 20 %
- **Configuración de reputación:**
Pesos dinámicos

Escenario B4.4:

- **Dataset:** CIFAR-10
- **Arquitectura de red:** CNN
- **Tipo de federación:** Descentralizada
- **Distribución de los datos:** non-IID
- **Ataque:** Flooding
- **Porcentaje de nodos maliciosos:** 20 %
- **Configuración de reputación:**
Pesos dinámicos

4.3.1. Comparaciones entre configuración de reputación estática y dinámica

La reputación de un nodo representa una puntuación que se asigna a cada cliente reflejando la calidad de sus actualizaciones, su consistencia a lo largo del tiempo, y su cumplimiento con los estándares de la federación, esto con el propósito de favorecer a los clientes más confiables, penalizar o excluir a los maliciosos, y ajustar su influencia durante la agregación. En NEBULA, esta reputación se calcula usando dos enfoques principales:

- **Model-Based Reputation:** Se analiza qué tan parecido es cada modelo respecto a los modelos de los vecinos, y se mide cuánto cambian los modelos entre rondas. Si un vecino constantemente envía modelos muy diferentes o incoherentes, su reputación disminuye.

- **Communication-Based Reputation:** Se analiza la frecuencia de los mensajes, de forma que si un nodo responde siempre de forma regular, inspira confianza. También se considera la latencia o el tiempo que tarda en responder, ya que si responde muy tarde o de forma errática, puede ser señal de problemas o comportamiento malicioso.

En este experimento se evaluará el rendimiento del sistema bajo dos configuraciones distintas de reputación, una en la que se asignarán pesos fijos a cada métrica (0,3 / 0,15 / 0,3 / 0,25), manteniéndose constantes a lo largo del entrenamiento, y otra en la que se usarán pesos dinámicos, donde la importancia de cada métrica se ajustará automáticamente en función del comportamiento observado. Así, el objetivo será analizar cómo afecta cada estrategia al desempeño del sistema frente a ataques de Model Poisoning y de Flooding, en cuanto a su capacidad para detectar y mitigar los efectos de los nodos maliciosos.

REPUTACIÓN ESTÁTICA VS DINÁMICA EN UN ATAQUE DE MODEL POISONING (B4.1 , B4.3)



Figura 4.1: Accuracy en train y test (global y local) en escenarios con model poisoning: reputación estática (↑) , reputación dinámica (↓).

En la **Figura 4.1** se muestra el accuracy en los conjuntos de train y test (global y local) para los escenarios B4.1 (con configuración de reputación estática) y B4.3 (con configuración de reputación dinámica).

Comenzando por el accuracy en el conjunto de train, se puede observar que en el escenario B4.1, el nodo malicioso, correspondiente al nodo azul, mantiene el peor rendimiento a lo largo de todas las rondas. Del resto de nodos, el morado parece tener un desempeño algo peor, aunque esto podría estar relacionado con la distribución non-IID de los datos, más que con un impacto directo del ataque. En general, los nodos benignos se mantienen relativamente estables.

Algo similar ocurre en el escenario B4.3, donde el nodo azul también es el malicioso. Los nodos benignos siguen siendo robustos, aunque el nodo verde muestra un rendimiento bastante bajo en comparación con los otros dos benignos. No obstante, esto también podría deberse a la distribución non-IID de los datos. En general, ambos esquemas de reputación logran contener el impacto del nodo malicioso, pero en este punto no se observa una ventaja clara de la reputación dinámica sobre la estática en cuanto a accuracy en train.

Por otro lado, al observar el accuracy en el conjunto de test, el análisis local muestra un resultado bastante similar al observado en train en ambos escenarios. Sin embargo, en el análisis global comienzan a evidenciarse diferencias notables entre ambas configuraciones. En los dos casos, los nodos benignos mantienen un buen rendimiento, mientras que el nodo malicioso obtiene valores de accuracy muy cercanos a 0, lo cual podría estar relacionado con que ha ganado una baja reputación y ha sido penalizado.

En cuanto a los nodos benignos, se observa que, con configuración de reputación estática (B4.1), estos nodos alcanzan un accuracy en torno a 0,48 en la última ronda, mientras que con configuración de reputación dinámica (B4.3), logran un valor cercano a 0,55. Esto sugiere que el ajuste dinámico de los pesos de reputación, al adaptarse mejor al comportamiento observado, permite una defensa más eficaz frente a ataques de tipo Model Poisoning. En consecuencia, se concluye que la reputación dinámica favorece un mejor desempeño global del sistema en los escenarios considerados.

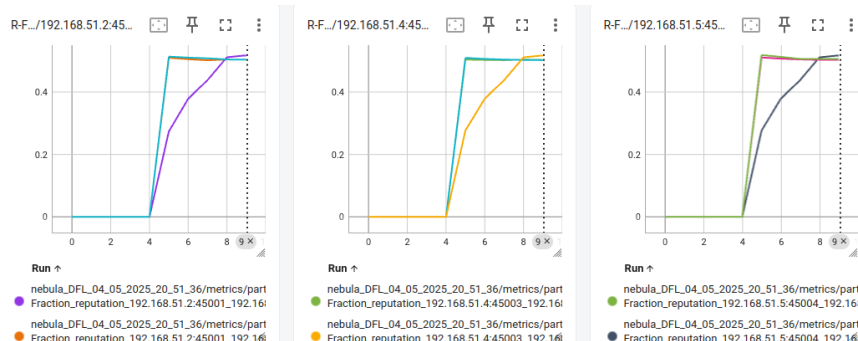
A continuación, en la **Figura 4.2** y en la **4.3**, se muestran algunas métricas de reputación para los escenarios B4.1 y B4.3. Estas se utilizan para evaluar la confiabilidad y comportamiento de los clientes, especialmente en escenarios con participantes maliciosos o datos no confiables. Entre estas, se incluyen:

- **Count messages:** mide cuántos mensajes envía cada nodo de la federación a un nodo benigno en particular.
- **Fraction reputation:** mide la proporción de reputación relativa de cada cliente respecto al total del sistema.
- **Similarity:** mide el grado de similitud entre el modelo enviado por un cliente y el modelo global actual o con los modelos de otros clientes.
- **Reputation:** es una puntuación agregada que refleja la confiabilidad y el historial de cada cliente en el proceso de FL.

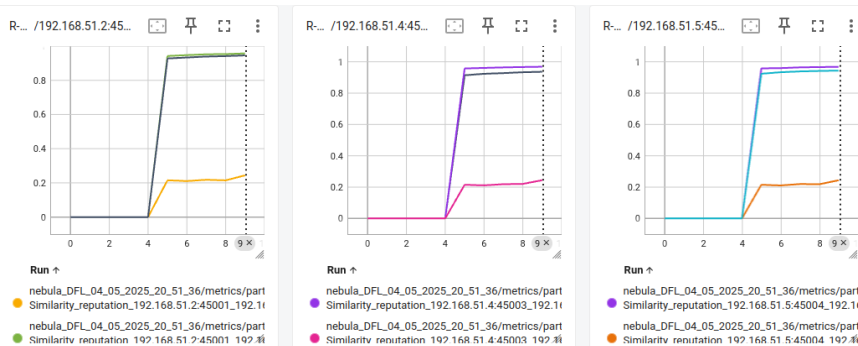
Count messages (Static)



Fraction reputation (Static)



Similarity (Static)



Reputation (Static)

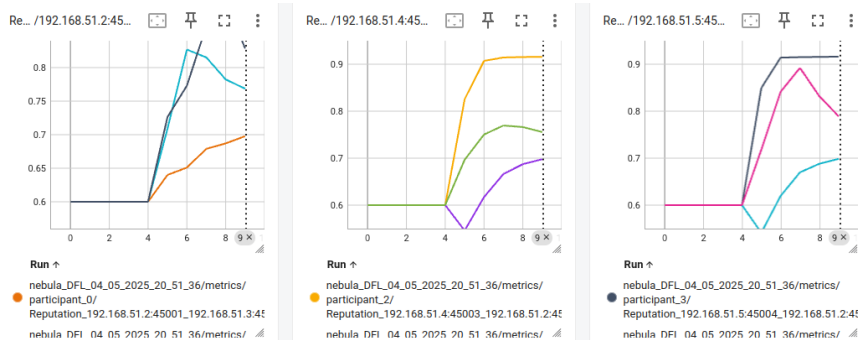
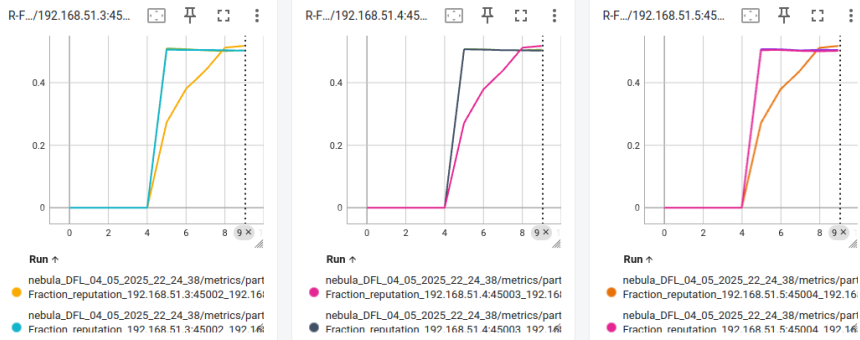


Figura 4.2: Métricas para reputación estática con model poisoning.

Count messages (Dynamic)



Fraction reputation (Dynamic)



Similarity (Dynamic)



Reputation (Dynamic)



Figura 4.3: Métricas para reputación dinámica con model poisoning.

En la **Figura 4.2**, se muestran las métricas de reputación que hemos explicado para el escenario B4.1, mientras que en la **Figura 4.3**, se muestran las del escenario B4.3. Recordando que estos dos escenarios estaban siendo atacados por un nodo malicioso mediante Model Poisoning, se puede observar que este ataque se ve reflejado en estas métricas.

Especialmente, se puede observar que para las 3 gráficas (correspondientes cada una de ellas a cada uno de los 3 nodos benignos) referentes a la similaridad, hay claramente un nodo que muestra un valor de similaridad mucho más bajo que el resto. En particular, mientras que los otros nodos se acercan a una similaridad de 1, existe un nodo (el nodo malicioso) que muestra una similaridad cercana a 0,2, y esto ocurre en ambos escenarios.

Por otro lado, mirando la fraction reputation, también se observa una curva que crece más lentamente que la del resto de nodos. Esta es la correspondiente al nodo malicioso, cuya reputación es más baja que la de los nodos benignos. Esto también se puede observar en las gráficas de reputación, donde claramente hay un nodo cuya reputación es más baja que la de los demás. Esto es especialmente notable en las gráficas del escenario B4.3, donde el nodo con menor reputación prácticamente tiene una reputación muy cercana a 0.

Al perder reputación el nodo maligno, este es penalizado, de forma que se limita su capacidad de participación en la federación. Esto se refleja en las gráficas de contador de mensajes, donde uno de los nodos envía menos mensajes que el resto en cada una de las rondas.

REPUTACIÓN ESTÁTICA VS DINÁMICA EN UN ATAQUE DE FLOODING (B4.2 , B4.4)



Figura 4.4: Accuracy en train y test (global y local) en escenarios con flooding: reputación estática (↑) , reputación dinámica (↓).

En la **Figura 4.4** se muestra un accuracy en los conjuntos de train y test (global y local) para los escenarios **B4.2** (con configuración estática) **B4.4** (con configuración de reputación dinámica).

Comenzando con el accuracy en el conjunto de train, se puede observar que en el escenario **B4.2**, el nodo malicioso, corresponde con el nodo amarillo, mediante el peor rendimiento a lo largo de todas las rondas. Del resto de nodos, el nodo marrón parece tener un desempeño algo peor, aunque esto podría estar relacionado con la distribución non-IID de los datos, más que con un impacto directo del ataque. En general, los nodos benignos se mantiene relativamente estables.

Una situación similar se presenta en el escenario **B4.5**, en el cual el nodo amarillo también actúa como el malicioso. Los nodos benignos continúan mostrando solidez, aunque el nodo verde presenta un desempeño considerablemente inferior en comparación con los demás nodos benignos. Esta diferencia podría estar relacionada con la distribución no IID de los datos. En términos generales, ambos sistemas de reputación logran mitigar el efecto del nodo

malicioso, y en este caso particular no se percibe una ventaja clara de la reputación dinámica frente a la estática en lo que respecta a la precisión durante el entrenamiento.

Por otro lado, al analizar la precisión en el conjunto de prueba, se observa que el enfoque local presenta un comportamiento muy similar al obtenido durante el entrenamiento en ambas configuraciones. No obstante, en el enfoque global comienzan a notarse diferencias significativas entre los dos escenarios. En ambos casos, los nodos benignos continúan mostrando un buen desempeño, mientras que el nodo malicioso alcanza niveles de precisión cercanos a cero, posiblemente debido a su baja reputación y a que ha sido objeto de penalizaciones.

En lo que respecta a los nodos benignos, se aprecia que bajo la configuración de reputación estática (B4.4) alcanzan un accuracy aproximado de 0.39 en la ronda final, mientras que con la configuración de reputación dinámica (B4.6) dicho valor se eleva a alrededor de 0.49. Esto indica que el ajuste dinámico de los pesos de reputación, al responder de manera más efectiva al comportamiento detectado, permite una defensa más robusta frente a ataques de tipo Model Poisoning. Por lo tanto, se puede concluir que el uso de reputación dinámica contribuye a un mejor rendimiento global del sistema en los escenarios analizados.

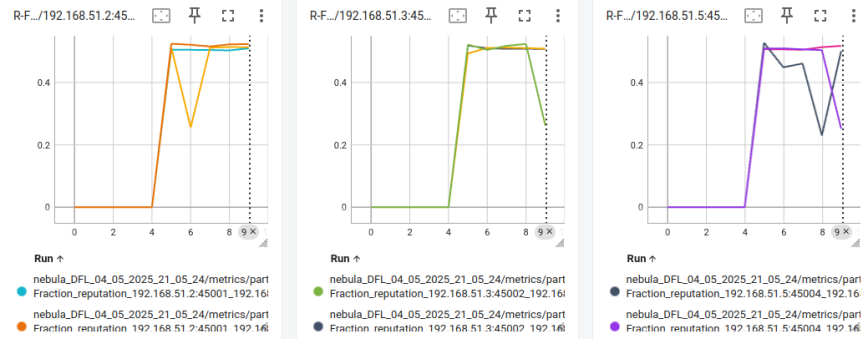
A continuación, en la **Figura 4.5** y la **Figura 4.6**, se presentan diversas métricas de reputación correspondientes a los escenarios B4.4 y B4.6. Estas métricas permiten analizar la confiabilidad y el desempeño de los clientes, especialmente en contextos donde existen participantes maliciosos o se manejan datos con poca fiabilidad. Entre estas, se incluyen:

- **Count messages:** mide cuántos mensajes envía cada nodo de la federación a un nodo benigno en particular.
- **Fraction reputation:** mide la proporción de reputación relativa de cada cliente respecto al total del sistema.
- **Similarity:** mide el grado de similitud entre el modelo enviado por un cliente y el modelo global actual o con los modelos de otros clientes.
- **Reputation:** es una puntuación agregada que refleja la confiabilidad y el historial de cada cliente en el proceso de FL.

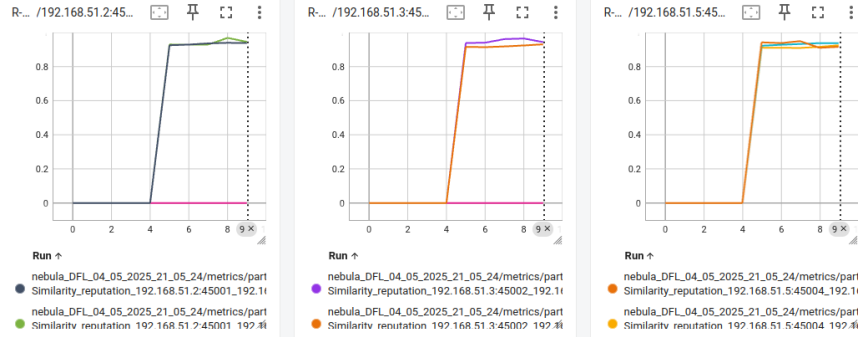
Count messages (Static)



Fraction reputation (Static)



Similarity (Static)

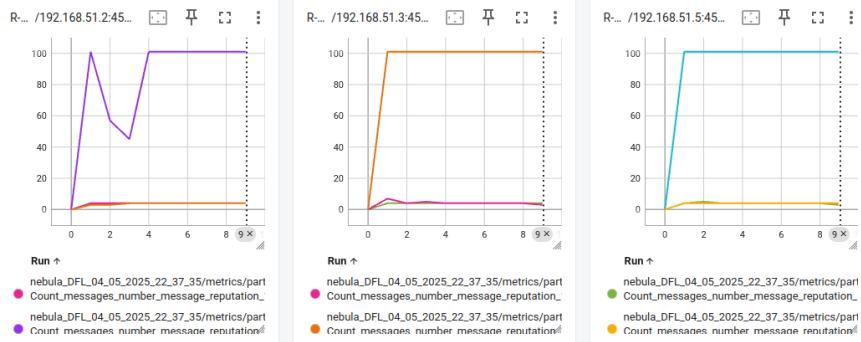


Reputation (Static)

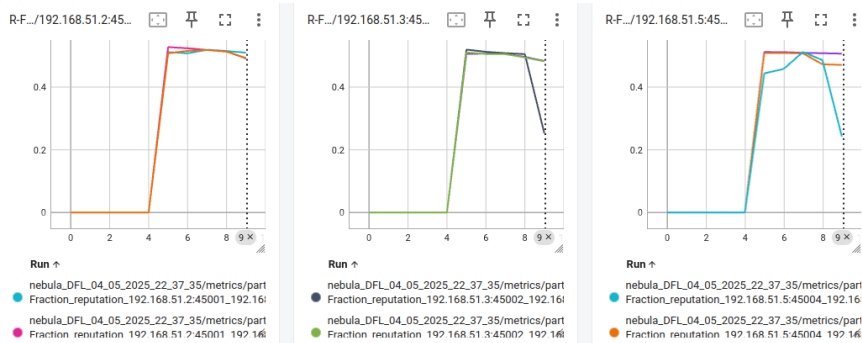


Figura 4.5: Métricas para reputación estática con flooding.

Count messages (Dynamic)



Fraction reputation (Dynamic)



Similarity (Dynamic)



Reputation (Dynamic)



Figura 4.6: Métricas para reputación dinámica con flooding.

En la **Figura 4.5**, se muestran las métricas de reputación que hemos explicado para el escenario B4.2, mientras que en la **Figura 4.6**, se muestran las del escenario B4.4. Recordando que estos dos escenarios estaban siendo atacados por un nodo malicioso mediante Flooding, se puede observar que este ataque se ve reflejado en estas métricas.

El ataque de Flooding consiste en que el nodo malicioso envía de forma masiva actualizaciones para saturar el sistema. Esto se ve reflejado en la métrica de contador de mensajes, donde se observa que uno de los nodos (el nodo malicioso) envía en cada ronda cerca de 100 mensajes a los otros nodos, mientras que los nodos benignos envían menos de 5. Sin embargo, en algunas rondas, el número de mensajes enviados por el nodo maligno disminuye, probablemente porque está siendo penalizado y se limita su participación. Esta disminución se ve más veces en el escenario B4.1, es decir, con reputación estática.

Además, en cuanto a la similaridad, se observa que uno de los nodos mantiene una similaridad de 0 a lo largo de todas las rondas en algunas de las gráficas. Esto podría deberse a que el nodo malicioso podría estar enviando modelos aleatorios o ruidosos, o gradientes completamente divergentes de los enviados por los nodos benignos.

En cuanto a la *fraction reputation*, se observa como uno de los nodos sufre ciertas caídas y oscilaciones, siendo más evidentes en el escenario B4.1. Además, en la gráfica de reputación, en el escenario B4.1 hay un nodo que claramente muestra una reputación más baja que el resto, este es, el nodo malicioso. Sin embargo, en el escenario B4.2, se observa como en cada gráfica, dos de los nodos muestran una reputación más baja. Así, parece que la estrategia de pesos dinámicos está detectando señales de comportamiento anómalo en más de un nodo. Esto puede deberse a que los pesos dinámicos permiten que el sistema reaccione de forma más sensible y adaptativa ante variaciones en el comportamiento, penalizando no solo al nodo malicioso evidente, sino también a otros que puedan estar mostrando inestabilidad, como latencias elevadas o cambios abruptos en sus actualizaciones.

Esto podría deberse a que ciertos nodos se están viendo saturados por el envío masivo de mensajes por parte del nodo malicioso, lo que puede estar causando que estos nodos saturados se demoren más tiempo para responder. Esto es indicativo de inestabilidad, por lo que la configuración de reputación dinámica puede estar detectando estas anomalías, disminuyendo la reputación de dichos nodos que se han visto saturados por el nodo maligno.

En resumen, mientras la reputación estática tiende a penalizar solo al atacante evidente, la reputación dinámica es más refinada y puede identificar múltiples señales de riesgo, lo que contribuye a una detección más robusta y una agregación más segura.

4.3.2. Respuestas a las preguntas

En esta sección, nos basaremos en los análisis realizados sobre cada uno de los escenarios para responder a las preguntas solicitadas.

PREGUNTA 1

¿Qué configuración (estática o dinámica) ofreció el mejor equilibrio entre robustez y accuracy en el aprendizaje?

RESPUESTA:

Tras el análisis realizado en la sección anterior para cada uno de los escenarios, es evidente que la configuración de reputación que ofreció el mejor equilibrio entre robustez y accuracy fue la configuración dinámica.

Por un lado, en escenarios afectados por ataques de tipo Model Poisoning (B4.1 y B4.3), se observó que ambos esquemas de reputación lograban contener parcialmente al nodo malicioso, manteniendo su accuracy global bajo y penalizando su reputación. Sin embargo, los nodos benignos alcanzaron un mayor rendimiento final con la configuración dinámica, logrando un accuracy global cercano a 0,55, frente al 0,48 obtenido con configuración estática. Esto sugiere que el sistema, al ajustar los pesos de las métricas de reputación en función del comportamiento observado, logra mejorar la agregación y obtener modelos más precisos.

Por otro lado, frente a ataques más agresivos como el Flooding (escenarios B4.2 y B4.4), la configuración dinámica también demostró una ventaja clara. Esta estrategia no solo logró detectar al nodo malicioso, penalizándolo tanto por sus modelos divergentes (similitud de 0) como por su comportamiento anómalo en la red (envío masivo de mensajes), sino que también fue capaz de identificar efectos colaterales en nodos que se vieron afectados por saturación o latencias elevadas. A diferencia de la reputación estática, que penaliza de forma fija según pesos predefinidos, la reputación dinámica ajusta la sensibilidad ante diferentes señales y logra reaccionar de forma más adaptativa ante las anomalías del sistema.

En resumen, la configuración de reputación dinámica no solo mantiene un buen rendimiento global, sino que permite una detección más sensible y eficaz de comportamientos maliciosos o inestables. Esto la convierte en una opción más robusta y precisa para escenarios bajo condiciones adversas.

PREGUNTA 2

¿El sistema de reputación logró detectar y aislar los nodos maliciosos desde el inicio del entrenamiento o a partir de una ronda específica?

RESPUESTA:

Tal como se puede observar en las gráficas de reputación, en las primeras rondas todos los nodos comienzan con una reputación de 0,6. Así se mantiene hasta la ronda 4 (en la mayoría de los nodos, excepto los correspondientes a la tercera gráfica de reputación en la **Figura 4.5**, donde ocurre hasta la 3) y a partir de la 5 (o la 4, en ese caso en concreto), ya empiezan a asignarse valores de reputación específicos a cada nodo, favoreciendo a los nodos con un buen comportamiento, y penalizando a aquellos con un comportamiento anómalo.

5. Boletín V

5.1. Introducción a la tarea y ejercicios a resolver

Para este boletín, se deben abordar los siguientes **ejercicios**:

1. Utilizar el escenario de DFL definido en la Sesión I, sin ataques ni participantes maliciosos.
2. Ejecutar experimentos con la optimización del modelo habilitada usando las siguientes configuraciones:
 - Pruning
 - Aplicar poda estructurada utilizando `l1_unstructured` con `amount=0.8`.
 - Cuantización
 - Usar `precision="bf16-mixed"` (para simular entrenamiento con precisión reducida).
 - Usar `precision="64-true"` (representación más compleja del modelo).
3. Rastrear las siguientes métricas durante el entrenamiento:
 - Accuracy del modelo, F1-score y matriz de confusión por ronda.
 - Convergencia del modelo y tiempo por ronda.

Con lo que se pretende conseguir:

- Comprender el impacto de las técnicas de optimización del modelo, específicamente la poda (pruning) y la cuantización, en el rendimiento y la robustez.
- Evaluar los compromisos entre el accuracy del modelo, la estabilidad de convergencia y el costo de comunicación cuando se aplican técnicas de optimización.
- Comparar el rendimiento del modelo, su robustez y la eficiencia de transmisión con y sin optimización.

5.2. Preguntas a responder

Según los resultados de los experimentos, responder las siguientes **preguntas**:

1. ¿Qué configuración (solo pruning, solo cuantización, o combinada) ofreció el mejor equilibrio entre accuracy, eficiencia y velocidad de convergencia?
2. ¿La optimización incrementó significativamente la velocidad de entrenamiento (o redujo el tiempo por ronda) sin comprometer el accuracy?

5.3. Desarrollo de la tarea

Los **escenarios de federación** que entrenaremos en NEBULA para esta tarea son los siguientes:

Escenario B5.1:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Optimización: Pruning

Escenario B5.2:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Optimización: Cuantización con precision="bf16-mixed"

Escenario B5.3:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Optimización: Cuantización con precision="64-true"

Escenario B5.4:

- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Optimización:
 - Pruning
 - Cuantización con precision="bf16-mixed"

Escenario B5.5:

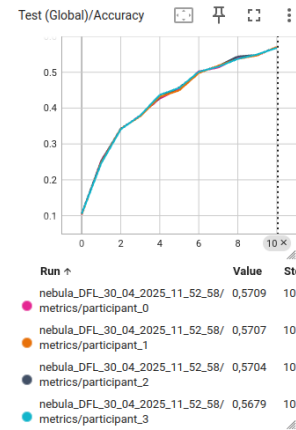
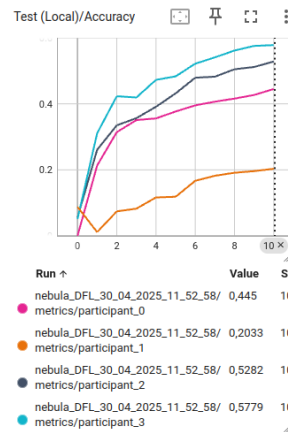
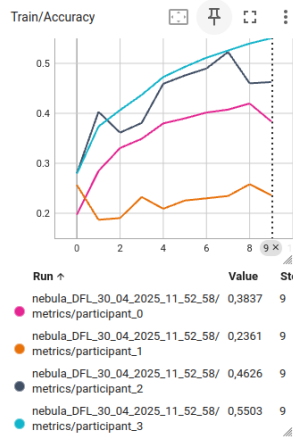
- Dataset: CIFAR-10
- Arquitectura de red: CNN
- Tipo de federación: Descentralizada
- Distribución de los datos: non-IID
- Algoritmo de agregación: FedAvg
- Optimización:
 - Pruning
 - Cuantización con precision="64-true"

PREGUNTA 1

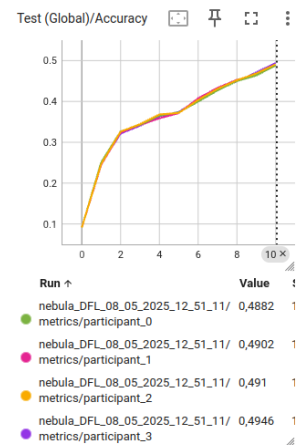
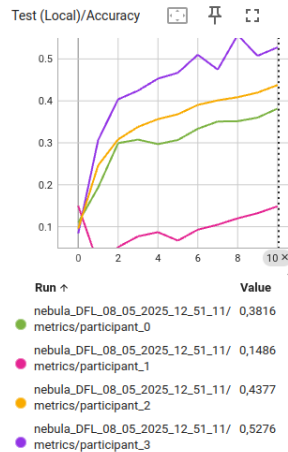
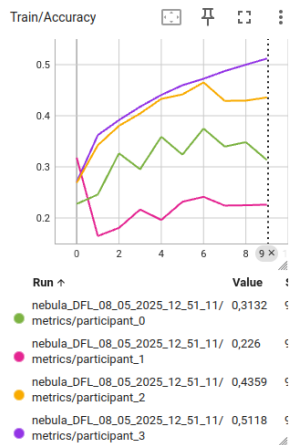
¿Qué configuración (solo pruning, solo cuantización, o combinada) ofreció el mejor equilibrio entre accuracy, eficiencia y velocidad de convergencia?

RESPUESTA: Para responder necesitamos comparar algunas estadísticas de los escenarios:

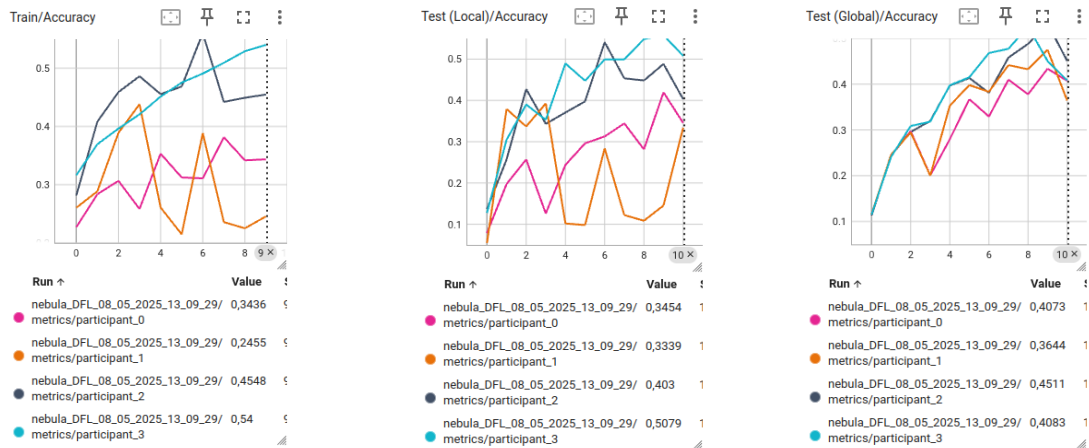
SCORES DE RENDIMIENTO PARA B1.4 (11 MIN.)



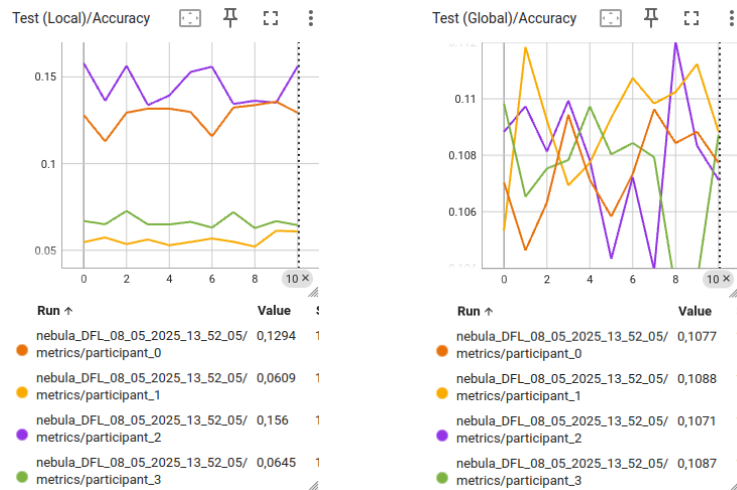
SCORES DE RENDIMIENTO PARA B5.1 (11 MIN.)



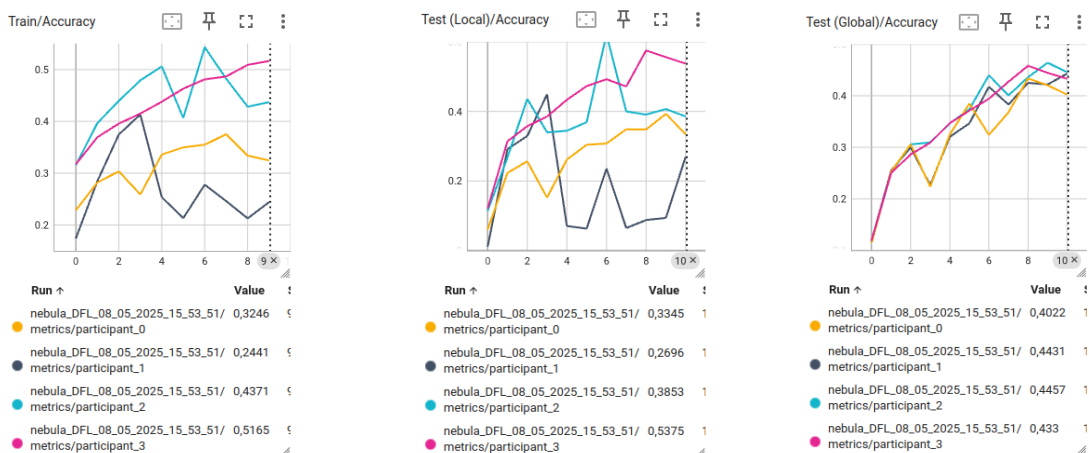
SCORES DE RENDIMIENTO PARA B5.2 (35 MIN.)



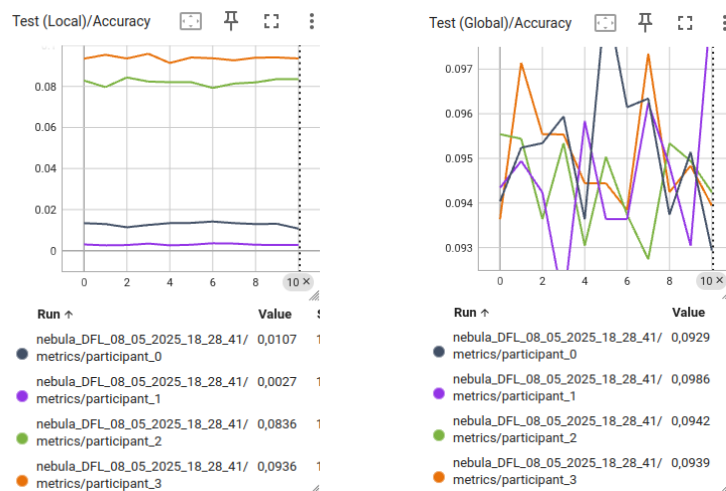
SCORES DE RENDIMIENTO PARA B5.3 (5 MIN.)



SCORES DE RENDIMIENTO PARA B5.4 (35 MIN.)



SCORES DE RENDIMIENTO PARA B5.5 (6 MIN.)



Podemos observar cómo al aplicar pruning sobre el escenario base (B1.4) existe una ligera pérdida de rendimiento, ya que el accuracy del modelo global en test cae de 0.57 a 0.5. En el conjunto de train la caída es de 0.6 a 0.52. Por otro lado, el tiempo de entrenamiento de la federación se mantiene en 11 minutos, lo cual nos resulta natural ya que al realizar el pruning de modelos no estamos añadiendo complejidad a la federación ni a los modelos. Tampoco reducimos la complejidad de los modelos, simplemente reducimos el peso de los mismos, lo cual puede aligerar levemente los tiempos de inferencia y entrenamiento, pero no de manera sustancial.

Como veremos de nuevo más adelante, sí que existe cierta incoherencia respecto de los tiempos de ejecución en los escenarios B5.2 y B5.4 (los que usan cuantización "bf16-mixed") así que sospechamos que podría haber algún error o ineficiencia en la aplicación que genera la máquina virtual donde se han realizado las simulaciones.

Si cambiamos el pruning por una cuantización "bf16-mixed", lo más notorio es cómo se dispara (presuntamente) el tiempo de ejecución, de 11 a 35 minutos. Creemos que debería reducirse incluso por debajo de 11 conociendo los efectos de la cuantización sobre modelos de red neuronal mejorando los tiempos de inferencia y optimizando las comunicaciones en la federación. Si nos fijamos en el accuracy en test en B5.2, observamos una clara bajada del rendimiento, ya sea comparándolo con el escenario base (B1.4) o con el pruning (B5.1). En este caso, el accuracy es de 0.45 aproximadamente en test, y la curva de aprendizaje se ve alterada en forma.

También es claro que al usar cuantización "64-true" (tanto de forma aislada como en combinación con pruning) el modelo experimenta una caída de rendimiento casi total, con accuracys en test en torno al valor de 0.1, además, la curva de aprendizaje en los modelos locales de estos escenarios es casi plana (B5.5) o ligeramente errática (B5.3).

En consecuencia de todo este análisis podemos concluir que la configuración que mantiene un mejor equilibrio entre accuracy, eficiencia y velocidad de convergencia es claramente el pruning, ya que además es el que mejor respeta la

forma de la curva de aprendizaje sobre el escenario base (los demás mecanismos solo la empeoran), y no hace empeorar el tiempo de ejecución.

PREGUNTA 2

¿La optimización incrementó significativamente la velocidad de entrenamiento (o redujo el tiempo por ronda) sin comprometer el accuracy?

RESPUESTA:

En cuanto a los tiempos de entrenamiento, el escenario baseline (B1.4) tardó 11 minutos de ejecución. Por lo que, al comparar este tiempo con los de los otros escenarios, podemos apreciar que en varios de ellos, de hecho, se incrementó este tiempo. Sin embargo, en otros, como B5.3 (5 minutos) y B5.6 (6 minutos) sí se incrementó la velocidad de entrenamiento. Estos dos escenarios usaron como método de optimización la cuantización con `precision="64-true"`, y en el caso de B5.6, este método se combinó además con pruning. Además, en B5.1 se usó también pruning, y el tiempo de ejecución fue de unos 11 minutos, muy similar al tiempo del baseline. Estas técnicas reducen la cantidad de operaciones en coma flotante y el tamaño de los modelos intermedios, lo que permite una ejecución más rápida sin comprometer significativamente la precisión, aunque en algunos casos, la diferencia en la velocidad de ejecución podría ser mínima.

En cuanto a los escenarios B5.2 y B5.4, ambos presentaron un tiempo de ejecución de 35 minutos, lo cual representa un aumento muy grande con respecto al baseline (B4.1, con 11 minutos). Este comportamiento podría deberse a algún fallo o ineficiencia en la implementación del método de cuantización con `precision="bf16-mixed"`. Suponemos que al ser una versión *CPU-friendly* de dicho método, podría dar problemas si el entorno no está preparado para gestionar esta versión específica. Dado que los experimentos fueron ejecutados en una máquina virtual, es muy probable que el entorno no contara con los recursos necesarios para ejecutar eficientemente las operaciones de cuantización, lo que habría obligado al sistema a emularlas o a realizar conversiones adicionales entre formatos, introduciendo así una sobrecarga considerable.